

QuantIQ

Master Reference Binder

Module 0

Master Index & Usage Guide

Version	18 May 2026
Project	QuantIQ — Algorithmic Trading on NSE
Repository	github.com/Quant-IQ/quantiq
Module Pages	~12
Print Format	A4, single-sided, spiral bind

0

Module Contents

§	Section	Page
1	About this Binder	3
2	How to Use this Binder	4
3	Module Map — All 11 Modules	5
4	Quick-Find Index	7
5	Convention Legend	9
6	Hard Constraints — Master List	10
7	Cross-Reference Matrix	11
8	Update Log & Versioning	12

1. About this Binder

This binder is the consolidated offline reference for every third-party tool, library, standard, and regulation used in the QuantIQ algorithmic trading project. It exists because trading-floor and exam-style work often needs fast, paper-based lookups without depending on internet access, search engines, or freshness of API docs that change between sessions.

Every module is self-contained. You can pull one module out of the binder, carry it to a co-working session, and return it later. The cover of each module declares its number, title, and the date its content was captured.

Why a printed binder?

- Survives Wi-Fi outages, Dhan API downtime, or campus VPN failures.
- Annotation-friendly — write your own margin notes during paper trading.
- Forces single-source-of-truth discipline. Code references this binder; this binder references official URLs; URLs are the original record.
- Reading code on screen tires the eye; reading printed docs while typing code on screen is the standard high-performance setup.

What it is not

- Not a replacement for the live docs. URLs at the bottom of every section point to the canonical source. When in doubt, check the URL — APIs change.
- Not legal advice. Regulatory sections (Module 9) summarise SEBI/NSE circulars for engineering use, not for compliance certification.
- Not a tutorial. Each module assumes you already know what the tool does; it gives you the reference surface, not the learning path.

2. How to Use this Binder

Three reading paths

Path A — Lookup

You know what you need (e.g. 'DhanHQ order types'). Open the Quick-Find Index (§4 of this module), find the term, jump to the module and page noted. Every module page has the module name in the footer for orientation.

Path B — Learn-a-domain

You want to understand a whole subsystem (e.g. backtesting). Pull Module 4 (vectorbt), read it front-to-back. Each module starts with a 1-page quick-reference card and ends with cross-references to dependent modules.

Path C — Compliance check

Before deploying anything that places live orders, open Module 9 (SEBI/NSE) and tick off the compliance checklist on its final page. Then verify your implementation against the Hard Constraints master list (§6 of this module).

Reading the modules

Every module follows the same internal structure:

- Cover page — module number, title, version date
- Module TOC — section list with page numbers
- Quick-reference card — 1–2 pages of cheat-sheet density
- Main content — sections ordered by importance, not by alphabet
- Constraint callouts — yellow boxes mark hard limits you must respect
- Code examples — monospace blocks with grey background
- See-also — cross-references to other modules in the binder

Annotation conventions

Recommended margin marks (use pencil; APIs change):

- + — verified working in QuantIQ codebase
- x — tried and failed (note date + Python version)
- !! — version-sensitive; recheck before next release
- -> M3§4 — cross-reference to Module 3, Section 4

3. Module Map — All 11 Modules

The binder is divided into 11 modules. Modules are bound separately and can be carried independently. The recommended reading sequence for new team members is: 0 -> 9 -> 1 -> 7 -> 2 -> 3 -> 4 -> 5 -> 6 -> 8 -> 10.

#	Module Title	Contents in Brief	Pages
0	Master Index & Usage Guide	This module. Navigation, conventions, hard-constraint master list.	12
1	DhanHQ v2 — Broker API	Auth, orders, portfolio, market quote, option chain, historical, Websocket, rate limits.	60-80
2	Python Data Stack	pandas, numpy, yfinance — quickstart and reference for data ingestion.	40-50
3	Technical Analysis — ta library	All indicators (SMA, EMA, RSI, MACD, ATR, Bollinger, ADX, etc.) — VWAP workaround.	25-35
4	Backtesting — vectorbt	Portfolio API, signals, indicator wrappers, plotting, statistics.	40-50
5	Dashboard — Streamlit	Core API, widgets, layouts, secrets management, deployment to Streamlit Community Cloud.	35-45
6	Charting — Plotly Python	Express, graph_objects, figure reference, financial chart patterns.	30-40
7	Python Standards	PEP 8, 257, 484, Google Style Guide (docstrings), stdlib logging.	30-40
8	Version Control & DevOps	Conventional Commits, SemVer, Git essentials, GitHub Actions, Branch protection.	20-30
9	Regulatory — SEBI & NSE	Feb 2025 retail-algo circular, Sept 2025 extension, NSE technical standards.	20-30
10	Environment & Tooling	python-dotenv, pip/venv, requirements pinning, .gitignore patterns.	10-15

4. Quick-Find Index

Alphabetical lookup of common terms. Module abbreviations: M0 = this module, M1 = DhanHQ, M2 = Python Data Stack, M3 = ta, M4 = vectorbt, M5 = Streamlit, M6 = Plotly, M7 = Python Standards, M8 = Version Control, M9 = Regulatory, M10 = Environment.

Access token (Dhan)	M1 §3 — Auth	Order rate limit (10/sec)	M9 §3 / M1 §10
ADX indicator	M3 §4 — Trend	pandas DataFrame	M2 §2
ATR indicator	M3 §4 — Volatility	PEP 8 (style)	M7 §2
Audit logging (5 yr retention)	M9 §3 — Operational	PEP 257 (docstrings)	M7 §3
Backtest setup	M4 §2 — Portfolio.from_signals	PEP 484 (type hints)	M7 §5
Bollinger Bands	M3 §4 — Volatility	Plotly figure structure	M6 §3
Branch protection rules	M8 §5 — GitHub	Plotly subplots	M6 §5
Charting basics (Plotly)	M6 §2 — Express	Portfolio metrics	M4 §5
Charting (vectorbt)	M4 §6 — Plotting	python-dotenv usage	M10 §2
Commit message format	M8 §2 — Conventional Commits	Rate limits (Dhan API)	M1 §10
Conventional Commits spec	M8 §2	requirements.txt pinning	M10 §4
CSV ingestion (pandas)	M2 §2 — pandas IO	RSI indicator	M3 §4 — Momentum
DataFrame basics	M2 §2 — pandas	SEBI Feb 2025 circular	M9 §2 — Full text
Dhan API rate limits	M1 §10 — Limits	Secrets (Streamlit)	M5 §6
Dhan order placement	M1 §4 — Orders	Semantic Versioning	M8 §3
Dhan WebSocket	M1 §9 — WebSocket	Session logout (daily)	M9 §3 — Operational
docstrings (Google style)	M7 §4 — Style Guide	Sharpe ratio	M4 §5 — Stats
EMA — exponential moving average	M3 §4 — Momentum	Signals (vectorbt)	M4 §3
Environment variables	M10 §2 — python-dotenv	SMA — simple moving average	M3 §4 — Trend
Equity curve plot	M4 §6 — Stats & plots	Static IP whitelisting	M9 §3 / M1 §3
GitHub Actions YAML	M8 §6	Streamlit widgets	M5 §3
Historical data (Dhan)	M1 §7 — Historical	Streamlit deployment	M5 §7
Indicator pipeline	M3 §3 — Usage patterns	ta library indicator list	M3 §4 — Full table
KPIs (Sharpe, drawdown)	M4 §5 — Stats	Type hints (PEP 484)	M7 §5
Logging config	M7 §6 — logging	vectorbt installation	M4 §1
MACD indicator	M3 §4 — Momentum	VWAP workaround (manual)	M3 §5 — Notes
Market quote API (Dhan)	M1 §5 — Quote	WebSocket (Dhan live feed)	M1 §9
Numpy arrays	M2 §3	yfinance NSE ticker (.NS)	M2 §4 — yfinance
Option chain (Dhan)	M1 §6		

5. Convention Legend

Typography

Bold — Defined term, hard constraint, or function name on first mention.

Italic — Emphasis or document title.

Monospace — Code, file names, paths, terminal commands, env variables.

Green text — Section headers and inline references to QuantIQ-specific decisions.

Grey text — Footnotes, captions, metadata, deprecated notes.

Callout boxes

Each callout type has a distinct label and colour:

HARD CONSTRAINT — Yellow box. Non-negotiable rule from API, SEBI, or NSE.

KNOWN ISSUE — Yellow box. Documented bug, gotcha, or surprising behaviour.

VERSION NOTE — Yellow box. Behaviour differs across library versions.

SEE ALSO — Plain inline reference at section end pointing to other modules.

Module colour coding

Every module uses the same QuantIQ-green primary colour for cover decoration and section headers. Modules can be visually distinguished by their cover page module number (large display digit). For colour-coded sticky tabs on the spiral binding, the team convention is:

Module	Topic	Suggested Tab Colour
M0	Index	White
M1	Broker API	Green
M2	Data Stack	Purple
M3	Indicators	Orange
M4	Backtest	Orange
M5	Dashboard	Pink
M6	Charts	Pink
M7	Standards	Blue
M8	DevOps	Blue
M9	Regulatory	Red
M10	Environment	Blue

6. Hard Constraints — Master List

Every rule on this page is non-negotiable in QuantIQ. Code that violates any item here is broken by definition. Verify against this list before every Phase 4 deployment.

API & broker

#	Constraint	Source
A1	Dhan access token expires every 24 hours. Bot must regenerate daily before 09:15 IST.	SEBI / M1 §3
A2	Order rate must never exceed 10 orders/sec/exchange/client.	SEBI Feb 2025 / M1 §10
A3	Static IP must be whitelisted before any live order. Use DhanLogin.set_ip().	SEBI / M1 §3
A4	No paper-trading sandbox in Dhan SDK. Use vectorbt for simulation.	M1 §3 / M4
A5	Pin dhanhq==2.1.x. v2.2.0 has breaking import changes.	Dhan releases / M1 §1

Data & libraries

#	Constraint	Source
D1	yfinance tickers for NSE must be suffixed .NS. Unsuffixed = US ticker.	M2 §4
D2	ta library has NO VWAP. Use manual cumulative formula.	M3 §5
D3	vectorbt: pin vectorbt==0.28.2. Never write >=1.0 (paid PRO product).	M4 §1
D4	Python version: pin >=3.11,<3.13 in pyproject.toml.	M10 §4
D5	python-dotenv >=1.2.2 requires Python >=3.10.	M10 §2

Security & credentials

#	Constraint	Source
S1	.streamlit/secrets.toml and .env must be in .gitignore.	M5 §6 / M10 §3
S2	No hardcoded API keys, client IDs, or tokens anywhere in repo.	M10 §2
S3	Streamlit secrets are pasted into Community Cloud UI, not committed.	M5 §7
S4	OAuth 2FA mandatory for Dhan login (per SEBI Feb 2025).	M9 §3

Operational (Phase 4 live trading)

#	Constraint	Source
O1	Daily session logout. No persistent session across trading days.	M9 §3
O2	Audit log retention >=5 years. Use TimedRotatingFileHandler.	M9 §3 / M7 §6
O3	All algo activity tagged with algo-ID for exchange reporting.	M9 §3
O4	Backtest before any strategy goes live. Use vectorbt Portfolio.	M4

7. Cross-Reference Matrix

Each module's dependencies on other modules. Read top-to-bottom: row module depends on column modules marked with ●.

	M1	M2	M3	M4	M5	M6	M7	M8	M9	M10
M1	—	■					●		●	●
M2		—					●			●
M3		●	—				●			
M4		●	●	—		■	●			
M5	●	●		■	—	●	●			●
M6		●		■	■	—	●			
M7							—			
M8							●	—		■
M9	●								—	
M10							●	■		—

● = strong dependency (read first) ■ = light reference (optional)

Read-order recommendations

New team member onboarding: M0 -> M9 -> M7 -> M8 -> M10 -> M1 -> M2 -> M3 -> M4 -> M5 -> M6

Quick start (already know Python): M0 -> M1 -> M9 -> M2 -> M4

Pre-deployment checklist: M9 -> M0 §6 (constraints) -> M1 §10 (rate limits) -> M10 §3 (secrets)

Dashboard build: M0 -> M5 -> M6 -> M4 (for data source)

8. Update Log & Versioning

Versioning scheme

Each module is dated independently. The binder as a whole is versioned by the latest module update date. Format: v.YYYY-MM-DD.

When to reprint a module

- Library major version bump (e.g. vectorbt 0.x -> 0.y).
- SEBI or NSE issues a new binding circular relevant to retail algo.
- Dhan API breaking change (check github.com/dhan-oss/DhanHQ-py/releases).
- Module accumulates more than 10 hand-annotated corrections.

Module version table

Module	Title	Captured	Source as of
M0	Master Index & Usage Guide	18 May 2026	—
M1	DhanHQ v2 — Broker API	18 May 2026	dhanhq.co/docs/v2/
M2	Python Data Stack	18 May 2026	pandas/numpy/yfinance docs
M3	Technical Analysis — ta	18 May 2026	ta.readthedocs.io
M4	Backtesting — vectorbt	18 May 2026	vectorbt.dev
M5	Dashboard — Streamlit	18 May 2026	docs.streamlit.io
M6	Charting — Plotly	18 May 2026	plotly.com/python/
M7	Python Standards	18 May 2026	peps.python.org
M8	Version Control & DevOps	18 May 2026	conventionalcommits.org / docs.github.com
M9	Regulatory — SEBI & NSE	18 May 2026	sebi.gov.in (Feb 2025 circular)
M10	Environment & Tooling	18 May 2026	pypi.org / packaging.python.org

Maintaining your binder

When a module is updated, print the new module, remove the old one from the spiral binder (cut the binding, replace, re-bind), and shred or archive the old version. Do not mix dated modules — always carry one consistent binder version per workstation. Tag the spine with the latest module date.

End of Module 0. Next: Module 1 — DhanHQ v2 Broker API.

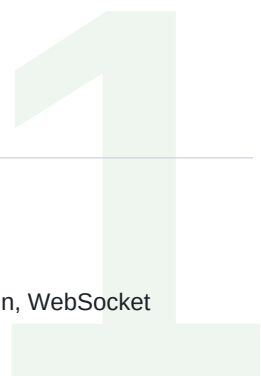
QuantIQ

Master Reference Binder

Module 1

DhanHQ v2 — Broker API

Version	18 May 2026
Source	dhanhq.co/docs/v2/ (DhanHQ v2.5)
Coverage	Auth, Orders, Portfolio, Market Data, Historical, Option Chain, WebSocket
Hard Constraints	10 orders/sec, 24-hour token, static IP mandatory
SDK	<code>pip install dhanhq</code> (pin: <code>dhanhq==2.1.x</code>)
Module Pages	approx 50



Module Contents

§	Section	Page
1	Quick Reference Card	3
2	Introduction & Getting Started	5
3	Authentication	8
4	Orders	15
5	Portfolio & Positions	25
6	Market Quote	29
7	Live Market Feed (WebSocket)	33
8	Historical Data	39
9	Option Chain	42
10	Funds & Margin	45
11	Annexure — Enums, Errors, Codes	47
12	Release Notes (v2 — v2.5)	52
13	Endpoints Not Covered Here	54

1. Quick Reference Card

Base URL

`https://api.dhan.co/v2/`

Headers (every request)

Header	Value
Content-Type	application/json
access-token	JWT from generateAccessToken
client-id	dhanClientId (Quote APIs only)

Rate Limits

Rate Limit	Order APIs	Data APIs	Quote APIs	Non Trading
per second	10	5	1	20
per minute	250	—	Unlimited	Unlimited
per hour	1,000	—	Unlimited	Unlimited
per day	7,000	100,000	Unlimited	Unlimited

Order Modifications: max 25 per order. Option Chain: 1 unique request per 3 seconds.

Python SDK quickstart

```
pip install dhanhq

from dhanhq import dhanhq
dhan = dhanhq("client_id", "access_token")

# Place market order
dhan.place_order(
    security_id="11536",
    exchange_segment=dhan.NSE,
    transaction_type=dhan.BUY,
    quantity=5,
    order_type=dhan.MARKET,
    product_type=dhan.INTRA,
    price=0
)
```

Endpoint cheat sheet

Method	Path	Purpose
POST	/orders	Place new order
PUT	/orders/{id}	Modify pending order
DELETE	/orders/{id}	Cancel pending order
GET	/orders	Get day's order book
GET	/trades	Get day's trade book
GET	/holdings	Demat holdings
GET	/positions	Open positions
DELETE	/positions	Exit all positions
GET	/fundlimit	Available funds
POST	/margincalculator	Pre-trade margin check
POST	/marketfeed/ltp	Up to 1000 LTPs
POST	/marketfeed/ohlc	Up to 1000 OHLC

Method	Path	Purpose
POST	/marketfeed/quote	Quote with depth
POST	/charts/historical	Daily candles
POST	/charts/intraday	Min candles (90d)
POST	/optionchain	Full option chain
POST	/optionchain/expirylist	Active expiries
GET	/profile	User profile / token validity

Hard constraints (Phase 4)

#	Constraint	Why
1	Access token expires every 24 hours.	SEBI Feb 2025 binding regulation. Regenerate daily.
2	Static IP whitelist mandatory for ALL order placement APIs.	Cannot be skipped. 7-day modification cooldown.
3	Order rate hard cap: 10 orders/sec.	>10/sec triggers algo-registration requirement with NSE.
4	Daily session logout — no carry-forward sessions.	Cannot persist sessions overnight.
5	Pin SDK: dhanhq==2.1.x.	Newer SDK has breaking import changes (DhanContext).

WebSocket endpoint

```
wss://api-feed.dhan.co?version=2&token={JWT}
&clientId={dhanClientId}&authType=2
```

Max 5 WebSocket connections per user.
Up to 5,000 instruments per connection.
Server pings every 10s; auto-disconnect after 40s of silence.

Common enums (full set in §11)

Field	Common Values
exchangeSegment	NSE_EQ, NSE_FNO, BSE_EQ, BSE_FNO, MCX_COMM, NSE_CURRENCY, BSE_CURRENCY, IDX_I
productType	CNC, INTRADAY, MARGIN, MTF, CO, BO
orderType	LIMIT, MARKET, STOP_LOSS, STOP_LOSS_MARKET
validity	DAY, IOC
transactionType	BUY, SELL
orderStatus	TRANSIT, PENDING, REJECTED, CANCELLED, PART_TRADED, TRADED, EXPIRED

Error code prefixes (full list in §11)

Code	Type
DH-901	Invalid auth (token/client ID expired)
DH-902	Invalid access (subscription/segment)
DH-904	Rate limit breach
DH-905	Input exception (missing fields)
DH-906	Order error
DH-908	Internal server error

2. Introduction & Getting Started

DhanHQ API is a REST-like set of endpoints for building trading and investment services on the NSE, BSE, and MCX exchanges. It accepts JSON or form-encoded requests, returns JSON-encoded responses, and uses standard HTTP response codes with access-token authentication.

Request structure (REST)

```
curl --request POST \
--url https://api.dhan.co/v2/{endpoint} \
--header 'Content-Type: application/json' \
--header 'access-token: JWT' \
--data '{Request JSON}'
```

All GET/DELETE parameters go as query parameters. POST/PUT parameters go as JSON body.

Python SDK install

```
pip install dhanhq

from dhanhq import dhanhq
dhan = dhanhq("client_id", "access_token")
```

This single package wraps every REST endpoint. The constructor stores credentials in the instance — every method call signs and sends automatically.

Error structure

All errors come back with consistent envelope:

```
{
  "errorType": "",
  "errorCode": "",
  "errorMessage": ""
}
```

Full error code list in §11 (Annexure).

Rate limits

These are HARD limits. Exceeding them returns DH-904 and may trigger temporary blocking.

Window	Order APIs	Data APIs	Quote APIs	Non Trading
per second	10	5	1	20
per minute	250	—	Unlimited	Unlimited
per hour	1,000	—	Unlimited	Unlimited
per day	7,000	100,000	Unlimited	Unlimited

Order Modifications cap: 25 modifications per order. After this, modification requests are rejected.

API categories

Category	Endpoints
Trading APIs	Orders, Super Order, Forever Order, Conditional Trigger, Portfolio & Positions, EDIS, Trader's Control, Funds & Margin, Statement, Postback, Live Order Update
Data APIs	Market Quote (LTP/OHLC/Quote), Live Market Feed (WebSocket), Full Market Depth, Historical Data, Expired Options Data, Option Chain
Reference	Annexure (enums, errors), Instrument List, Release Notes

Trading APIs are free for all Dhan users. Data APIs require a separate subscription (charges listed on dhan.co).

3. Authentication

Two user categories: Individual (DIY traders) and Partners (platforms serving end users). Two access methods for individuals: (a) directly generate access token from Dhan Web, (b) OAuth-style API key + secret flow for one-year key validity.

3.1 Access Token (24-hour direct method)

Manual generation steps:

- Login to web.dhan.co
- Click 'My Profile' → 'Access DhanHQ APIs'
- Generate Access Token with 24-hour validity
- Optionally provide a Postback URL for order updates

3.2 Generate Token via API (requires TOTP setup)

POST <https://auth.dhan.co/app/generateAccessToken>

```
curl --location --request POST \
'https://auth.dhan.co/app/generateAccessToken
?dhanClientId=1000000001&pin=11111&totp=000000'
```

Query parameters

Field	Description
dhanClientId	Client ID of the user
pin	Dhan PIN — 6-digit numeric
totp	TOTP code from authenticator app

Response

```
{
  "dhanClientId": "1000000401",
  "dhanClientName": "JOHN DOE",
  "dhanClientUcc": "ABCD12345E",
  "givenPowerOfAttorney": false,
  "accessToken": "eyJ...",
  "expiryTime": "2026-01-01T00:00:00.000"
}
```

3.3 Renew Token

GET <https://api.dhan.co/v2/RenewToken>

Refreshes an ACTIVE token for another 24 hours. Replaces the old token. Cannot renew expired tokens — those require fresh generation. Valid only for tokens generated from Dhan Web.

```
curl --location 'https://api.dhan.co/v2/RenewToken' \
--header 'access-token: {JWT}' \
--header 'dhanClientId: {Client ID}'
```

3.4 API Key + Secret (OAuth-style, 12-month validity)

For users wanting a longer-lived credential. Three-step flow:

Step 1 — Generate Consent

POST <https://auth.dhan.co/app/generate-consent>

```
curl --request POST \
'https://auth.dhan.co/app/generate-consent?client_id={dhanClientId}' \
--header 'app_id: {API key}' \
--header 'app_secret: {API secret}'
```

Returns consentAppId. Maximum 25 consent IDs per day.

Step 2 — Browser Login

```
https://auth.dhan.co/login/consentApp-login?consentAppId={...}
```

Open in browser. User enters credentials and 2FA. Response is a 302 redirect to your redirect_URL with ?tokenId={...} appended.

Step 3 — Consume Consent

POST <https://auth.dhan.co/app/consumeApp-consent>

```
curl --location \
'https://auth.dhan.co/app/consumeApp-consent?tokenId={Token ID}' \
--header 'app_id: {API Key}' \
--header 'app_secret: {API Secret}'
```

Response includes the actual accessToken for use in all subsequent API calls. Token validity is 24 hours regardless of API key validity (which is 12 months).

3.5 Static IP Whitelisting (MANDATORY)

As of SEBI Feb 2025, static IP whitelisting is mandatory for ALL Order Placement APIs (Orders, Super Order, Forever Order). Not required for order/trade GET APIs. Each user needs one or two unique IPs — Primary and Secondary. Once whitelisted, IPs cannot be modified for 7 days.

Set IP

POST <https://api.dhan.co/v2/ip/setIP>

```
{
  "dhanClientId": "1000000001",
  "ip": "10.200.10.10",
  "ipFlag": "PRIMARY"
}
```

ipFlag must be PRIMARY or SECONDARY. Both IPv4 and IPv6 are accepted.

Modify IP

PUT <https://api.dhan.co/v2/ip/modifyIP>

Same body as Set IP. Only allowed after 7-day cooldown window.

Get IP

GET <https://api.dhan.co/v2/ip/getIP>

```
{
  "modifyDateSecondary": "2025-09-30",
  "secondaryIP": "10.420.43.12",
  "modifyDatePrimary": "2025-09-28",
  "primaryIP": "10.420.29.14"
}
```

Use this endpoint daily on bot startup to verify your IP whitelist hasn't drifted (ISP issues, VPN changes).

3.6 TOTP Setup

Time-based One-Time Password (RFC 6238). Used as a 2FA option for API authentication. Generates a 6-digit code from a shared secret + current time. Code rotates every 30 seconds.

Setup procedure:

1. Go to Dhan Web → DhanHQ Trading APIs section

2. Select Setup TOTP
3. Confirm with OTP on mobile/email
4. Scan the QR code via authenticator app, or enter the displayed code
5. Confirm by entering the first generated TOTP

Once setup, TOTP becomes available as a 2FA option during login on partner platforms and within the API key authentication flow.

3.7 User Profile (token validity check)

GET <https://api.dhan.co/v2/profile>

Lightweight endpoint to verify access token and account setup. Recommended first call on bot startup.

```
{
  "dhanClientId": "1100003626",
  "tokenValidity": "30/03/2025 15:37",
  "activeSegment": "Equity, Derivative, Currency, Commodity",
  "ddpi": "Active",
  "mtf": "Active",
  "dataPlan": "Active",
  "dataValidity": "2024-12-05 09:37:52.0"
}
```

Field reference

Field	Type	Description
dhanClientId	string	User identifier
tokenValidity	string	Token expiry date/time
activeSegment	string	Comma-list of active exchange segments
ddpi	enum	DDPI status: Active or Deactive
mtf	enum	MTF consent: Active or Deactive
dataPlan	enum	Data API subscription: Active or Deactive
dataValidity	string	Data subscription expiry

4. Orders

The Order Management API handles placement, modification, cancellation, slicing, and retrieval. All order placement APIs require Static IP whitelisting (see §3.5).

Endpoint summary

Method	Path	Purpose
POST	/orders	Place new order
PUT	/orders/{order-id}	Modify pending order
DELETE	/orders/{order-id}	Cancel pending order
POST	/orders/slicing	Slice over freeze-limit F&O;
GET	/orders	Order book (today)
GET	/orders/{order-id}	Order status by ID
GET	/orders/external/{correlation-id}	Order status by correlation ID
GET	/trades	Trade book (today)
GET	/trades/{order-id}	Trades for an order

4.1 Place Order

POST /orders

```
{
  "dhanClientId": "1000000003",
  "correlationId": "123abc678",
  "transactionType": "BUY",
  "exchangeSegment": "NSE_EQ",
  "productType": "INTRADAY",
  "orderType": "MARKET",
  "validity": "DAY",
  "securityId": "11536",
  "quantity": "5",
  "disclosedQuantity": "",
  "price": "",
  "triggerPrice": "",
  "afterMarketOrder": false,
  "amoTime": "",
  "boProfitValue": "",
  "boStopLossValue": ""
}
```

Required parameters

Field	Type	Notes
dhanClientId	string	User identifier (required)
transactionType	enum	BUY or SELL
exchangeSegment	enum	NSE_EQ, NSE_FNO, BSE_EQ, etc. (see §11)
productType	enum	CNC, INTRADAY, MARGIN, MTF, CO, BO
orderType	enum	LIMIT, MARKET, STOP_LOSS, STOP_LOSS_MARKET
validity	enum	DAY or IOC
quantity	int	Number of shares
price	float	Order price (0 for MARKET)

Optional / conditional parameters

Field	Type	Notes
correlationId	string	Your tag, max 30 chars, alphanumeric+space+_-
securityId	string	Exchange-standard scrip ID (see Instrument List)
disclosedQuantity	int	Iceberg visible qty (must be $\geq 30\%$ of total)
triggerPrice	float	Required for STOP_LOSS / STOP_LOSS_MARKET
afterMarketOrder	bool	True for AMO
amoTime	enum	PRE_OPEN, OPEN, OPEN_30, OPEN_60
boProfitValue	float	Bracket Order target offset
boStopLossValue	float	Bracket Order stop loss offset

Response

```
{
  "orderId": "112111182198",
  "orderStatus": "PENDING"
}
```

orderStatus values: TRANSIT, PENDING, REJECTED, CANCELLED, TRADED, EXPIRED.
PART_TRADED appears in order book responses after partial fills.

4.2 Modify Order

PUT /orders/{order-id}

Modifiable fields: price, quantity, order type, validity. For Bracket and Cover Orders, you must specify the leg (ENTRY_LEG, TARGET_LEG, STOP_LOSS_LEG).

```
{
  "dhanClientId": "1000000009",
  "orderId": "112111182045",
  "orderType": "LIMIT",
  "legName": "",
  "quantity": "40",
  "price": "3345.8",
  "disclosedQuantity": "10",
  "triggerPrice": "",
  "validity": "DAY"
}
```

Important (v2 breaking change): the quantity field must contain the TOTAL order quantity, not the pending quantity. Earlier versions expected pending quantity which caused race conditions during partial fills.

Maximum 25 modifications per order. Beyond that, the request is rejected.

4.3 Cancel Order

DELETE /orders/{order-id}

No request body. Returns 202 Accepted on success.

```
{
  "orderId": "112111182045",
  "orderStatus": "CANCELLED"
}
```

4.4 Order Slicing (F&O; over freeze limit)

POST /orders/slicing

Use this endpoint when an F&O; order quantity exceeds the exchange freeze limit. Dhan auto-slices into multiple child orders to fit within limits. Request structure is identical to /orders.

4.5 Order Book

GET /orders

Returns array of all orders placed today. No request body. Response is a list of order objects with the full schema below.

Common response fields

Field	Type	Description
orderId	string	Dhan-generated order ID
correlationId	string	Your tag
orderStatus	enum	TRANSIT/PENDING/REJECTED/CANCELLED/PART_TRADED/TRADED/EXPIRED
transactionType	enum	BUY or SELL
exchangeSegment	enum	Exchange segment
productType	enum	CNC/INTRADAY/MARGIN/MTF/CO/BO
orderType	enum	LIMIT/MARKET/STOP_LOSS/STOP_LOSS_MARKET
validity	enum	DAY or IOC
tradingSymbol	string	Symbol (see Instrument List)
securityId	string	Exchange security ID
quantity	int	Order quantity
disclosedQuantity	int	Visible quantity
price	float	Order price
triggerPrice	float	Trigger for SL/SL-M, CO, BO
afterMarketOrder	bool	Whether AMO
boProfitValue	float	BO target offset
boStopLossValue	float	BO stop-loss offset
legName	enum	BO/CO leg: ENTRY_LEG/TARGET_LEG/STOP_LOSS_LEG
createTime	string	Order creation timestamp
updateTime	string	Last update timestamp
exchangeTime	string	Exchange-side receipt time
drvExpiryDate	string	F&O; expiry
drvOptionType	enum	CALL or PUT
drvStrikePrice	float	Option strike
omsErrorCode	string	Error code if rejected
omsErrorDescription	string	Error message if rejected
algId	string	Exchange algo-ID for Dhan
remainingQuantity	int	Pending qty (quantity – filledQty)
averageTradedPrice	float	VWAP of fills
filledQty	int	Quantity filled

4.6 Get Order by Order ID

GET /orders/{order-id}

Same response object as a single entry in the Order Book.

4.7 Get Order by Correlation ID

GET /orders/external/{correlation-id}

Fallback when you've lost the orderId but kept your correlationId. Useful for crash-recovery in bots that track their own correlations.

4.8 Trade Book

GET /trades

Returns all executed trades for the day. Trade-level granularity (an order with multiple partial fills produces multiple trades).

Trade response fields

Field	Type	Description
orderId	string	Dhan order ID
exchangeOrderId	string	Exchange order ID
exchangeTradeId	string	Exchange trade ID
transactionType	enum	BUY or SELL
exchangeSegment	enum	Exchange segment
productType	enum	Trade product type
orderType	enum	Order type for this trade
tradingSymbol	string	Symbol
securityId	string	Security ID
tradedQuantity	int	Shares executed in this fill
tradedPrice	float	Fill price
createTime	string	Trade creation timestamp
updateTime	string	Last update
exchangeTime	string	Exchange fill time
drvExpiryDate	string	F&O; expiry
drvOptionType	enum	CALL or PUT
drvStrikePrice	float	Strike

4.9 Trades of an Order

GET /trades/{order-id}

Returns all individual trades produced by a single order. Essential for reconciling partial fills on BO and CO orders.

```
# Python SDK
dhan.get_trade_book(order_id)
```


5. Portfolio & Positions

Method	Path	Purpose
GET	/holdings	Demat holdings
GET	/positions	Open positions
POST	/positions/convert	Convert intraday ↔ delivery
DELETE	/positions	Exit all positions + cancel open orders

5.1 Holdings

GET /holdings

Returns all holdings bought or sold in previous trading sessions. Includes T1 (settling) and delivered quantities.

```
[
{
  "exchange": "ALL",
  "tradingSymbol": "HDFC",
  "securityId": "1330",
  "isin": "INE001A01036",
  "totalQty": 1000,
  "dpQty": 1000,
  "t1Qty": 0,
  "availableQty": 1000,
  "collateralQty": 0,
  "avgCostPrice": 2655.0
}
```

Field	Type	Description
exchange	enum	Source exchange
tradingSymbol	string	Symbol
securityId	string	Exchange ID
isin	string	ISIN (universal ID)
totalQty	int	Total quantity held
dpQty	int	Quantity in demat
t1Qty	int	Quantity pending settlement
availableQty	int	Quantity free for trading
collateralQty	int	Quantity placed as collateral
avgCostPrice	float	VWAP of buys

5.2 Positions

GET /positions

Returns all open positions for the current day, including F&O; carry-forward.

Field	Type	Description
tradingSymbol	string	Symbol
securityId	string	Exchange ID
positionType	enum	LONG, SHORT, CLOSED
exchangeSegment	enum	Exchange segment
productType	enum	CNC/INTRADAY/MARGIN/MTF/CO/BO

Field	Type	Description
buyAvg	float	Mark-to-market buy VWAP
buyQty	int	Total bought
costPrice	float	Actual cost
sellAvg	float	Mark-to-market sell VWAP
sellQty	int	Total sold
netQty	int	buyQty – sellQty
realizedProfit	float	Booked P&L;
unrealizedProfit	float	Open position P&L;
rbiReferenceRate	float	RBI forex reference rate
multiplier	int	Currency F&O; multiplier
carryForwardBuyQty	int	F&O; carry-forward long qty
carryForwardSellQty	int	F&O; carry-forward short qty
carryForwardBuyValue	float	Carry-forward long value
carryForwardSellValue	float	Carry-forward short value
dayBuyQty	int	Today's buy qty
daySellQty	int	Today's sell qty
dayBuyValue	float	Today's buy value
daySellValue	float	Today's sell value
drvExpiryDate	string	F&O; expiry
drvOptionType	enum	CALL or PUT
drvStrikePrice	float	Strike
crossCurrency	bool	Non-INR pair flag

5.3 Convert Position

POST /positions/convert

Convert open position from intraday → delivery or delivery → intraday. Useful for adapting positions across product types during the trading day.

```
{
  "dhanClientId": "1000000009",
  "fromProductType": "INTRADAY",
  "exchangeSegment": "NSE_EQ",
  "positionType": "LONG",
  "securityId": "11536",
  "tradingSymbol": "",
  "convertQty": "40",
  "toProductType": "CNC"
}
```

Returns 202 Accepted on success.

5.4 Exit All Positions (Kill Switch)

DELETE /positions

Exits all open positions AND cancels all open orders for the current trading day. Single nuclear button. Use in panic / circuit-breaker scenarios.

```
{
  "status": "SUCCESS",
  "message": "All orders and positions exited successfully"
}
```

6. Market Quote

Snapshot APIs for live market data. Three modes: LTP only, OHLC + LTP, or full Quote with market depth and OI. Up to 1,000 instruments per request, 1 request/second.

Method	Path	Purpose
POST	/marketfeed/ltp	LTP only (lightest)
POST	/marketfeed/ohlc	LTP + day OHLC
POST	/marketfeed/quote	Full quote with depth + OI

Headers (all three)

Header	Description
access-token	JWT access token
client-id	dhanClientId (required for Quote APIs)

Request body format (common)

```
{
  "NSE_EQ": [11536],
  "NSE_FNO": [49081, 49082]
}
```

Keys are Exchange Segment enums (see §11). Values are arrays of Security IDs. Up to 1,000 instruments total per request.

6.1 Ticker Data (LTP)

POST /marketfeed/ltp

Response

```
{
  "data": {
    "NSE_EQ": {
      "11536": {"last_price": 4520}
    },
    "NSE_FNO": {
      "49081": {"last_price": 368.15},
      "49082": {"last_price": 694.35}
    }
  },
  "status": "success"
}
```

6.2 OHLC Data

POST /marketfeed/ohlc

Returns LTP + day's open/high/low/close per instrument.

```
{
  "data": {
    "NSE_EQ": {
      "11536": {
        "last_price": 4525.55,
        "ohlc": {
          "open": 4521.45,
          "close": 4507.85,
          "high": 4530,
          "low": 4500
        }
      }
    }
  }
}
```

```
},  
"status": "success"  
}
```

6.3 Market Depth (Full Quote)

POST `/marketfeed/quote`

Returns full quote: LTP, OHLC, OI, volume, 5-level depth, circuit limits, last-trade time, average price, buy/sell qty.

Full quote response fields

Field	Type	Description
last_price	float	LTP
last_quantity	int	Last traded quantity
last_trade_time	string	Last trade timestamp
average_price	float	VWAP for the day
volume	int	Total day volume
net_change	float	Change vs previous close
buy_quantity	int	Total pending buy quantity
sell_quantity	int	Total pending sell quantity
upper_circuit_limit	float	Today's upper circuit
lower_circuit_limit	float	Today's lower circuit
ohlc.open	float	Day open
ohlc.close	float	Day close (post market only)
ohlc.high	float	Day high
ohlc.low	float	Day low
oi	int	Open Interest (derivatives only)
oi_day_high	int	Highest OI today (NSE_FNO only)
oi_day_low	int	Lowest OI today (NSE_FNO only)
depth.buy[i].price	float	Bid price at depth level i
depth.buy[i].quantity	int	Bid qty at level i
depth.buy[i].orders	int	Number of buy orders at level i
depth.sell[i].price	float	Ask price at depth level i
depth.sell[i].quantity	int	Ask qty at level i
depth.sell[i].orders	int	Number of sell orders at level i

Depth contains 5 levels for buy and sell. For 20-level or 200-level depth, see `/full-market-depth` WebSocket (not covered in this binder; see Dhan v2 docs).

7. Live Market Feed (WebSocket)

Real-time tick-by-tick market data via WebSocket. Request messages are JSON, response messages are binary. Use the DhanHQ Python library to avoid implementing the binary parser yourself.

Connection limits

- Up to 5 WebSocket connections per user.
- Up to 5,000 instruments per connection.
- Up to 100 instruments per single subscribe message (send multiple).
- Server pings every 10 seconds; library auto-pongs.
- Connection drops if pong silence exceeds 40 seconds.
- Establishing a 6th connection disconnects the oldest with code 805.

7.1 Connect

```
wss://api-feed.dhan.co
?version=2
&token={JWT}
&clientId={dhanClientId}
&authType=2
```

Param	Value / Description
version	2 (for DhanHQ v2)
token	Access Token JWT
clientId	User Dhan Client ID
authType	2 (default)

7.2 Subscribe to Instruments

Send JSON over the open connection:

```
{
  "RequestCode": 15,
  "InstrumentCount": 2,
  "InstrumentList": [
    {"ExchangeSegment": "NSE_EQ", "SecurityId": "1333"},
    {"ExchangeSegment": "BSE_EQ", "SecurityId": "532540"}
  ]
}
```

Request code reference (see §11 for full list)

Code	Action
11	Connect Feed
12	Disconnect Feed
15 / 16	Subscribe / Unsubscribe — Ticker Packet
17 / 18	Subscribe / Unsubscribe — Quote Packet
21 / 22	Subscribe / Unsubscribe — Full Packet
23 / 24	Subscribe / Unsubscribe — Full Market Depth

7.3 Response Format

All responses are binary in **Little Endian**. Convert byte order if your system is Big Endian. Every response begins with an 8-byte Response Header:

Bytes	Type	Size	Description
1	byte	1	Feed Response Code (see §11)
2–3	int16	2	Total message length
4	byte	1	Exchange Segment
5–8	int32	4	Security ID

Response codes (full set in §11):

Code	Packet Type
1	Index
2	Ticker
4	Quote
5	OI
6	Prev Close
7	Market Status
8	Full
50	Feed Disconnect

7.4 Ticker Packet (code 2)

Smallest packet — just LTP + last trade time.

Bytes	Type	Size	Description
0–8	header	8	Response header with code 2
9–12	float32	4	Last Traded Price
13–16	int32	4	Last Trade Time (EPOCH)

Prev Close (code 6) — auto-sent with any subscription

Bytes	Type	Size	Description
0–8	header	8	Response header with code 6
9–12	float32	4	Previous close price
13–16	int32	4	Previous OI

7.5 Quote Packet (code 4)

Mid-size packet — LTP, LTQ, ATP, volume, buy/sell qty, OHLC.

Bytes	Type	Size	Description
0–8	header	8	Response header code 4
9–12	float32	4	Last Traded Price
13–14	int16	2	Last Traded Quantity
15–18	int32	4	Last Trade Time (EPOCH)
19–22	float32	4	Average Trade Price
23–26	int32	4	Volume
27–30	int32	4	Total Sell Quantity
31–34	int32	4	Total Buy Quantity
35–38	float32	4	Day Open

Bytes	Type	Size	Description
39–42	float32	4	Day Close (post-market only)
43–46	float32	4	Day High
47–50	float32	4	Day Low

OI sub-packet (code 5) — auto-sent for derivatives

Bytes	Type	Size	Description
0–8	header	8	Response header code 5
9–12	int32	4	Open Interest

7.6 Full Packet (code 8)

Largest packet — Quote + OI + Market Depth (5 levels) in one 162-byte payload.

Bytes	Type	Size	Description
0–8	header	8	Response header code 8
9–12	float32	4	Last Traded Price
13–14	int16	2	Last Traded Quantity
15–18	int32	4	Last Trade Time (EPOCH)
19–22	float32	4	Average Trade Price
23–26	int32	4	Volume
27–30	int32	4	Total Sell Quantity
31–34	int32	4	Total Buy Quantity
35–38	int32	4	Open Interest (derivatives)
39–42	int32	4	OI day high (NSE_FNO only)
43–46	int32	4	OI day low (NSE_FNO only)
47–50	float32	4	Day Open
51–54	float32	4	Day Close (post-market)
55–58	float32	4	Day High
59–62	float32	4	Day Low
63–162	depth	100	5 × 20-byte depth packets

Each depth packet (20 bytes)

Bytes	Type	Size	Description
1–4	int32	4	Bid Quantity
5–8	int32	4	Ask Quantity
9–10	int16	2	Number of Bid Orders
11–12	int16	2	Number of Ask Orders
13–16	float32	4	Bid Price
17–20	float32	4	Ask Price

7.7 Disconnect

Send JSON message:

```
{"RequestCode": 12}
```

Server-side disconnects (code 50) carry a 2-byte disconnect reason code in bytes 9–10. Common codes (full list in §11):

Code	Reason
800	Internal server error
805	Too many connections (6th opened — oldest disconnected)
807	Access token expired
808	Authentication failed

8. Historical Data

Historical OHLC + Volume candles. Two endpoints — daily and intraday (1, 5, 15, 25, 60 min). Daily data is available since instrument inception; intraday is available for last 5 years, but only 90 days can be polled per request.

Method	Path	Purpose
POST	/charts/historical	Daily candles (full history)
POST	/charts/intraday	1/5/15/25/60-min candles (5yr, 90d/call)

8.1 Daily Historical Data

POST /charts/historical

Request

```
{
  "securityId": "1333",
  "exchangeSegment": "NSE_EQ",
  "instrument": "EQUITY",
  "expiryCode": 0,
  "oi": false,
  "fromDate": "2022-01-08",
  "toDate": "2022-02-08"
}
```

Field	Type	Required	Description
securityId	string	Yes	Exchange security ID
exchangeSegment	enum	Yes	Exchange segment (see §11)
instrument	enum	Yes	INDEX/EQUITY/FUTIDX/FUTSTK/OPTIDX/OPTSTK/etc.
expiryCode	int	Optional	Derivatives expiry: 0=current, 1=next, 2=far
oi	bool	Optional	Include OI data (F&O; only)
fromDate	string	Yes	Range start (YYYY-MM-DD)
toDate	string	Yes	Range end (exclusive)

Response (column-oriented, parallel arrays)

```
{
  "open": [3978, 3856, 3925, ...],
  "high": [3978, 3925, 3929, ...],
  "low": [3861, 3856, 3836.55, ...],
  "close": [3879.85, 3915.9, ...],
  "volume": [3937092, 1906106, ...],
  "timestamp": [1326220200, 1326306600, ...],
  "open_interest": [0, 0, ...]
}
```

Note: response is parallel arrays, NOT array of candles. Use pandas to zip into DataFrame:

```
import pandas as pd
df = pd.DataFrame(response)
df['timestamp'] = pd.to_datetime(df['timestamp'], unit='s')
df = df.set_index('timestamp')
```

8.2 Intraday Historical Data

POST /charts/intraday

Request

```
{
  "securityId": "1333",
  "exchangeSegment": "NSE_EQ",
  "instrument": "EQUITY",
  "interval": "1",
  "oi": false,
  "fromDate": "2024-09-11 09:30:00",
  "toDate": "2024-09-15 13:00:00"
}
```

Field	Type	Required	Description
securityId	string	Yes	Exchange security ID
exchangeSegment	enum	Yes	Exchange segment
instrument	enum	Yes	EQUITY/FUTIDX/OPTIDX/etc.
interval	enum	Yes	Minute interval: 1, 5, 15, 25, 60
oi	bool	Optional	Include OI (F&O; only)
fromDate	string	Yes	Range start: YYYY-MM-DD HH:MM:SS (IST)
toDate	string	Yes	Range end

Limits: max 90 days of data per request. The API recommends storing this data locally for analytics rather than refetching.

Response format is identical to Daily Historical — column-oriented parallel arrays.

9. Option Chain

Full option chain in one API call. Returns OI, Greeks, IV, volume, top bid/ask, and price across all strikes for any underlying. Works for NSE, BSE, and MCX options. Rate limit: 1 unique request per 3 seconds (multiple distinct queries can be concurrent).

Method	Path	Purpose
POST	/optionchain	Full option chain for an underlying + expiry
POST	/optionchain/expirylist	Active expiries for an underlying

9.1 Option Chain

POST /optionchain

Request

```
{
  "UnderlyingScrip": 13,
  "UnderlyingSeg": "IDX_I",
  "Expiry": "2024-10-31"
}
```

Field	Type	Description
UnderlyingScrip	int	Security ID of underlying (required)
UnderlyingSeg	enum	Exchange segment of underlying
Expiry	string	Expiry date (YYYY-MM-DD). Get from /expirylist

Response shape

```
{
  "data": {
    "last_price": 25642.8, // underlying spot
    "oc": {
      "25650.000000": { // strike key
        "ce": { ... call data ... },
        "pe": { ... put data ... }
      },
      "25700.000000": { ... }
    }
  },
  "status": "success"
}
```

Per-strike Call/Put data fields

Field	Type	Description
average_price	float	Day VWAP
last_price	float	LTP
volume	int	Today's volume
oi	int	Open Interest
security_id	int	Option contract security ID
implied_volatility	float	IV (%)
top_bid_price	float	Best bid
top_bid_quantity	int	Bid qty at top
top_ask_price	float	Best ask
top_ask_quantity	int	Ask qty at top
previous_close_price	float	Yesterday close

Field	Type	Description
previous_oi	int	Yesterday OI
previous_volume	int	Yesterday volume
greeks.delta	float	Δ — premium change per $\blacksquare$ 1 in spot
greeks.gamma	float	Γ — Δ change rate
greeks.theta	float	Θ — time decay
greeks.vega	float	v — IV sensitivity

9.2 Expiry List

POST /optionchain/expirylist

```
{
  "UnderlyingScrip": 13,
  "UnderlyingSeg": "IDX_I"
}
```

Response

```
{
  "data": [
    "2024-10-17", "2024-10-24", "2024-10-31",
    "2024-11-07", "2024-11-14", "2024-11-28",
    "2024-12-26", "2025-03-27", ...
  ],
  "status": "success"
}
```

All expiry dates returned in YYYY-MM-DD format. Use these values as the Expiry parameter in /optionchain calls.

10. Funds & Margin

Method	Path	Purpose
POST	/margincalculator	Margin / brokerage for an order
POST	/margincalculator/multi	Margin for multiple orders
GET	/fundlimit	Available funds and balances

10.1 Single-order Margin Calculator

POST /margincalculator

Pre-trade check for required margin, brokerage, and leverage.

```
{
  "dhanClientId": "1000000132",
  "exchangeSegment": "NSE_EQ",
  "transactionType": "BUY",
  "quantity": 5,
  "productType": "CNC",
  "securityId": "1333",
  "price": 1428,
  "triggerPrice": 1427
}
```

Response

```
{
  "totalMargin": 2800.00,
  "spanMargin": 1200.00,
  "exposureMargin": 1003.00,
  "availableBalance": 10500.00,
  "variableMargin": 1000.00,
  "insufficientBalance": 0.00,
  "brokerage": 20.00,
  "leverage": "4.00"
}
```

10.2 Multi-order Margin Calculator

POST /margincalculator/multi

Computes total margin requirement across multiple scrips at once. Supports including existing positions and open orders for accurate net-margin calculation.

```
{
  "includePosition": true,
  "includeOrders": true,
  "scripts": [
    {
      "exchangeSegment": "NSE_EQ",
      "transactionType": "BUY",
      "quantity": 100,
      "productType": "CNC",
      "securityId": "12345",
      "price": 250.50
    }
  ]
}
```

Response includes breakdown: total_margin, span_margin, exposure_margin, equity_margin, fo_margin, commodity_margin, hedge_benefit. Margins are indicative and valid only for the current trading session.

10.3 Fund Limit

GET /fundlimit

Account balance and margin utilisation snapshot.

Field	Type	Description
availabelBalance	float	Free trading balance (note Dhan typo: 'availabel')
sodLimit	float	Start-of-day balance
collateralAmount	float	Margin from pledged collateral
receiveableAmount	float	Amount due from sold deliveries
utilizedAmount	float	Margin used today
blockedPayoutAmount	float	Blocked for payout requests
withdrawableBalance	float	Free balance for bank withdrawal

11. Annexure — Enums, Errors, Codes

Complete enum and code reference. Every API parameter that accepts an enum string maps to one of the tables in this section.

11.1 Exchange Segment

Attribute	Exchange	Segment	Enum
IDX_I	Index	Index Value	0
NSE_EQ	NSE	Equity Cash	1
NSE_FNO	NSE	Futures & Options	2
NSE_CURRENCY	NSE	Currency	3
BSE_EQ	BSE	Equity Cash	4
MCX_COMM	MCX	Commodity	5
BSE_CURRENCY	BSE	Currency	7
BSE_FNO	BSE	Futures & Options	8

11.2 Product Type

CO and BO are valid only for Intraday.

Attribute	Detail
CNC	Cash & Carry for equity deliveries
INTRADAY	Intraday for Equity, Futures, Options
MARGIN	Carry-forward in Futures & Options
MTF	Margin Trading Facility
CO	Cover Order (intraday with mandatory SL)
BO	Bracket Order (intraday with target + SL)

11.3 Order Status

Attribute	Detail
TRANSIT	Did not reach the exchange server
PENDING	Awaiting execution
CLOSED	Super Order — both entry and exit completed
TRIGGERED	Super Order — target or SL triggered
REJECTED	Rejected by broker or exchange
CANCELLED	Cancelled by user
PART_TRADED	Partial quantity traded successfully
TRADED	Executed in full
EXPIRED	Validity elapsed

11.4 After-Market Order Timing

Attribute	Detail
PRE_OPEN	AMO pumped at pre-market session
OPEN	AMO pumped at market open

Attribute	Detail
OPEN_30	AMO pumped 30 minutes after open
OPEN_60	AMO pumped 60 minutes after open

11.5 Expiry Code (derivatives)

Code	Detail
0	Current / Near Expiry
1	Next Expiry
2	Far Expiry

11.6 Instrument Type

Attribute	Detail
INDEX	Index
FUTIDX	Futures of Index
OPTIDX	Options of Index
EQUITY	Equity
FUTSTK	Futures of Stock
OPTSTK	Options of Stock
FUTCOM	Futures of Commodity
OPTFUT	Options of Commodity Futures
FUTCUR	Futures of Currency
OPTCUR	Options of Currency

11.7 Feed Request Codes (WebSocket)

Code	Action
11	Connect Feed
12	Disconnect Feed
15	Subscribe — Ticker
16	Unsubscribe — Ticker
17	Subscribe — Quote
18	Unsubscribe — Quote
21	Subscribe — Full Packet
22	Unsubscribe — Full Packet
23	Subscribe — Full Market Depth
24	Unsubscribe — Full Market Depth

11.8 Feed Response Codes (WebSocket)

Code	Packet
1	Index Packet
2	Ticker Packet
4	Quote Packet
5	OI Packet
6	Prev Close Packet

Code	Packet
7	Market Status Packet
8	Full Packet
50	Feed Disconnect

11.9 Trading API Errors

Type	Code	Message
Invalid Authentication	DH-901	Client ID or access token is invalid or expired.
Invalid Access	DH-902	Not subscribed to Data APIs or no access to Trading APIs.
User Account	DH-903	User account error — check segment activation.
Rate Limit	DH-904	Rate limit breached. Throttle API calls.
Input Exception	DH-905	Missing required fields or bad parameter values.
Order Error	DH-906	Order request incorrect, cannot be processed.
Data Error	DH-907	Cannot fetch data due to incorrect parameters or no data.
Internal Server Error	DH-908	Server could not process the request (rare).
Network Error	DH-909	Network error — backend communication failed.
Others	DH-910	Error originating from other reasons.

11.10 Data API / WebSocket Errors

Code	Description
800	Internal Server Error
804	Requested instruments exceed limit
805	Too many requests/connections. May trigger block.
806	Data APIs not subscribed
807	Access token expired
808	Authentication failed — Client ID or Access Token invalid
809	Access token invalid
810	Client ID invalid
811	Invalid expiry date
812	Invalid date format
813	Invalid Security ID
814	Invalid request

11.11 Conditional Trigger — Comparison Types

Type	Description	Mandatory fields
TECHNICAL_WITH_VALUE	Compare indicator vs fixed value	indicatorName, operator, timeFrame, comparingValue
TECHNICAL_WITH_INDICATOR	Compare two indicators	indicatorName, operator, timeFrame, comparingIndicatorName
TECHNICAL_WITH_CLOSE	Compare indicator with closing price	indicatorName, operator, timeFrame
PRICE_WITH_VALUE	Compare market price vs fixed value	operator, comparingValue

11.12 Conditional Trigger — Indicator Names

Indicator	Description
SMA_5 / 10 / 20 / 50 / 100 / 200	Simple Moving Average (periods)
EMA_5 / 10 / 20 / 50 / 100 / 200	Exponential Moving Average
BB_UPPER	Upper Bollinger Band
BB_LOWER	Lower Bollinger Band
RSI_14	Relative Strength Index (14)
ATR_14	Average True Range (14)
STOCHASTIC	Stochastic Oscillator
STOCHRSI_14	Stochastic RSI
MACD_26	MACD long-term component
MACD_12	MACD short-term component
MACD_HIST	MACD histogram

11.13 Conditional Trigger — Operators

Operator	Description
CROSSING_UP	Crosses above
CROSSING_DOWN	Crosses below
CROSSING_ANY_SIDE	Crosses either side
GREATER_THAN	Greater than
LESS_THAN	Less than
GREATER_THAN_EQUAL	Greater than or equal
LESS_THAN_EQUAL	Less than or equal
EQUAL	Equal
NOT_EQUAL	Not equal

12. Release Notes

Versions v2.0 through v2.5. Listed newest-first. Track release page at dhanhq.co/docs/v2/releases/ for newer versions.

Version 2.5

Released: Feb 09 2026

New / Featured: Conditional Trigger Orders, P&L-based; exit (Trader's Control), Exit All API, Access Token Generation via API (TOTP).

Improvements: Option Chain rate limit increased (multiple unique requests per 3 seconds for different expiries/strikes); added `average_price` and `security_id` fields.

Version 2.4

Released: Sep 22 2025

New / Featured: API Key + Secret based login (12-month key validity).

Breaking: Access token now 24-hour validity (was 30 days). Static IP whitelisting mandatory for ALL order APIs.

Version 2.3

Released: Sep 08 2025

New / Featured: Full Market Depth (200-level) on WebSocket. Historical Options Data endpoint (expired contracts, rolling basis with $ATM \pm N$ strikes).

Improvements: Order API rate limit changed to 10 orders/sec.

Version 2.2

Released: Mar 07 2025

New / Featured: Super Orders — combine entry, target, SL, trailing into one order. User Profile API.

Improvements: Intraday Historical extended to 5 years. OI parameter added to historical APIs. CorrelationId and Remarks in Live Order Update. Rate limits increased: Data APIs 5/sec, 100,000/day.

Version 2.1

Released: Jan 06 2025

New / Featured: 20 Market Depth (Level 3) via WebSocket. Option Chain API (with Greeks, IV, OI per strike).

Improvements: expiryCode optional in Daily Historical.

Version 2.0

Released: Sep 15 2024

New / Featured: Market Quote API (LTP/OHLC/Quote). Forever Orders (single + OCO). Live Order Update via WebSocket. Margin Calculator.

Breaking: Order Modification quantity field now expects TOTAL quantity (was pending qty). 'symbol' replaced with 'securityId' in Daily Historical. Epoch timestamps in Historical APIs. 'Market Depth' mode replaced by FULL packet. Trade History endpoint moved to /trades. Error codes prefixed with DH-900 series.

Migration notes for QuantIQ

QuantIQ pins dhanhq==2.1.x in requirements.txt. The v2.4 changes affect us most: 24-hour token rotation must be implemented in the Phase 4 bot's daily startup routine, and the bot's hosting IP must be static (set via /ip/setIP).

Future migrations to v2.5+ will require:

- Adopting Conditional Trigger Orders for strategy automation
- Migrating from /generateAccessToken cron to TOTP-based auto-regen
- Evaluating Super Orders to replace bracketed BO/CO logic
- Considering Option Chain rate-limit changes for multi-expiry strategies

13. Endpoints Not Covered Here

These endpoints exist in DhanHQ v2 but are not used in QuantIQ's current scope. They are documented at the URLs below for when they become relevant in later phases. Refer to the live docs rather than this binder for these.

Endpoint Group	Purpose	URL
Super Order	Combined entry + target + SL + trailing	dhanhq.co/docs/v2/super-order/
Forever Order	GTT orders (single and OCO)	dhanhq.co/docs/v2/forever/
Conditional Trigger	Trigger on indicator/price conditions	dhanhq.co/docs/v2/conditional-trigger/
EDIS	Electronic Delivery Instruction Slip	dhanhq.co/docs/v2/edis/
Trader's Control	P&L; exit, kill switch settings	dhanhq.co/docs/v2/traders-control/
Statement	Ledger and contract notes	dhanhq.co/docs/v2/statements/
Postback	Webhook order updates	dhanhq.co/docs/v2/postback/
Live Order Update	Real-time order status via WebSocket	dhanhq.co/docs/v2/order-update/
Full Market Depth	20 / 200 level depth via WebSocket	dhanhq.co/docs/v2/full-market-depth/
Expired Options Data	Historical OHLC of expired options	dhanhq.co/docs/v2/expired-options-data/
Instrument List	CSV of all tradable securityIds	dhanhq.co/docs/v2/instruments/

When to bring these in

- Super Order — when basic Bracket/Cover Order limits become a bottleneck.
- Forever Order — when GTT-style alerts and execution become part of strategy.
- Conditional Trigger — when strategy logic moves into broker-side rules.
- Live Order Update WebSocket — when polling /orders becomes a rate-limit issue.
- Full Market Depth — when order-book-based strategies (HFT-adjacent) are scoped.
- Postback — when an external dashboard needs realtime push for order events.
- Instrument List — required for any new symbol lookup. Download nightly.

End of Module 1. Next: Module 2 — Python Data Stack (pandas, numpy, yfinance).

QuantIQ

Master Reference Binder

Module 2

Python Data Stack

Version	18 May 2026
Libraries	pandas, numpy, yfinance
Scope	Data ingestion, manipulation, and NSE market data
Pages	approx 30

2

Module Contents

§	Section	Page
1	Quick Reference Card	3
2	pandas — Core Concepts	6
3	pandas — Data Operations	9
4	pandas — Time Series	14
5	pandas — Trading Patterns	17
6	numpy — Essentials	20
7	yfinance — NSE Data Fetching	24
8	Known Issues & Gotchas	28
9	See Also	30

1. Quick Reference Card

Install

```
pip install pandas numpy yfinance
```

Import conventions

```
import pandas as pd
import numpy as np
import yfinance as yf
```

pandas — most-used one-liners

Expression	What it does
<code>pd.DataFrame(data)</code>	Create DataFrame from dict / list / array
<code>pd.read_csv('f.csv')</code>	Load CSV into DataFrame
<code>df.head(n) / df.tail(n)</code>	First / last n rows
<code>df.info()</code>	Schema: dtypes, non-null counts, memory
<code>df.describe()</code>	Stats: count, mean, std, min, quartiles, max
<code>df.shape</code>	Tuple (rows, cols)
<code>df.dtypes</code>	Series of column dtypes
<code>df['col']</code>	Select single column (returns Series)
<code>df[['a', 'b']]</code>	Select multiple columns (returns DataFrame)
<code>df.loc[row, col]</code>	Label-based indexing
<code>df.iloc[i, j]</code>	Integer-position indexing
<code>df[df['col'] > x]</code>	Boolean filter
<code>df.sort_values('col', ascending=False)</code>	Sort by column
<code>df.groupby('col').agg({'v': 'mean'})</code>	Group + aggregate
<code>df.merge(df2, on='key', how='left')</code>	SQL-style join
<code>pd.concat([df1, df2])</code>	Stack DataFrames vertically
<code>df.drop('col', axis=1)</code>	Drop column
<code>df.rename(columns={'old': 'new'})</code>	Rename columns
<code>df.fillna(0)</code>	Fill NaN with 0 (or method='ffill')
<code>df.dropna()</code>	Drop rows with any NaN
<code>df.reset_index()</code>	Reset to integer index
<code>df.set_index('date')</code>	Set date column as index
<code>df.resample('W').mean()</code>	Downsample time series weekly
<code>df['col'].rolling(20).mean()</code>	20-period rolling mean
<code>df['col'].pct_change()</code>	Period-over-period % change
<code>df.to_csv('out.csv', index=False)</code>	Export to CSV

numpy — most-used one-liners

Expression	What it does
<code>np.array([1,2,3])</code>	Create 1-D array
<code>np.zeros((m,n))</code> / <code>np.ones((m,n))</code>	Array of zeros / ones
<code>np.arange(start, stop, step)</code>	Evenly-spaced values
<code>np.linspace(0, 1, 100)</code>	100 values from 0 to 1
<code>arr.shape</code> / <code>arr.dtype</code>	Shape tuple / dtype
<code>arr.reshape(m, n)</code>	Reshape without copy
<code>np.mean(arr)</code> / <code>np.std(arr)</code>	Mean / standard deviation
<code>np.sum(arr, axis=0)</code>	Sum along axis
<code>np.max(arr)</code> / <code>np.argmax(arr)</code>	Max value / index of max
<code>np.where(cond, a, b)</code>	Conditional element selection
<code>np.log(arr)</code> / <code>np.exp(arr)</code>	Log / exp (element-wise)
<code>np.dot(a, b)</code>	Dot product / matrix multiply
<code>np.nan</code> / <code>np.isnan(arr)</code>	NaN sentinel / NaN mask
<code>np.clip(arr, lo, hi)</code>	Clamp values to [lo, hi]
<code>np.percentile(arr, 95)</code>	95th percentile
<code>np.cumsum(arr)</code>	Cumulative sum
<code>arr[arr > 0]</code>	Boolean indexing

yfinance — most-used one-liners

Expression	What it does
<code>yf.download('RELIANCE.NS', period='1y')</code>	Daily OHLCV, 1 year
<code>yf.download('TCS.NS', start='2024-01-01', end='2025-01-01')</code>	Custom date range
<code>yf.download('INFY.NS', interval='5m', period='5d')</code>	5-min bars, last 5 days
<code>yf.Ticker('HDFC.NS')</code>	Create Ticker object
<code>ticker.history(period='6mo')</code>	Historical via Ticker object
<code>ticker.info</code>	Dict of fundamentals (market cap, P/E, etc.)
<code>ticker.dividends</code>	Dividend history Series
<code>ticker.splits</code>	Stock split history
<code>ticker.options</code>	Tuple of option expiry dates
<code>yf.download(['TCS.NS', 'INFY.NS'], period='1y')</code>	Multiple tickers — MultiIndex result

NSE suffix rules: NSE stocks -> .NS | BSE stocks -> .BO | Indices -> ^NSEI (Nifty 50), ^NSEBANK (Bank Nifty)

2. pandas — Core Concepts

Official docs: pandas.pydata.org/docs/ | 10-min intro: pandas.pydata.org/docs/user_guide/10min.html

2.1 Data Structures

Type	Description	Created by
Series	1-D labelled array. One column of data.	<code>pd.Series([1,2,3], index=['a','b','c'])</code>
DataFrame	2-D table. Collection of Series sharing an index.	<code>pd.DataFrame({'A':[1,2], 'B':[3,4]})</code>

2.2 Creating DataFrames

```
# From dict
df = pd.DataFrame({'open':[100,102], 'close':[101,104],
                  'volume':[1e6, 2e6]})

# From CSV
df = pd.read_csv('data.csv', parse_dates=['date'],
                index_col='date')

# From NumPy array
arr = np.random.randn(5, 3)
df = pd.DataFrame(arr, columns=['A', 'B', 'C'])

# From yfinance (already a DataFrame)
df = yf.download('RELIANCE.NS', period='1y')
```

2.3 Inspection

Method	Output
<code>df.head(5)</code>	First 5 rows
<code>df.tail(5)</code>	Last 5 rows
<code>df.info()</code>	Schema — dtypes, non-null count, memory
<code>df.describe()</code>	Numeric stats (count, mean, std, quartiles)
<code>df.shape</code>	(n_rows, n_cols)
<code>df.columns.tolist()</code>	Column name list
<code>df.index</code>	Index object
<code>df.dtypes</code>	dtype per column
<code>df.memory_usage(deep=True)</code>	Bytes per column
<code>df.nunique()</code>	Unique values per column
<code>df.value_counts()</code>	Frequency table (Series only)

2.4 Selecting Data

```
# Single column -> Series
close = df['Close']

# Multiple columns -> DataFrame
ohlcv = df[['Open', 'High', 'Low', 'Close']]

# Label-based (row, col)
val = df.loc['2024-01-15', 'Close']
row = df.loc['2024-01-15'] # full row
sli = df.loc['2024-01':'2024-03'] # date slice
```

```
# Integer position
first = df.iloc[0] # first row
cell = df.iloc[0, 3] # row 0, col 3
block = df.iloc[0:5, 0:4] # rows 0-4, cols 0-3

# Boolean filter
big_vol = df[df['Volume'] > 1_000_000]
green = df[df['Close'] > df['Open']]
combo = df[(df['Close'] > 100) & (df['Volume'] > 5e5)]
```

3. pandas — Data Operations

3.1 Adding and Modifying Columns

```
# New computed column
df['Range'] = df['High'] - df['Low']
df['Returns'] = df['Close'].pct_change()
df['LogRet'] = np.log(df['Close'] / df['Close'].shift(1))

# Conditional column
df['Direction'] = np.where(df['Close'] > df['Open'], 'UP', 'DOWN')

# Apply function
df['AbsRet'] = df['Returns'].apply(abs)

# Rename
df = df.rename(columns={'Adj Close': 'AdjClose'})

# Drop column
df = df.drop('Range', axis=1)
df = df.drop(columns=['Open', 'High', 'Low'])
```

3.2 Handling Missing Data

```
# Detect
df.isna().sum() # NaN count per column
df.isna().any() # True if any NaN

# Fill
df.fillna(0) # with constant
df.fillna(method='ffill') # forward fill (propagate last valid)
df.fillna(method='bfill') # backward fill
df['Close'].fillna(df['Close'].mean())

# Drop
df.dropna() # drop any row with NaN
df.dropna(subset=['Close']) # drop only if Close is NaN
df.dropna(thresh=4) # keep if >=4 non-NaN values

# Interpolate (good for time series gaps)
df['Close'] = df['Close'].interpolate(method='linear')
```

3.3 Sorting and Ranking

```
df.sort_values('Volume', ascending=False) # by column
df.sort_values(['Date', 'Close']) # multi-column
df.sort_index() # by index
df['Rank'] = df['Close'].rank(ascending=False) # rank column
```

3.4 GroupBy and Aggregation

```
# Simple group + aggregate
monthly = df.groupby(df.index.month)['Volume'].sum()

# Multiple aggregations
stats = df.groupby('Symbol').agg({
    'Close': ['mean', 'std', 'min', 'max'],
    'Volume': 'sum',
})

# Custom function
def sharpe(x): return x.mean() / x.std() * (252**0.5)
sr = df.groupby('Symbol')['Returns'].apply(sharpe)

# Transform (keeps original shape – good for z-score)
```

```
df['ZScore'] = df.groupby('Symbol')['Close'].transform(
    lambda x: (x - x.mean()) / x.std()
)
```

3.5 Merging and Joining

```
# SQL-style merge
merged = pd.merge(df_prices, df_meta,
on='Symbol', how='left')

# Index-based join
combined = df1.join(df2, how='inner')

# Stack vertically (same columns)
all_data = pd.concat([df_2023, df_2024], axis=0)

# Stack horizontally (same index)
wide = pd.concat([df_close, df_volume], axis=1)
```

3.6 String Operations on Columns

```
# Series.str accessor
df['Symbol'] = df['Symbol'].str.upper()
df['Ticker'] = df['Symbol'].str.replace('.NS', '', regex=False)
mask = df['Symbol'].str.endswith('.NS') # Boolean mask
df['len'] = df['Symbol'].str.len()
```

3.7 Pivot and Reshape

```
# Pivot table
pivot = df.pivot_table(
    values='Close', index='Date', columns='Symbol',
    aggfunc='mean'
)

# Stack / unstack multi-index
stacked = pivot.stack() # wide -> long
unstacked = stacked.unstack() # long -> wide

# Melt (wide -> long, useful for plotting)
long = pd.melt(df, id_vars=['Date'],
    value_vars=['TCS_Close', 'INFY_Close'],
    var_name='Symbol', value_name='Close')
```

4. pandas — Time Series

Time series handling is central to algo trading. pandas has first-class support via `DatetimeIndex`, `Timedelta`, and `resample`.

4.1 `DatetimeIndex` Basics

```
# Convert string column to datetime
df['Date'] = pd.to_datetime(df['Date'])
df = df.set_index('Date')

# Parse at read time
df = pd.read_csv('data.csv', parse_dates=['Date'],
index_col='Date')

# Access components
df.index.year
df.index.month
df.index.day
df.index.dayofweek # 0=Mon, 4=Fri
df.index.hour

# Date range generation
dates = pd.date_range(start='2024-01-01',
end='2024-12-31', freq='B') # B = business days
```

4.2 Resampling (Aggregating to Lower Frequency)

```
# 1-min bars -> 5-min OHLCV
ohlcv_5m = df.resample('5T').agg({
    'Open': 'first',
    'High': 'max',
    'Low': 'min',
    'Close': 'last',
    'Volume': 'sum',
})

# Daily -> Weekly
weekly = df['Close'].resample('W').last()

# Daily -> Monthly mean
monthly_avg = df['Close'].resample('ME').mean()

# Common freq strings
# 'T' / 'min' = 1 minute '5T' = 5 min
# 'H' = hour 'B' = business day
# 'W' = week 'ME' = month end
# 'QE' = quarter end 'YE' = year end
```

4.3 Rolling and Expanding Windows

```
# Rolling mean (SMA)
df['SMA20'] = df['Close'].rolling(window=20).mean()
df['SMA50'] = df['Close'].rolling(window=50).mean()

# Rolling std
df['Volatility'] = df['Returns'].rolling(20).std() * (252**0.5)

# Exponential weighted mean (EMA)
df['EMA12'] = df['Close'].ewm(span=12, adjust=False).mean()
df['EMA26'] = df['Close'].ewm(span=26, adjust=False).mean()

# Rolling min/max (useful for Donchian channels)
df['DonchUpper'] = df['High'].rolling(20).max()
df['DonchLower'] = df['Low'].rolling(20).min()
```

```
# Expanding (cumulative from start)
df['CumMaxClose'] = df['Close'].expanding().max()
```

4.4 Shifting and Lagging

```
# Shift values forward / backward
df['PrevClose'] = df['Close'].shift(1) # yesterday's close
df['NextOpen'] = df['Open'].shift(-1) # tomorrow's open

# Percentage returns
df['DailyReturn'] = df['Close'].pct_change() # (t - t-1) / t-1
df['WeeklyReturn'] = df['Close'].pct_change(5) # 5-day return

# Log returns (better for compounding)
df['LogReturn'] = np.log(df['Close'] / df['Close'].shift(1))
```

4.5 Timezone Handling (IST for NSE)

```
# yfinance returns UTC for intraday, date-only for daily
# For intraday, convert to IST
df.index = df.index.tz_convert('Asia/Kolkata')

# Localize naive datetime
df.index = df.index.tz_localize('UTC').tz_convert('Asia/Kolkata')

# NSE trading hours (IST)
# Pre-open: 09:00 - 09:15
# Regular: 09:15 - 15:30
# Post-close: 15:30 - 16:00
session = df.between_time('09:15', '15:30')
```


5. pandas — Trading Patterns

Reusable patterns specific to the QuantIQ algo stack. All examples assume a DataFrame with DatetimeIndex and columns: Open, High, Low, Close, Volume.

5.1 Loading DhanHQ Historical Response into DataFrame

```
import pandas as pd, numpy as np

def dhan_to_df(response: dict) -> pd.DataFrame:
    """Convert DhanHQ /charts/historical response to DataFrame."""
    df = pd.DataFrame({
        'Open': response['open'],
        'High': response['high'],
        'Low': response['low'],
        'Close': response['close'],
        'Volume': response['volume'],
    }, index=pd.to_datetime(response['timestamp'], unit='s',
        utc=True).tz_convert('Asia/Kolkata'))
    df.index.name = 'Date'
    return df
```

5.2 VWAP (Manual — ta library has no VWAP)

```
def vwap(df: pd.DataFrame) -> pd.Series:
    """Intraday VWAP. Reset df to intraday session first."""
    typical = (df['High'] + df['Low'] + df['Close']) / 3
    cum_tp_vol = (typical * df['Volume']).cumsum()
    cum_vol = df['Volume'].cumsum()
    return cum_tp_vol / cum_vol

df['VWAP'] = vwap(df)
```

5.3 MACD (Manual)

```
def macd(close: pd.Series, fast=12, slow=26, sig=9):
    ema_fast = close.ewm(span=fast, adjust=False).mean()
    ema_slow = close.ewm(span=slow, adjust=False).mean()
    macd_line = ema_fast - ema_slow
    signal = macd_line.ewm(span=sig, adjust=False).mean()
    histogram = macd_line - signal
    return macd_line, signal, histogram

df['MACD'], df['Signal'], df['Hist'] = macd(df['Close'])
```

5.4 ATR (Average True Range)

```
def atr(df: pd.DataFrame, period=14) -> pd.Series:
    hl = df['High'] - df['Low']
    hpc = (df['High'] - df['Close'].shift(1)).abs()
    lpc = (df['Low'] - df['Close'].shift(1)).abs()
    tr = pd.concat([hl, hpc, lpc], axis=1).max(axis=1)
    return tr.ewm(span=period, adjust=False).mean()

df['ATR'] = atr(df)
```

5.5 Drawdown Calculation

```
def max_drawdown(equity: pd.Series) -> float:
    rolling_max = equity.cummax()
    drawdown = (equity - rolling_max) / rolling_max
    return drawdown.min() # most negative value

df['Equity'] = (1 + df['DailyReturn']).cumprod()
df['DrawdPct'] = (df['Equity'] - df['Equity'].cummax()) \
```

```
/ df['Equity'].cummax() * 100
```

5.6 Sharpe Ratio

```
def sharpe(returns: pd.Series, risk_free=0.065,
           periods=252) -> float:
    """Annualised Sharpe. risk_free = annual rate (decimal)."""
    rf_daily = risk_free / periods
    excess = returns - rf_daily
    return (excess.mean() / excess.std()) * (periods ** 0.5)

print(f'Sharpe: {sharpe(df["DailyReturn"]):.2f}')
```

5.7 Position Sizing (Fixed Fractional)

```
def position_size(capital: float, risk_pct: float,
                  entry: float, stop: float) -> int:
    """How many shares to buy given risk per trade."""
    risk_amount = capital * risk_pct
    risk_per_share = abs(entry - stop)
    if risk_per_share == 0: return 0
    return int(risk_amount / risk_per_share)

# Example: 1% risk on 1L capital, entry=500, stop=490
qty = position_size(100_000, 0.01, 500, 490) # -> 100 shares
```

6. numpy — Essentials

Official docs: numpy.org/doc/stable/ | PyPI: pypi.org/project/numpy/

numpy provides the numerical backbone — fast array operations that pandas and vectorbt run on top of. In QuantIQ, numpy is used directly for signal generation, statistical calculations, and when pandas overhead needs to be bypassed for speed.

6.1 Array Creation

```
import numpy as np

a = np.array([1, 2, 3, 4, 5]) # 1-D
m = np.array([[1,2],[3,4]]) # 2-D
z = np.zeros((3, 3)) # zeros
o = np.ones((3, 3)) # ones
e = np.eye(3) # identity matrix
r = np.arange(0, 10, 0.5) # 0 to 9.5, step 0.5
l = np.linspace(0, 1, 100) # 100 evenly-spaced pts
rng = np.random.default_rng(42) # seeded generator (modern)
n = rng.normal(0, 1, size=(252, 10)) # 252x10 normal random
```

6.2 Array Properties

Attribute/Method	Returns
<code>arr.shape</code>	Tuple of dimension sizes
<code>arr.ndim</code>	Number of dimensions
<code>arr.dtype</code>	Data type (float64, int32, etc.)
<code>arr.size</code>	Total number of elements
<code>arr.nbytes</code>	Memory usage in bytes
<code>arr.T</code>	Transpose
<code>arr.flatten()</code>	1-D copy of array
<code>arr.reshape(m, n)</code>	New shape (no copy if possible)

6.3 Arithmetic and Broadcasting

```
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])

a + b # [11, 22, 33] element-wise
a * b # [10, 40, 90]
a / 2 # [0.5, 1.0, 1.5] broadcast scalar
a ** 2 # [1, 4, 9]

# 2-D broadcasting
m = np.ones((3,3))
v = np.array([1, 2, 3])
m * v # each row multiplied by v
```

6.4 Statistical Functions

Function	Description
<code>np.mean(arr) / np.mean(arr, axis=0)</code>	Mean (overall or per column)
<code>np.median(arr)</code>	Median
<code>np.std(arr, ddof=1)</code>	Standard deviation (ddof=1 for sample)
<code>np.var(arr, ddof=1)</code>	Variance

Function	Description
<code>np.min(arr) / np.max(arr)</code>	Min / max
<code>np.argmin(arr) / np.argmax(arr)</code>	Index of min / max
<code>np.percentile(arr, [25, 50, 75])</code>	Quartiles
<code>np.cumsum(arr)</code>	Running sum
<code>np.cumprod(arr)</code>	Running product
<code>np.diff(arr)</code>	First difference (<code>arr[i+1] - arr[i]</code>)
<code>np.corrcoef(x, y)</code>	Correlation matrix
<code>np.cov(x, y)</code>	Covariance matrix

6.5 Boolean Indexing and where

```
prices = np.array([100, 102, 98, 105, 103])

# Boolean mask
above_100 = prices[prices > 100] # [102, 105, 103]
idx = np.where(prices > 100) # (array([1, 3, 4]),)

# Conditional assignment (vectorised if/else)
signal = np.where(prices > 100, 1, -1) # [-1, 1, -1, 1, 1]

# Clip
capped = np.clip(prices, 100, 103) # [100, 102, 100, 103, 103]
```

6.6 Useful Math for Trading

```
# Log returns
log_ret = np.log(prices[1:] / prices[:-1])

# Annualised volatility from daily log returns
vol_annual = np.std(log_ret, ddof=1) * np.sqrt(252)

# Z-score normalisation
z = (prices - np.mean(prices)) / np.std(prices, ddof=1)

# Rolling correlation using strides (advanced)
# Prefer pandas .rolling().corr() for simplicity

# Vectorised signal: SMA crossover
close = np.array([...]) # your price array
sma20 = np.convolve(close, np.ones(20)/20, mode='valid')
sma50 = np.convolve(close, np.ones(50)/50, mode='valid')
```

6.7 Performance Notes

- Prefer vectorised numpy ops over Python loops — 10-100x faster.
- Use `dtype=np.float32` for large arrays when float64 precision isn't needed.
- `np.nan` is float — integer arrays cannot hold NaN; cast to float first.
- Avoid repeated concatenation inside loops — pre-allocate with `np.empty()`.
- `np.memmap` for arrays too large for RAM (useful for multi-year tick data).

7. yfinance — NSE Data Fetching

Official docs: ranaroussi.github.io/yfinance/ | GitHub: github.com/ranaroussi/yfinance | PyPI: pypi.org/project/yfinance/

yfinance is an unofficial Yahoo Finance scraper. Not affiliated with or endorsed by Yahoo. Intended for personal/educational use. For live trading data use DhanHQ APIs (Module 1).

7.1 NSE Ticker Convention

This is the most common source of silent failures.

Exchange	Suffix	Example
NSE (National Stock Exchange)	.NS	RELIANCE.NS, TCS.NS, INFY.NS
BSE (Bombay Stock Exchange)	.BO	RELIANCE.BO, TCS.BO
NSE Indices	^prefix	^NSEI (Nifty 50), ^NSEBANK (Bank Nifty)
BSE Index	^prefix	^BSESN (Sensex)

An unsuffixed NSE ticker (e.g. 'RELIANCE') silently returns US equity data for a different company with that symbol. Always validate suffix before passing to yfinance. See GitHub issue #825 for the canonical bug report.

```
def nse(symbol: str) -> str:
    """Ensure symbol has .NS suffix."""
    if not symbol.endswith(('NS', 'BO')):
        return symbol + '.NS'
    return symbol
```

7.2 yf.download() — Batch Download

```
import yfinance as yf

# Single ticker, period shorthand
df = yf.download('RELIANCE.NS', period='1y')

# Custom date range
df = yf.download('TCS.NS',
start='2024-01-01',
end='2025-01-01',
auto_adjust=True) # adjusts for splits/dividends

# Intraday (max 60 days for 1m, 730 days for 1h)
df = yf.download('INFY.NS',
period='5d',
interval='5m',
auto_adjust=True)

# Multiple tickers at once -> MultiIndex columns
data = yf.download(['TCS.NS', 'INFY.NS', 'WIPRO.NS'],
period='1y', auto_adjust=True)
tcs_close = data['Close']['TCS.NS']
```

period shorthand values

period	Meaning
1d	1 day
5d	5 days
1mo	1 month
3mo	3 months
6mo	6 months
1y	1 year

period	Meaning
2y	2 years
5y	5 years
10y	10 years
ytd	Year to date
max	Maximum available

interval values

interval	Max history
1m	7 days
2m	60 days
5m	60 days
15m	60 days
30m	60 days
60m / 1h	730 days
1d	Full history
1wk	Full history
1mo	Full history

7.3 Ticker Object — Fundamentals

```

ticker = yf.Ticker('HDFCBANK.NS')

# Historical OHLCV
hist = ticker.history(period='6mo')

# Fundamental info dict
info = ticker.info
market_cap = info.get('marketCap')
pe_ratio = info.get('trailingPE')
sector = info.get('sector')

# Corporate actions
divs = ticker.dividends # pd.Series
splits = ticker.splits # pd.Series

# Options (if listed)
expiries = ticker.options # tuple of date strings
chain = ticker.option_chain(expiries[0]) # calls, puts DataFrames

```

7.4 Downloading NSE Index Data

```

# Nifty 50
nifty = yf.download('^NSEI', period='1y', auto_adjust=True)

# Bank Nifty
banknifty = yf.download('^NSEBANK', period='1y')

# Sensex
sensex = yf.download('^BSESN', period='1y')

# Note: indices have no volume in Yahoo Finance;
# Volume column will be 0 or NaN for index tickers.

```

7.5 Building a Multi-Stock DataFrame

```

symbols = ['RELIANCE.NS', 'TCS.NS', 'INFY.NS',
           'HDFCBANK.NS', 'ICICIBANK.NS']

```

```
# Download all at once
raw = yf.download(symbols, period='2y',
auto_adjust=True, progress=False)

# Extract close prices into a clean DataFrame
close = raw['Close'].copy()
close.columns = [c.replace('.NS','') for c in close.columns]

# Compute daily returns matrix
returns = close.pct_change().dropna()

# Correlation matrix
corr = returns.corr()

print(corr.round(2))
```

7.6 Getting NSE Symbol List from NSE Website

```
# Download EQUITY_L.csv from:
# nseindia.com/market-data/securities-available-for-trading

import pandas as pd
equity = pd.read_csv('EQUITY_L.csv')
symbols_ns = [sym + '.NS' for sym in equity['SYMBOL']]

# Fetch batch (use chunks to avoid timeouts)
import time
chunks = [symbols_ns[i:i+50] for i in range(0, len(symbols_ns), 50)]
frames = []
for chunk in chunks:
    df = yf.download(chunk, period='1y', auto_adjust=True,
progress=False)
    frames.append(df['Close'])
    time.sleep(1) # be polite to Yahoo's servers
all_close = pd.concat(frames, axis=1)
```

8. Known Issues & Gotchas

8.1 yfinance Issues

Issue	Symptom	Fix
Missing .NS suffix	Returns US equity data silently — no error	Always validate suffix. Use nse() wrapper (\$7.1).
'symbol may be delisted' for live NSE stock	Exception on yf.download() even though stock trades	Known bug (GitHub #2612). Retry, or use yf.Ticker().history() instead.
MultiIndex column confusion with multiple tickers	data['Close'] returns all tickers; data['RELIANCE.NS'] fails	Access as data['Close']['RELIANCE.NS'] or data.xs('RELIANCE.NS', axis=1, level=1)
Intraday data limited	1-min bars only available for last 7 days	Store locally after fetching. DhanHQ provides 5-year intraday history.
Yahoo Finance rate limiting	Empty DataFrame returned silently during bulk download	Add time.sleep(1) between batches. Download in chunks of 50 stocks.
Dividends not reflected (auto_adjust=False default in older versions)	Prices don't account for stock splits	Set auto_adjust=True in all downloads.
Timezone inconsistency	Daily data has no timezone; intraday is UTC	Convert intraday to IST: df.index.tz_convert('Asia/Kolkata')

8.2 pandas Gotchas

Issue	Detail
SettingWithCopyWarning	df['col'] = val on a slice raises warning. Use df.loc[:, 'col'] = val or work on a .copy().
Chained indexing silently fails	df[mask]['col'] = val does NOT modify df. Use df.loc[mask, 'col'] = val.
inplace=True is rarely faster	Most inplace ops copy internally anyway. Return a new object and reassign.
pct_change() first row is NaN	Plan for NaN in returns Series. Use .dropna() before passing to indicators.
Integer index after dropna()	Index becomes non-contiguous. Call .reset_index(drop=True) if integer positions matter.
read_csv() date parsing	parse_dates=['col'] infers format; slow on large files. Specify format= explicitly.
Memory: object dtype for strings	String columns stored as Python objects — expensive. Cast to pd.CategoricalDtype() for symbols.

8.3 numpy Gotchas

Issue	Detail
Integer arrays cannot hold NaN	np.array([1, np.nan]) auto-casts to float64. Use float arrays from the start.
In-place operations and views	arr2 = arr1 creates a VIEW not a copy. arr2 = arr1.copy() for independence.
Overflow in integer arithmetic	np.int32 wraps at 2 ³¹ . Use np.int64 or float for large price*volume products.
np.arange with floats	Floating-point steps can produce unexpected element counts. Use np.linspace instead.

9. See Also

Resource	URL
pandas official docs	pandas.pydata.org/docs/
10 Minutes to pandas	pandas.pydata.org/docs/user_guide/10min.html
pandas API reference	pandas.pydata.org/docs/reference/index.html
numpy official docs	numpy.org/doc/stable/
numpy quickstart	numpy.org/doc/stable/user/quickstart.html
numpy API reference	numpy.org/doc/stable/reference/
yfinance official docs	ranaroussi.github.io/yfinance/
yfinance API reference	ranaroussi.github.io/yfinance/reference/index.html
yfinance GitHub	github.com/ranaroussi/yfinance
NSE symbol list (EQUITY_L.csv)	nseindia.com/market-data/securities-available-for-trading
yfinance NSE issue #2612	github.com/ranaroussi/yfinance/issues/2612
yfinance NSE issue #825	github.com/ranaroussi/yfinance/issues/825

Cross-references to other binder modules

Module	Relevance
M1 — DhanHQ v2	Use DhanHQ historical API instead of yfinance for production; <code>dhan_to_df()</code> pattern in §5.1
M3 — ta library	Pass pandas DataFrames directly to ta indicator functions
M4 — vectorbt	vectorbt consumes pandas DataFrames as signal inputs
M5 — Streamlit	<code>st.dataframe(df)</code> and <code>st.line_chart(df)</code> for dashboard display
M6 — Plotly	<code>px.line(df, x=df.index, y='Close')</code> for interactive charts
M7 — Python Standards	Type annotation: <code>pd.DataFrame</code> , <code>np.ndarray</code> , <code>pd.Series</code>

End of Module 2. Next: Module 3 — Technical Analysis (ta library).

QuantIQ

Master Reference Binder

Module 3

Technical Analysis — ta library

Version	18 May 2026
Library	ta (bukosabino) — pip install ta
Scope	All 42 indicators: momentum, volume, volatility, trend, other
Key note	ta has NO VWAP. See §6 for manual implementation.
Pages	approx 28

3

Module Contents

§	Section	Page
1	Quick Reference Card	3
2	Installation & Setup	5
3	Momentum Indicators	7
4	Volume Indicators	12
5	Volatility Indicators	15
6	Trend Indicators	19
7	Other Indicators	23
8	VWAP — Manual Implementation	25
9	Adding All Indicators at Once	26
10	Indicator Patterns for QuantIQ	27
11	See Also	29

1. Quick Reference Card

Install & import

```
pip install ta

from ta import add_all_ta_features
from ta.utils import dropna
# Category imports
from ta.momentum import RSIIndicator, MACD, StochasticOscillator
from ta.volatility import BollingerBands, AverageTrueRange
from ta.trend import SMAIndicator, EMAIndicator, ADXIndicator
from ta.volume import OnBalanceVolumeIndicator, MFIIndicator
```

All indicators at a glance

Category	Indicators (42 total)
Momentum (11)	RSI, MACD, Stochastic, StochRSI, ROC, Williams %R, Awesome Oscillator, KAMA, PPO, PVO, TSI, Ultimate Oscillator
Volume (9)	OBV, Chaikin MF, Force Index, EoM, VPT, SMA EoM, MFI, NVI, VWAP (not in ta — see §8)
Volatility (8)	Bollinger Bands, ATR, Keltner Channel, Donchian Channel, UI (Ulcer Index)
Trend (14)	SMA, EMA, WMA, DEMA, TEMA, TRIX, Mass Index, CCI, DPO, KST, Ichimoku, PSAR, STC, ADX
Other (2)	Daily Log Return, Cumulative Return

Key usage patterns

Pattern	Example
Class-based (OOP)	<code>ind = RSIIndicator(close=df['Close'], window=14) df['RSI'] = ind.rsi()</code>
Function-based (inline)	<code>from ta.momentum import rsi df['RSI'] = rsi(df['Close'], window=14)</code>
Add all at once	<code>df = add_all_ta_features(df, open='Open', high='High', low='Low', close='Close', volume='Volume')</code>
<code>fillna=True</code>	Pass <code>fillna=True</code> to any indicator to fill NaN startup values
<code>dropna</code> helper	<code>df = dropna(df)</code> # removes rows where any column is NaN

Most-used indicators cheat sheet

One-liner	What it gives
<code>RSIIndicator(close=df['Close'], window=14).rsi()</code>	RSI oscillator 0–100
<code>MACD(close=df['Close']).macd()</code>	MACD line
<code>MACD(close=df['Close']).macd_signal()</code>	Signal line (EMA of MACD)
<code>MACD(close=df['Close']).macd_diff()</code>	Histogram (MACD - Signal)
<code>BollingerBands(close=df['Close'], window=20, window_dev=2).bollinger_hband()</code>	Upper band
<code>BollingerBands(...).bollinger_lband()</code>	Lower band
<code>BollingerBands(...).bollinger_mavg()</code>	Middle band (SMA20)
<code>BollingerBands(...).bollinger_wband()</code>	Bandwidth %
<code>BollingerBands(...).bollinger_pband()</code>	%B position
<code>AverageTrueRange(high=df['High'], low=df['Low'], close=df['Close'], window=14).average_true_range()</code>	ATR
<code>SMAIndicator(close=df['Close'], window=20).sma_indicator()</code>	SMA 20
<code>EMAIndicator(close=df['Close'], window=12).ema_indicator()</code>	EMA 12

One-liner	What it gives
<code>ADXIndicator(high=df['High'], low=df['Low'], close=df['Close'], window=14).adx()</code>	ADX strength
<code>StochasticOscillator(high=df['High'], low=df['Low'], close=df['Close']).stoch()</code>	Stoch %K
<code>OnBalanceVolumeIndicator(close=df['Close'], volume=df['Volume']).on_balance_volume()</code>	OBV

WARNING: ta has NO VWAP indicator. See §8 for the manual implementation required in QuantIQ.

2. Installation & Setup

Official docs: technical-analysis-library-in-python.readthedocs.io | GitHub: github.com/bukosabino/ta | PyPI: pypi.org/project/ta/

2.1 Install

```
pip install ta
# or with exact version
pip install ta==0.11.0
```

2.2 DataFrame requirements

The ta library expects a pandas DataFrame with these EXACT column names (case-sensitive). yfinance and DhanHQ both deliver these names after the dhan_to_df() converter (Module 2 §5.1).

Column	dtype	Notes
Open	float64	Opening price
High	float64	High price
Low	float64	Low price
Close	float64	Closing price
Volume	float/int64	Trade volume

2.3 Data prep pattern

```
import pandas as pd
import yfinance as yf
from ta.utils import dropna

# 1. Load data
df = yf.download('RELIANCE.NS', period='1y', auto_adjust=True)

# 2. Drop NaN rows before adding indicators
df = dropna(df)

# 3. Ensure column names match ta expectations
# yfinance already uses Open/High/Low/Close/Volume
# DhanHQ data needs the dhan_to_df() wrapper from M2 §5.1

# 4. Add indicators
from ta.momentum import RSIIndicator
df['RSI'] = RSIIndicator(close=df['Close'], window=14).rsi()

# 5. Drop NaN again (indicator warmup period creates NaN at top)
df = df.dropna()
```

2.4 Two usage styles

Class-based (OOP — recommended for multiple outputs from same indicator):

```
from ta.momentum import MACD

macd = MACD(close=df['Close'],
            window_slow=26, window_fast=12, window_sign=9)

df['MACD_line'] = macd.macd()
df['MACD_signal'] = macd.macd_signal()
df['MACD_hist'] = macd.macd_diff()
```

Function-based (inline — one-liners for single output):

```
from ta.momentum import rsi, macd, macd_signal, macd_diff
```

```
df['RSI'] = rsi(df['Close'], window=14)
df['MACD_line'] = macd(df['Close'], window_slow=26, window_fast=12, window_sign=9)
df['MACD_signal'] = macd_signal(df['Close'], window_slow=26, window_fast=12,
window_sign=9)
df['MACD_hist'] = macd_diff(df['Close'], window_slow=26, window_fast=12, window_sign=9)
```

3. Momentum Indicators

Import: from ta.momentum import ...

3.1 RSI — Relative Strength Index

Measures speed and change of price movements. Primary use: overbought (>70) / oversold (<30) signals.

Parameter	Default	Description
close	required	Close price Series
window	14	Lookback period
fillna	False	Fill NaN startup values

```
from ta.momentum import RSIIndicator
ind = RSIIndicator(close=df['Close'], window=14)
df['RSI'] = ind.rsi()

# Signal logic
df['RSI_ob'] = df['RSI'] > 70 # Overbought
df['RSI_os'] = df['RSI'] < 30 # Oversold
```

3.2 MACD — Moving Average Convergence Divergence

Trend-following momentum indicator. Three outputs: MACD line, signal line, histogram.

Parameter	Default	Description
close	required	Close price Series
window_slow	26	Slow EMA period
window_fast	12	Fast EMA period
window_sign	9	Signal line EMA period
fillna	False	Fill NaN

```
from ta.momentum import MACD
macd = MACD(close=df['Close'])
df['MACD'] = macd.macd() # MACD line
df['MACD_signal'] = macd.macd_signal() # Signal line
df['MACD_hist'] = macd.macd_diff() # Histogram

# Crossover signal: MACD crosses above signal -> bullish
df['MACD_cross_up'] = (df['MACD'] > df['MACD_signal']) & \
(df['MACD'].shift(1) <= df['MACD_signal'].shift(1))
```

3.3 Stochastic Oscillator

%K (fast) and %D (slow signal). Overbought >80, oversold <20.

Parameter	Default	Description
high	required	High price Series
low	required	Low price Series
close	required	Close price Series
window	14	Lookback period
smooth_window	3	SMA period over %K to produce %D
fillna	False	Fill NaN

```
from ta.momentum import StochasticOscillator
stoch = StochasticOscillator(high=df['High'], low=df['Low'],
close=df['Close'], window=14, smooth_window=3)
df['Stoch_K'] = stoch.stoch() # %K
```



```
df['Stoch_D'] = stoch.stoch_signal() # %D (signal)
```

3.4 Stochastic RSI

Applies Stochastic formula to RSI values rather than price. More sensitive than standard RSI.

```
from ta.momentum import StochRSIIndicator
srsi = StochRSIIndicator(close=df['Close'], window=14, smooth1=3, smooth2=3)
df['StochRSI'] = srsi.stochrsi()
df['StochRSI_K'] = srsi.stochrsi_k()
df['StochRSI_D'] = srsi.stochrsi_d()
```

3.5 ROC — Rate of Change

Pure momentum oscillator: % price change vs n periods ago. Oscillates above/below zero.

```
from ta.momentum import ROCIndicator
df['ROC'] = ROCIndicator(close=df['Close'], window=12).roc()
```

3.6 Williams %R

Oscillates 0 to -100. Overbought: 0 to -20. Oversold: -80 to -100.

```
from ta.momentum import WilliamsRIndicator
df['WilliamsR'] = WilliamsRIndicator(
    high=df['High'], low=df['Low'],
    close=df['Close'], lbp=14).williams_r()
```

3.7 Awesome Oscillator

SMA(5) of midpoints minus SMA(34) of midpoints. Zero-line crossovers indicate momentum shifts.

```
from ta.momentum import AwesomeOscillatorIndicator
df['AO'] = AwesomeOscillatorIndicator(
    high=df['High'], low=df['Low'],
    window1=5, window2=34).awesome_oscillator()
```

3.8 KAMA — Kaufman's Adaptive Moving Average

Adapts speed based on market noise. Tracks closely in low-noise markets; slows in choppy conditions.

```
from ta.momentum import KAMAIndicator
df['KAMA'] = KAMAIndicator(close=df['Close'],
    window=10, pow1=2, pow2=30).kama()
```

3.9 PPO — Percentage Price Oscillator

MACD expressed as percentage of the slower EMA. Allows comparison across stocks of different prices.

```
from ta.momentum import PercentagePriceOscillator
ppo = PercentagePriceOscillator(close=df['Close'],
    window_slow=26, window_fast=12, window_sign=9)
df['PPO'] = ppo.ppo()
df['PPO_signal'] = ppo.ppo_signal()
df['PPO_hist'] = ppo.ppo_hist()
```

3.10 TSI — True Strength Index

```
from ta.momentum import TSIIndicator
df['TSI'] = TSIIndicator(close=df['Close'],
    window_slow=25, window_fast=13).tsi()
```

3.11 Ultimate Oscillator

Captures momentum across three timeframes (7, 14, 28) to reduce false divergence signals.

```
from ta.momentum import UltimateOscillator
df['UO'] = UltimateOscillator(
    high=df['High'], low=df['Low'], close=df['Close'],
    window1=7, window2=14, window3=28).ultimate_oscillator()
```

4. Volume Indicators

Import: from ta.volume import ...

4.1 OBV — On Balance Volume

Cumulative volume: add on up days, subtract on down days. OBV trending diverges from price -> distribution/accumulation signal.

```
from ta.volume import OnBalanceVolumeIndicator
df['OBV'] = OnBalanceVolumeIndicator(
    close=df['Close'], volume=df['Volume']).on_balance_volume()
```

4.2 MFI — Money Flow Index

RSI-like oscillator using volume-weighted typical price. Overbought >80, oversold <20.

Parameter	Default	Description
high/low/close/volume	required	OHLCV Series
window	14	Lookback period
fillna	False	Fill NaN

```
from ta.volume import MFIIndicator
df['MFI'] = MFIIndicator(
    high=df['High'], low=df['Low'],
    close=df['Close'], volume=df['Volume'],
    window=14).money_flow_index()
```

4.3 CMF — Chaikin Money Flow

Oscillates -1 to +1. Sustained above 0: buying pressure. Sustained below 0: selling pressure.

```
from ta.volume import ChaikinMoneyFlowIndicator
df['CMF'] = ChaikinMoneyFlowIndicator(
    high=df['High'], low=df['Low'],
    close=df['Close'], volume=df['Volume'],
    window=20).chaikin_money_flow()
```

4.4 Force Index

Price change × volume. Spikes indicate strong moves.

```
from ta.volume import ForceIndexIndicator
df['FI'] = ForceIndexIndicator(
    close=df['Close'], volume=df['Volume'],
    window=13).force_index()
```

4.5 EoM — Ease of Movement

Relates price change to volume. High EoM: price moves easily with little volume (trend likely to continue).

```
from ta.volume import EaseOfMovementIndicator
eom = EaseOfMovementIndicator(
    high=df['High'], low=df['Low'],
    volume=df['Volume'], window=14)
df['EoM'] = eom.ease_of_movement()
df['EoM_SMA'] = eom.sma_ease_of_movement()
```

4.6 VPT — Volume Price Trend

```
from ta.volume import VolumePriceTrendIndicator
df['VPT'] = VolumePriceTrendIndicator(
    close=df['Close'], volume=df['Volume']).volume_price_trend()
```

4.7 NVI — Negative Volume Index

Tracks price changes on days when volume decreases. Theory: smart money trades on low-volume days.

```
from ta.volume import NegativeVolumeIndexIndicator
df['NVI'] = NegativeVolumeIndexIndicator(
    close=df['Close'], volume=df['Volume']).negative_volume_index()
```

4.8 PVO — Percentage Volume Oscillator

```
from ta.volume import PercentageVolumeOscillator
pvo = PercentageVolumeOscillator(
    volume=df['Volume'],
    window_slow=26, window_fast=12, window_sign=9)
df['PVO'] = pvo.pvo()
df['PVO_signal'] = pvo.pvo_signal()
df['PVO_hist'] = pvo.pvo_hist()
```

5. Volatility Indicators

Import: from ta.volatility import ...

5.1 Bollinger Bands

SMA $\pm$ n standard deviations. Squeeze (narrow bands) precedes breakout. %B shows position within bands. Bandwidth measures band width.

Parameter	Default	Description
close	required	Close price Series
window	20	SMA/std period
window_dev	2	Number of standard deviations
fillna	False	Fill NaN

```
from ta.volatility import BollingerBands
bb = BollingerBands(close=df['Close'], window=20, window_dev=2)

df['BB_upper'] = bb.bollinger_hband() # Upper band
df['BB_mid'] = bb.bollinger_mavg() # Middle (SMA 20)
df['BB_lower'] = bb.bollinger_lband() # Lower band
df['BB_width'] = bb.bollinger_wband() # Bandwidth %
df['BB_pct'] = bb.bollinger_pband() # %B: 0=lower, 1=upper
df['BB_hi_ind'] = bb.bollinger_hband_indicator() # 1 if close > upper
df['BB_lo_ind'] = bb.bollinger_lband_indicator() # 1 if close < lower
```

5.2 ATR — Average True Range

Measures market volatility. Used for stop-loss sizing: stop = entry - (ATR $\times$ multiplier). NOT a directional indicator.

Parameter	Default	Description
high/low/close	required	OHLC Series
window	14	EMA period for smoothing TR
fillna	False	Fill NaN

```
from ta.volatility import AverageTrueRange
atr = AverageTrueRange(high=df['High'], low=df['Low'],
close=df['Close'], window=14)
df['ATR'] = atr.average_true_range()

# ATR-based stop loss example
entry = df['Close'].iloc[-1]
atr_val = df['ATR'].iloc[-1]
stop = entry - (2 * atr_val) # 2x ATR trailing stop
```

5.3 Keltner Channel

EMA $\pm$ (ATR $\times$ multiplier). When Bollinger Bands squeeze inside Keltner Channel: high-probability breakout setup.

```
from ta.volatility import KeltnerChannel
kc = KeltnerChannel(high=df['High'], low=df['Low'],
close=df['Close'], window=20, window_atr=10)
df['KC_upper'] = kc.keltner_channel_hband()
df['KC_mid'] = kc.keltner_channel_mband()
df['KC_lower'] = kc.keltner_channel_lband()
df['KC_width'] = kc.keltner_channel_wband()
df['KC_pct'] = kc.keltner_channel_pband()
```

5.4 Donchian Channel

Highest high and lowest low over n periods. Breakout above upper band = bullish; below lower = bearish.

```
from ta.volatility import DonchianChannel
dc = DonchianChannel(high=df['High'], low=df['Low'],
close=df['Close'], window=20)
df['DC_upper'] = dc.donchian_channel_hband()
df['DC_mid'] = dc.donchian_channel_mband()
df['DC_lower'] = dc.donchian_channel_lband()
df['DC_width'] = dc.donchian_channel_wband()
df['DC_pct'] = dc.donchian_channel_pband()
```

5.5 Ulcer Index (UI)

Measures downside risk (drawdown depth and duration). Lower is better. Useful for risk-adjusted performance.

```
from ta.volatility import UlcerIndex
df['UI'] = UlcerIndex(close=df['Close'], window=14).ulcer_index()
```

6. Trend Indicators

Import: from ta.trend import ...

6.1 SMA — Simple Moving Average

```
from ta.trend import SMAIndicator
df['SMA20'] = SMAIndicator(close=df['Close'], window=20).sma_indicator()
df['SMA50'] = SMAIndicator(close=df['Close'], window=50).sma_indicator()
df['SMA200'] = SMAIndicator(close=df['Close'], window=200).sma_indicator()

# Golden cross signal: SMA50 crosses above SMA200
df['Golden'] = (df['SMA50'] > df['SMA200']) & \
(df['SMA50'].shift(1) <= df['SMA200'].shift(1))
```

6.2 EMA — Exponential Moving Average

```
from ta.trend import EMAIndicator
df['EMA12'] = EMAIndicator(close=df['Close'], window=12).ema_indicator()
df['EMA26'] = EMAIndicator(close=df['Close'], window=26).ema_indicator()
```

6.3 WMA — Weighted Moving Average

```
from ta.trend import WMAIndicator
df['WMA10'] = WMAIndicator(close=df['Close'], window=10).wma()
```

6.4 ADX — Average Directional Index

Measures trend strength (not direction). ADX > 25 = strong trend. +DI > -DI = bullish; -DI > +DI = bearish.

Parameter	Default	Description
high/low/close	required	OHLC Series
window	14	Lookback period
fillna	False	Fill NaN

```
from ta.trend import ADXIndicator
adx = ADXIndicator(high=df['High'], low=df['Low'],
close=df['Close'], window=14)
df['ADX'] = adx.adx() # Trend strength 0-100
df['ADX_pos'] = adx.adx_pos() # +DI (bullish directional)
df['ADX_neg'] = adx.adx_neg() # -DI (bearish directional)

# Trend filter: only trade when ADX > 25
df['Trending'] = df['ADX'] > 25
```

6.5 CCI — Commodity Channel Index

Oscillates around 0. Above +100: overbought / potential reversal down. Below -100: oversold / potential reversal up.

```
from ta.trend import CCIIndicator
df['CCI'] = CCIIndicator(high=df['High'], low=df['Low'],
close=df['Close'], window=20).cci()
```

6.6 PSAR — Parabolic SAR

Trailing stop-and-reverse system. When price crosses SAR: reverse position. Good for trending markets.

```
from ta.trend import PSARIndicator
psar = PSARIndicator(high=df['High'], low=df['Low'],
close=df['Close'], step=0.02, max_step=0.2)
df['PSAR'] = psar.psar() # SAR value
```

```
df['PSAR_up'] = psar.psar_up() # SAR when uptrend
df['PSAR_down'] = psar.psar_down() # SAR when downtrend
df['PSAR_up_ind'] = psar.psar_up_indicator() # 1 in uptrend
df['PSAR_dn_ind'] = psar.psar_down_indicator() # 1 in downtrend
```

6.7 Ichimoku Cloud

Japanese system with five lines. Price above cloud = bullish; below cloud = bearish. Cloud twist = potential trend change.

```
from ta.trend import IchimokuIndicator
ichi = IchimokuIndicator(high=df['High'], low=df['Low'],
window1=9, window2=26, window3=52)
df['Ichi_a'] = ichi.ichimoku_a() # Senkou Span A
df['Ichi_b'] = ichi.ichimoku_b() # Senkou Span B
df['Ichi_base'] = ichi.ichimoku_base_line() # Kijun-sen
df['Ichi_conv'] = ichi.ichimoku_conversion_line() # Tenkan-sen
```

6.8 DEMA, TEMA, TRIX

```
from ta.trend import DEMAIndicator, TEMAIndicator, TRIXIndicator
df['DEMA'] = DEMAIndicator(close=df['Close'], window=15).dema_indicator()
df['TEMA'] = TEMAIndicator(close=df['Close'], window=15).tema_indicator()
df['TRIX'] = TRIXIndicator(close=df['Close'], window=15).trix()
```

6.9 KST — Know Sure Thing

Momentum oscillator based on smoothed rate-of-change across four timeframes.

```
from ta.trend import KSTIndicator
kst = KSTIndicator(close=df['Close'])
df['KST'] = kst.kst()
df['KST_signal'] = kst.kst_sig()
```

6.10 Mass Index

Identifies trend reversals using the high-low range. 'Bulge' above 27 then drop below 26.5 = reversal signal.

```
from ta.trend import MassIndex
df['MassIdx'] = MassIndex(high=df['High'], low=df['Low'],
window_fast=9, window_slow=25).mass_index()
```

6.11 DPO — Detrended Price Oscillator

```
from ta.trend import DPOIndicator
df['DPO'] = DPOIndicator(close=df['Close'], window=20).dpo()
```

6.12 STC — Schaff Trend Cycle

Combines MACD and Stochastic to detect trend cycles faster. Above 75 = overbought; below 25 = oversold.

```
from ta.trend import STCIndicator
df['STC'] = STCIndicator(close=df['Close'],
window_slow=50, window_fast=23,
cycle=10, smooth1=3, smooth2=3).stc()
```


7. Other Indicators

Import: from ta.others import ...

7.1 Daily Return

```
from ta.others import DailyReturnIndicator
df['DailyReturn'] = DailyReturnIndicator(
    close=df['Close']).daily_return()
```

7.2 Daily Log Return

```
from ta.others import DailyLogReturnIndicator
df['LogReturn'] = DailyLogReturnIndicator(
    close=df['Close']).daily_log_return()
```

7.3 Cumulative Return

```
from ta.others import CumulativeReturnIndicator
df['CumReturn'] = CumulativeReturnIndicator(
    close=df['Close']).cumulative_return()
```

8. VWAP — Manual Implementation

CRITICAL: The `ta` library does NOT include a VWAP indicator. Any code that tries `ta.volume.VolumeWeightedAveragePrice()` will raise `ImportError`. Implement manually using the pattern below.

VWAP formula

Typical Price = (High + Low + Close) / 3

VWAP = $\text{cumsum}(\text{Typical Price} \times \text{Volume}) / \text{cumsum}(\text{Volume})$

VWAP resets at the start of each trading session (9:15 IST for NSE).

8.1 Daily VWAP (single session)

```
import pandas as pd

def vwap_session(df: pd.DataFrame) -> pd.Series:
    """VWAP for a single intraday session.
    df must contain: High, Low, Close, Volume.
    Assumes df is already filtered to one trading session.
    """
    typical = (df['High'] + df['Low'] + df['Close']) / 3
    cum_tp_vol = (typical * df['Volume']).cumsum()
    cum_vol = df['Volume'].cumsum()
    return cum_tp_vol / cum_vol

# Usage on intraday data (1-min bars)
df_intra = yf.download('RELIANCE.NS', period='1d', interval='1m')
df_intra.index = df_intra.index.tz_convert('Asia/Kolkata')
session = df_intra.between_time('09:15', '15:30')
session['VWAP'] = vwap_session(session)
```

8.2 Multi-day VWAP (resets each session)

```
def add_vwap(df: pd.DataFrame) -> pd.DataFrame:
    """Adds VWAP column resetting each trading day."""
    df = df.copy()
    df['Date'] = df.index.date
    df['TP'] = (df['High'] + df['Low'] + df['Close']) / 3
    df['TP_V'] = df['TP'] * df['Volume']
    df['VWAP'] = (
        df.groupby('Date')['TP_V'].cumsum() /
        df.groupby('Date')['Volume'].cumsum()
    )
    return df.drop(columns=['Date', 'TP', 'TP_V'])

df_5m = yf.download('TCS.NS', period='5d', interval='5m')
df_5m.index = df_5m.index.tz_convert('Asia/Kolkata')
df_5m = df_5m.between_time('09:15', '15:30')
df_5m = add_vwap(df_5m)
```

9. Adding All Indicators at Once

The `add_all_ta_features()` function adds all 42 indicators as new columns to the DataFrame. Use for feature engineering / ML pipelines. Results in ~60 additional columns.

```
from ta import add_all_ta_features
from ta.utils import dropna

df = dropna(df) # Required before adding features

df = add_all_ta_features(
    df,
    open='Open', # Column name for open
    high='High', # Column name for high
    low='Low', # Column name for low
    close='Close', # Column name for close
    volume='Volume' # Column name for volume
)
```

Output column naming convention

Prefix	Category	Example columns
momentum_	Momentum	momentum_rsi, momentum_macd, momentum_stoch
volume_	Volume	volume_obv, volume_mfi, volume_cmf
volatility_	Volatility	volatility_bbh, volatility_bbl, volatility_atr
trend_	Trend	trend_sma_fast, trend_ema_fast, trend_adx
others_	Others	others_dr, others_dlr, others_cr

Caution: `add_all_ta_features()` requires ALL five columns (open, high, low, close, volume) to be present and non-null. Run `dropna()` before calling. The function adds ~60 columns which may slow DataFrame operations — drop unused columns after.

10. Indicator Patterns for QuantIQ

10.1 Standard QuantIQ indicator pipeline

```
import pandas as pd, numpy as np, yfinance as yf
from ta.utils import dropna
from ta.momentum import RSIIndicator, MACD
from ta.volatility import BollingerBands, AverageTrueRange
from ta.trend import SMAIndicator, EMAIndicator, ADXIndicator
from ta.volume import OnBalanceVolumeIndicator

def build_features(df: pd.DataFrame) -> pd.DataFrame:
    df = dropna(df.copy())

    # Trend
    df['SMA20'] = SMAIndicator(df['Close'], 20).sma_indicator()
    df['SMA50'] = SMAIndicator(df['Close'], 50).sma_indicator()
    df['EMA12'] = EMAIndicator(df['Close'], 12).ema_indicator()
    df['EMA26'] = EMAIndicator(df['Close'], 26).ema_indicator()
    adx = ADXIndicator(df['High'], df['Low'], df['Close'], 14)
    df['ADX'] = adx.adx()
    df['ADX_pos'] = adx.adx_pos()
    df['ADX_neg'] = adx.adx_neg()

    # Momentum
    df['RSI'] = RSIIndicator(df['Close'], 14).rsi()
    macd = MACD(df['Close'])
    df['MACD'] = macd.macd()
    df['MACD_s'] = macd.macd_signal()
    df['MACD_h'] = macd.macd_diff()

    # Volatility
    bb = BollingerBands(df['Close'], 20, 2)
    df['BB_h'] = bb.bollinger_hband()
    df['BB_l'] = bb.bollinger_lband()
    df['BB_pct'] = bb.bollinger_pband()
    df['ATR'] = AverageTrueRange(df['High'], df['Low'], df['Close'], 14).average_true_range()

    # Volume
    df['OBV'] = OnBalanceVolumeIndicator(df['Close'], df['Volume']).on_balance_volume()

    return df.dropna()

df = yf.download('RELIANCE.NS', period='2y', auto_adjust=True)
df = build_features(df)
```

10.2 Simple signal generation example

```
def generate_signals(df: pd.DataFrame) -> pd.DataFrame:
    """Basic trend + momentum confluence signal."""
    # Trend filter: price above SMA50 and ADX > 25
    trend_up = (df['Close'] > df['SMA50']) & (df['ADX'] > 25)

    # Momentum entry: RSI crosses 50 from below
    rsi_cross = (df['RSI'] > 50) & (df['RSI'].shift(1) <= 50)

    # MACD confirms: MACD > signal
    macd_bull = df['MACD'] > df['MACD_s']

    df['Signal'] = (trend_up & rsi_cross & macd_bull).astype(int)
    df['Signal_exit'] = (df['RSI'] > 70).astype(int) # exit overbought
    return df

df = generate_signals(df)
entries = df[df['Signal'] == 1]
```

11. See Also

Resource	URL
ta official docs (API reference)	technical-analysis-library-in-python.readthedocs.io/en/latest/ta.html
ta GitHub	github.com/bukosabino/ta
ta PyPI	pypi.org/project/ta/
pandas rolling docs	pandas.pydata.org/docs/reference/api/pandas.Series.rolling.html
Investopedia — RSI	investopedia.com/terms/r/rsi.asp
Investopedia — MACD	investopedia.com/terms/m/macd.asp
Investopedia — Bollinger Bands	investopedia.com/terms/b/bollingerbands.asp
Investopedia — ATR	investopedia.com/terms/a/atr.asp
Investopedia — ADX	investopedia.com/terms/a/adx.asp
StockCharts — Stochastic	school.stockcharts.com/doku.php?id=technical_indicators:stochastic_oscillator_fast_slow_and_full

Cross-references to other binder modules

Module	Relevance
M2 — Python Data Stack	Provides the pandas DataFrames that ta indicators consume; VWAP workaround pattern
M4 — vectorbt	Consumes signals produced by ta; Portfolio.from_signals() takes boolean Series
M6 — Plotly	Plot indicator outputs alongside price: px.line(df, y=['Close', 'SMA20', 'BB_h', 'BB_l'])
M1 — DhanHQ	Historical data API to populate DataFrames before running indicators

End of Module 3. Next: Module 4 — Backtesting (vectorbt).

QuantIQ

Master Reference Binder

Module 4

Backtesting — vectorbt

Version	18 May 2026
Library	vectorbt==0.28.2 (open-source). NOT 1.x (paid PRO).
Scope	Portfolio simulation, signal processing, stats, plotting
Python	Requires >=3.11,<3.13 for stable Numba wheels
Pages	approx 30



Module Contents

§	Section	Page
1	Quick Reference Card	3
2	Installation & Version Constraints	5
3	Core Concepts	6
4	Portfolio.from_signals() — Full Reference	8
5	Portfolio.from_holding() and from_orders()	15
6	Built-in Indicators (vbt.MA, vbt.RSI, etc.)	17
7	Performance Stats — Full Reference	19
8	Plotting	23
9	Parameter Sweeps & Optimisation	25
10	QuantIQ Backtest Patterns	27
11	See Also	30

1. Quick Reference Card

Install & import

```
pip install vectorbt==0.28.2

import vectorbt as vbt
import pandas as pd
import numpy as np
```

Minimal backtest — 3 lines

```
pf = vbt.Portfolio.from_signals(close, entries, exits)
print(pf.stats())
pf.plot().show()
```

from_signals() key parameters cheat sheet

Parameter	Type	Default	Purpose
close	Series/DataFrame	required	Price series (used for valuation)
entries	bool Series/DataFrame	required	True = open long position
exits	bool Series/DataFrame	required	True = close long position
open	Series/DataFrame	None	Open price (for slippage model)
high	Series/DataFrame	None	High price (for stop orders)
low	Series/DataFrame	None	Low price (for stop orders)
init_cash	float	100	Starting capital
size	float	inf	Order size (shares or fraction of cash)
size_type	str	'amount'	'amount', 'value', 'percent', 'percent_of_equity'
fees	float	0.0	Fee per trade as decimal (0.001 = 0.1%)
slippage	float	0.0	Slippage as decimal of price
sl_stop	float	None	Stop-loss as % of entry price (0.05 = 5%)
tp_stop	float	None	Take-profit as % of entry price (0.10 = 10%)
ts_stop	float	None	Trailing stop as % from peak
freq	str	None	'1D', '5T', '1H' — required for time-based stats
direction	str	'longonly'	'longonly', 'shortonly', 'both'
cash_sharing	bool	False	Share capital across columns (multi-asset)
group_by	array	None	Group columns for combined portfolio stats

Key stats output (from pf.stats())

Metric	Description
Start / End	Backtest period boundaries
Period	Total backtest duration
Start Value / End Value	Portfolio value at start/end
Total Return [%]	Overall return from Start Value to End Value
Benchmark Return [%]	Buy-and-hold return on same data
Max Gross Exposure [%]	Peak capital deployed
Total Fees Paid	Total brokerage cost
Max Drawdown [%]	Worst peak-to-trough decline
Max Drawdown Duration	Duration of worst drawdown
Total Trades	Number of completed round trips
Win Rate [%]	% of trades that were profitable
Best Trade [%]	Best single trade return
Worst Trade [%]	Worst single trade return
Avg Winning Trade [%]	Mean return of winning trades
Avg Losing Trade [%]	Mean return of losing trades
Profit Factor	Gross profit / gross loss
Expectancy	Average expected profit per trade
Sharpe Ratio	Risk-adjusted return (annualised)
Calmar Ratio	Annual return / max drawdown
Omega Ratio	Probability-weighted ratio of gains vs losses
Sortino Ratio	Sharpe using downside deviation only

2. Installation & Version Constraints

WARNING: vectorbt open-source is 0.28.x. Version 1.x is the paid VectorBT PRO product. Never pin vectorbt>=1.0 in requirements.txt.

2.1 Install

```
# Pin exactly to open-source version
pip install vectorbt==0.28.2

# With optional dependencies for full plotting
pip install vectorbt==0.28.2 plotly kaleido
```

2.2 Python version

Pin Python to `>=3.11,<3.13` in `pyproject.toml`. `vectorbt` depends on `Numba` which requires `llvmlite`. The 3.13 `llvmlite` wheel availability is not stable as of May 2026. Set in `pyproject.toml`:

```
[project]
requires-python = ">=3.11,<3.13"
```

2.3 requirements.txt entry

```
vectorbt==0.28.2
numba>=0.57,<0.61
llvmlite>=0.40,<0.44
```

2.4 Numba warm-up

First run of any `vectorbt` simulation triggers `Numba` JIT compilation. This takes 10–60 seconds. Subsequent calls in the same Python session use cached compiled functions and are instant. Add a warm-up call in your bot startup script:

```
# bot_startup.py
import vectorbt as vbt, pandas as pd, numpy as np

def warmup_vbt():
    """Force Numba JIT compilation at startup."""
    dummy = pd.Series(range(100), dtype=float)
    entries = pd.Series([True if i%10==0 else False for i in range(100)])
    exits = pd.Series([True if i%10==5 else False for i in range(100)])
    vbt.Portfolio.from_signals(dummy, entries, exits)
    print('vectorbt warmed up.')

warmup_vbt()
```

3. Core Concepts

3.1 How vectorbt works

vectorbt packs all backtesting logic into NumPy arrays and processes them with Numba-accelerated functions. Instead of a Python loop over bars, the entire simulation runs as a compiled C kernel. This lets you test thousands of parameter combinations in the time a traditional backtester takes for one.

Concept	Description
close (price)	The primary price Series driving valuation. Can be a DataFrame for multi-asset runs.
entries	Boolean Series/DataFrame. True at the bar where you want to enter a position.
exits	Boolean Series/DataFrame. True at the bar where you want to close.
Portfolio	Result object holding all simulation state: trades, equity curve, stats.
freq	Frequency string ('1D','5T','1H'). Required for annualised stats like Sharpe.
size=np.inf	Default: invest all available cash at entry. Change to float for fixed size.
fees	Fraction of trade value charged per fill. 0.001 = 0.1% (typical NSE brokerage).

3.2 Signal rules vectorbt enforces

- Entry and exit signals on the SAME bar: exit takes priority, then entry.
- Default: no pyramiding — a second entry while in a position is ignored.
- If entries and exits overlap on the same bar: treated as exit then re-entry.
- Stop orders (sl_stop, tp_stop, ts_stop) are checked against high/low of each bar.
- fillna=True on signals: vectorbt tolerates NaN signals (treats as False).

3.3 Vectorised vs event-driven simulation

Mode	When to use	Method
Vectorised (default)	Pre-computed boolean signal arrays. Fast. Good for strategy research.	Portfolio.from_signals(close, entries, exits)
Order-based	More control over order sizing, partial fills, dependencies between orders.	Portfolio.from_orders(close, size, ...)
Function-based	Full event-driven: custom logic per bar via Numba callback functions.	Portfolio.from_order_func(close, order_func_nb, ...)
Holding	Benchmark: buy at start, hold until end.	Portfolio.from_holding(close)

3.4 Multi-asset and multi-parameter runs

Pass a DataFrame of prices (one column per asset) and matching DataFrames for entries/exits. vectorbt broadcasts them and returns a Portfolio object with one result per column. Access individual results by column name.

```
# Multi-asset: one column per stock
prices = pd.DataFrame({'TCS': tcs_close, 'INFY': infy_close})
entries_df = pd.DataFrame({'TCS': tcs_entries, 'INFY': infy_entries})
exits_df = pd.DataFrame({'TCS': tcs_exits, 'INFY': infy_exits})

pf = vbt.Portfolio.from_signals(prices, entries_df, exits_df,
                               fees=0.001, freq='1D')

# Access individual columns
print(pf['TCS'].stats())
```

```
print(pf['INFY'].total_return())
```

4. Portfolio.from_signals() — Full Reference

The primary simulation method for QuantIQ strategies. Takes pre-computed boolean entry/exit arrays and simulates a portfolio.

4.1 Minimal working example

```
import vectorbt as vbt
import yfinance as yf
import pandas as pd
from ta.momentum import RSIIndicator
from ta.trend import SMAIndicator

# 1. Load data
df = yf.download('RELIANCE.NS', period='2y', auto_adjust=True)
close = df['Close']

# 2. Build signals
rsi = RSIIndicator(close, 14).rsi()
sma50 = SMAIndicator(close, 50).sma_indicator()
entries = (rsi < 30) & (close > sma50)
exits = rsi > 70

# 3. Simulate
pf = vbt.Portfolio.from_signals(
    close, entries, exits,
    init_cash=100_000,
    fees=0.001,
    slippage=0.001,
    freq='1D'
)

# 4. Results
print(pf.stats())
pf.plot().show()
```

4.2 All parameters — from_signals()

Parameter	Type	Default	Notes
close	Series/DF	required	Primary valuation price
entries	bool Series/DF	required	Long entry signals
exits	bool Series/DF	required	Long exit signals
short_entries	bool Series/DF	None	Short entry signals (direction='both')
short_exits	bool Series/DF	None	Short exit signals
open	Series/DF	None	Open price; if None, close used for all fills
high	Series/DF	None	High price; needed for stop-loss simulation
low	Series/DF	None	Low price; needed for stop-loss simulation
init_cash	float	100.0	Starting capital per column
size	float	np.inf	Order size in shares (or % if size_type set)
size_type	str	'amount'	'amount', 'value', 'percent', 'percent_of_equity'
fees	float	0.0	Fee per trade (fraction of value)
fixed_fees	float	0.0	Fixed cost per trade (in cash)
slippage	float	0.0	Slippage per trade (fraction of price)
min_size	float	0.0	Skip orders smaller than this quantity
max_size	float	np.inf	Cap order size at this quantity
reject_prob	float	0.0	Probability (0-1) of randomly rejecting an order

Parameter	Type	Default	Notes
lock_cash	bool	False	Prevent cash going below 0 on short side
sl_stop	float	None	Stop loss: fraction of entry price (0.05 = 5%)
sl_trail	bool	False	Make sl_stop a trailing stop
tp_stop	float	None	Take profit: fraction of entry price (0.10 = 10%)
ts_stop	float	None	Trailing stop: fraction from running peak
stop_entry_price	str	'close'	'open', 'close', 'slippage'
stop_exit_price	str	'stop'	'stop', 'close', 'open'
upon_stop_exit	str	'close'	'close', 'ignore', 'reverse'
direction	str	'longonly'	'longonly', 'shortonly', 'both'
freq	str	None	Bar frequency for annualised stats
cash_sharing	bool	False	Share cash pool across columns
group_by	array	None	Group columns for portfolio-level stats
call_seq	str	'default'	Order execution sequence across grouped columns
log	bool	False	Record detailed log of every order

4.3 Size and position sizing

```
# Default: invest ALL available cash at entry
pf = vbt.Portfolio.from_signals(close, entries, exits)

# Fixed number of shares
pf = vbt.Portfolio.from_signals(close, entries, exits, size=10)

# Fixed INR value per trade
pf = vbt.Portfolio.from_signals(close, entries, exits,
size=10_000, size_type='value')

# Percent of current equity (rebalancing style)
pf = vbt.Portfolio.from_signals(close, entries, exits,
size=0.95, size_type='percent_of_equity')

# ATR-based position sizing (pre-compute size Series)
atr = AverageTrueRange(df['High'], df['Low'], close, 14).average_true_range()
risk_per_trade = 0.01 * 100_000 # 1% of capital
size_series = (risk_per_trade / (2 * atr)).fillna(0).astype(int)
pf = vbt.Portfolio.from_signals(close, entries, exits,
size=size_series, size_type='amount')
```

4.4 Stop-loss and take-profit

```
# Fixed stop-loss (5%) and take-profit (15%)
pf = vbt.Portfolio.from_signals(
close, entries, exits,
high=df['High'], low=df['Low'],
sl_stop=0.05,
tp_stop=0.15,
stop_exit_price='close',
freq='1D'
)

# Trailing stop (3% from running high)
pf = vbt.Portfolio.from_signals(
close, entries, exits,
high=df['High'], low=df['Low'],
ts_stop=0.03,
freq='1D'
)

# ATR-based trailing stop (manual – more flexible)
atr = AverageTrueRange(df['High'], df['Low'], close, 14).average_true_range()
stop_pct = (2 * atr / close).clip(upper=0.10) # cap at 10%
pf = vbt.Portfolio.from_signals(
close, entries, exits,
high=df['High'], low=df['Low'],
sl_stop=stop_pct, sl_trail=True, freq='1D'
)
```

4.5 Fees and slippage for NSE

```
# NSE intraday typical costs:
# Brokerage (Dhan flat): 0 for equity delivery, Rs.20/order intraday
# STT (Securities Transaction Tax): 0.025% on sell
# Exchange transaction charge: ~0.00345%
# GST on brokerage: 18%
# Total round-trip estimate: ~0.05% to 0.15% of trade value

pf = vbt.Portfolio.from_signals(
close, entries, exits,
fees=0.001, # 0.1% per side as conservative estimate
slippage=0.001, # 0.1% slippage estimate
freq='1D'
)
```

5. Other Portfolio Constructors

5.1 from_holding() — Buy-and-hold benchmark

```
pf_bh = vbt.Portfolio.from_holding(close, init_cash=100_000, freq='1D')
print('Buy-and-hold return:', pf_bh.total_return())
```

Useful as the benchmark comparison column in `pf.stats()` (Benchmark Return is calculated from buy-and-hold on the same price data).

5.2 from_orders() — Explicit order sizing

More control than `from_signals()`. Pass a Series of signed sizes: positive = buy, negative = sell, 0 = do nothing.

```
import numpy as np

# Build order size array: +100 shares on entry, -100 on exit
size = pd.Series(0.0, index=close.index)
size[entries] = 100.0
size[exits] = -100.0

pf = vbt.Portfolio.from_orders(
    close, size,
    price=close, # fill price
    fees=0.001,
    init_cash=100_000,
    freq='1D'
)
```

5.3 from_random_signals() — Monte Carlo baseline

Generate random entry/exit signals and simulate. Useful to check if your strategy beats random luck.

```
# Run 100 random strategies, same number of trades
pf_random = vbt.Portfolio.from_random_signals(
    close, n=10, seed=42, fees=0.001, freq='1D'
)

# Compare your strategy against random baseline
your_return = pf.total_return()
random_returns = pf_random.total_return()
print(f'Your strategy: {your_return:.2%}')
print(f'Random mean: {random_returns.mean():.2%}')
```


6. Built-in Indicators (vbt.MA, vbt.RSI, etc.)

vectorbt wraps common indicators with a `.run()` interface that supports parameter broadcasting for sweeps. For QuantIQ's standard indicator pipeline, use the `ta` library (Module 3) instead — it's more complete. Use `vbt` indicators only for parameter sweeps.

6.1 Available built-in indicators

Indicator	Class	Key method
Moving Average	<code>vbt.MA</code>	<code>ma_crossed_above(slow)</code> , <code>ma_crossed_below(slow)</code>
Bollinger Bands	<code>vbt.BBANDS</code>	<code>percent_b</code> , <code>bandwidth</code> , <code>upper</code> , <code>lower</code>
RSI	<code>vbt.RSI</code>	<code>rsi</code>
Stochastic	<code>vbt.STOCH</code>	<code>percent_k</code> , <code>percent_d</code>
MACD	<code>vbt.MACD</code>	<code>macd</code> , <code>signal</code> , <code>histogram</code>
ATR	<code>vbt.ATR</code>	<code>atr</code>
OBV	<code>vbt.OBV</code>	<code>obv</code>

6.2 Single run

```
fast_ma = vbt.MA.run(close, window=10)
slow_ma = vbt.MA.run(close, window=50)

entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(close, entries, exits, freq='1D')
```

6.3 Parameter sweep with `run_combs()`

`run_combs()` generates all combinations of two parameter sets — ideal for exhaustive window optimisation.

```
import numpy as np

windows = np.arange(5, 51, 5) # [5, 10, 15, ..., 50]

# All fast/slow window combinations
fast_ma, slow_ma = vbt.MA.run_combs(
    close, window=windows, r=2,
    short_names=['fast', 'slow']
)

entries = fast_ma.ma_crossed_above(slow_ma)
exits = fast_ma.ma_crossed_below(slow_ma)

pf = vbt.Portfolio.from_signals(
    close, entries, exits,
    size=np.inf, fees=0.001, freq='1D'
)

# Best total return
best = pf.total_return().idxmax()
print(f'Best: fast={best[0]}, slow={best[1]}')
```

7. Performance Stats — Full Reference

7.1 pf.stats() — Summary object

```
# Full stats
print(pf.stats())

# Single metric
print(pf.sharpe_ratio())

# Multi-column: returns a Series
print(pf.total_return())
```

7.2 Portfolio-level metrics

Method	Returns
<code>pf.total_return()</code>	Fractional total return (0.5 = 50%)
<code>pf.annualized_return()</code>	CAGR (requires freq)
<code>pf.sharpe_ratio()</code>	Annualised Sharpe (requires freq)
<code>pf.sortino_ratio()</code>	Downside Sharpe (requires freq)
<code>pf.calmar_ratio()</code>	Annual return / max drawdown
<code>pf.omega_ratio()</code>	Omega ratio
<code>pf.max_drawdown()</code>	Max drawdown as fraction (e.g. -0.25 = -25%)
<code>pf.max_drawdown_duration()</code>	Duration of worst drawdown (Timedelta)
<code>pf.profit_factor()</code>	Gross profit / gross loss
<code>pf.expectancy()</code>	Average expected P&L; per trade
<code>pf.total_profit()</code>	Absolute profit in cash
<code>pf.final_value()</code>	Portfolio value at end
<code>pf.value()</code>	Time series of portfolio value
<code>pf.returns()</code>	Time series of period returns
<code>pf.benchmark_return()</code>	Buy-and-hold return on same price
<code>pf.total_fees_paid()</code>	Total fees paid in cash

7.3 Trade-level analysis

Method/Property	Returns
<code>pf.trades.records_readable</code>	DataFrame of all trades with human-readable columns
<code>pf.trades.count()</code>	Total number of closed trades
<code>pf.trades.win_rate()</code>	Win rate fraction
<code>pf.trades.pnl.mean()</code>	Average P&L; per trade
<code>pf.trades.return_.mean()</code>	Average % return per trade
<code>pf.trades.duration.mean()</code>	Average trade duration
<code>pf.orders.records_readable</code>	DataFrame of all orders placed
<code>pf.positions.records_readable</code>	DataFrame of positions (open + closed)
<code>pf.drawdowns.records_readable</code>	DataFrame of drawdown periods
<code>pf.drawdowns.max_drawdown()</code>	Max drawdown per column

7.4 Accessing time series

```
equity = pf.value() # portfolio equity curve (pd.Series/DataFrame)
returns = pf.returns() # period returns
cum_ret = pf.cumulative_returns()
cash = pf.cash() # uninvested cash over time
assets = pf.assets() # asset value (invested) over time
gross_exp = pf.gross_exposure() # % of portfolio in positions

# Export to CSV
equity.to_csv('equity_curve.csv')
```

7.5 Walk-forward analysis

```
# Split into 5 equal periods
close_splits, split_labels = close.vbt.range_split(n=5)

# Run strategy on each split
entries_splits = entries[close_splits.index]
exits_splits = exits[close_splits.index]

pf_splits = vbt.Portfolio.from_signals(
    close_splits, entries_splits, exits_splits,
    fees=0.001, freq='1D'
)

print(pf_splits.sharpe_ratio()) # Sharpe per split period
```

8. Plotting

vectorbt uses Plotly under the hood. All `.plot()` calls return a Plotly figure that you can display with `fig.show()` or export.

8.1 Portfolio overview plot

```
# Full portfolio chart: equity, drawdown, positions
fig = pf.plot()
fig.show()

# Save as HTML
fig.write_html('backtest_results.html')

# Save as PNG (requires kaleido)
fig.write_image('backtest_results.png', width=1200, height=800)
```

8.2 Individual plot methods

Method	What it shows
<code>pf.plot_value()</code>	Portfolio equity curve over time
<code>pf.plot_returns()</code>	Period returns as bar chart
<code>pf.plot_cumulative_returns()</code>	Cumulative return vs benchmark
<code>pf.plot_drawdowns()</code>	Drawdown periods shaded on equity curve
<code>pf.plot_gross_exposure()</code>	% capital deployed over time
<code>pf.trades.plot_pnl()</code>	P&L; per trade as scatter
<code>pf.trades.plot_returns()</code>	Trade return distribution
<code>pf.orders.plot()</code>	Buy/sell markers on price chart

8.3 Heatmap for parameter sweep results

```
# After running run_combs() sweep:
fig = pf.total_return().vbt.heatmap(
    x_level='fast_window',
    y_level='slow_window',
    symmetric=True,
    trace_kwargs=dict(colorbar=dict(title='Total Return', tickformat='%'))
)
fig.show()
```

8.4 Custom Plotly overlay with trade markers

```
import plotly.graph_objects as go

fig = go.Figure()

# Price candles
fig.add_trace(go.Scatter(x=close.index, y=close, name='Close', line_color='#555'))

# Entry markers
entry_dates = close.index[entries]
fig.add_trace(go.Scatter(
    x=entry_dates, y=close[entries],
    mode='markers', name='Entry',
    marker=dict(symbol='triangle-up', size=10, color='green')
))

# Exit markers
exit_dates = close.index[exits]
fig.add_trace(go.Scatter(
```

```
x=exit_dates, y=close[exits],  
mode='markers', name='Exit',  
marker=dict(symbol='triangle-down', size=10, color='red')  
)  
fig.show()
```

9. Parameter Sweeps & Optimisation

vectorbt's core advantage: run thousands of parameter combinations in seconds via array broadcasting.

9.1 RSI window optimisation

```
import numpy as np, vectorbt as vbt
from ta.momentum import RSIIndicator

# Test RSI windows from 5 to 30
windows = np.arange(5, 31)
rsi_matrix = pd.DataFrame(
    {w: RSIIndicator(close, w).rsi() for w in windows}
)

entries_matrix = rsi_matrix < 30
exits_matrix = rsi_matrix > 70

pf = vbt.Portfolio.from_signals(
    close, entries_matrix, exits_matrix,
    fees=0.001, freq='1D'
)

results = pd.DataFrame({
    'sharpe': pf.sharpe_ratio(),
    'return': pf.total_return(),
    'drawdown': pf.max_drawdown(),
    'n_trades': pf.trades.count(),
}, index=windows)

print(results.sort_values('sharpe', ascending=False).head(5))
```

9.2 Grid search: RSI thresholds

```
# Vary oversold level (20 to 40) and overbought level (60 to 80)
os_levels = np.arange(20, 41, 5) # [20, 25, 30, 35, 40]
ob_levels = np.arange(60, 81, 5) # [60, 65, 70, 75, 80]

rsi14 = RSIIndicator(close, 14).rsi()

results = {}
for os in os_levels:
    for ob in ob_levels:
        ent = rsi14 < os
        ex = rsi14 > ob
        pf = vbt.Portfolio.from_signals(close, ent, ex,
            fees=0.001, freq='1D')
        results[(os, ob)] = {
            'sharpe': pf.sharpe_ratio(),
            'return': pf.total_return(),
        }

results_df = pd.DataFrame(results).T
results_df.index.names = ['oversold', 'overbought']
print(results_df.sort_values('sharpe', ascending=False).head(10))
```

10. QuantIQ Backtest Patterns

10.1 Full strategy pipeline — ta + vectorbt

```
import vectorbt as vbt
import yfinance as yf
import pandas as pd
from ta.utils import dropna
from ta.momentum import RSIIndicator, MACD
from ta.volatility import BollingerBands, AverageTrueRange
from ta.trend import SMAIndicator, ADXIndicator

def backtest_strategy(symbol: str, period: str = '3y') -> vbt.Portfolio:
    # 1. Data
    df = yf.download(symbol, period=period, auto_adjust=True, progress=False)
    df = dropna(df)
    close, high, low = df['Close'], df['High'], df['Low']

    # 2. Indicators
    rsi = RSIIndicator(close, 14).rsi()
    sma50 = SMAIndicator(close, 50).sma_indicator()
    adx = ADXIndicator(high, low, close, 14).adx()
    macd = MACD(close)
    bb = BollingerBands(close, 20, 2)
    atr = AverageTrueRange(high, low, close, 14).average_true_range()

    # 3. Signals
    trend_up = (close > sma50) & (adx > 25)
    rsi_entry = (rsi > 50) & (rsi.shift(1) <= 50)
    macd_bull = macd.macd() > macd.macd_signal()
    entries = trend_up & rsi_entry & macd_bull
    exits = (rsi > 70) | (close < bb.bollinger_lband())

    # 4. Position sizing (ATR-based)
    risk_amt = 100_000 * 0.01 # 1% risk per trade
    size = (risk_amt / (2 * atr)).clip(lower=1).astype(int)

    # 5. Simulate
    return vbt.Portfolio.from_signals(
        close, entries, exits,
        open=df['Open'], high=high, low=low,
        size=size, size_type='amount',
        init_cash=100_000,
        fees=0.001, slippage=0.001,
        sl_stop=0.05, sl_trail=True,
        freq='1D'
    )

pf = backtest_strategy('RELIANCE.NS')
print(pf.stats())
```

10.2 Multi-stock comparison

```
symbols = ['RELIANCE.NS', 'TCS.NS', 'INFY.NS', 'HDFCBANK.NS', 'ICICIBANK.NS']

results = {}
for sym in symbols:
    pf = backtest_strategy(sym)
    results[sym] = {
        'Total Return': pf.total_return(),
        'Sharpe': pf.sharpe_ratio(),
        'Max DD': pf.max_drawdown(),
        'Win Rate': pf.trades.win_rate(),
        'Trades': pf.trades.count(),
    }

summary = pd.DataFrame(results).T
```

```
print(summary.sort_values('Sharpe', ascending=False))
```


11. See Also

Resource	URL
vectorbt docs	vectorbt.dev
vectorbt GitHub (open-source 0.28.x)	github.com/polakowo/vectorbt
vectorbt PyPI	pypi.org/project/vectorbt/
Python 3.13 compatibility issue	github.com/polakowo/vectorbt/issues/807
vectorbt usage guide	vectorbt.dev/getting-started/usage/
vectorbt API: portfolio.base	vectorbt.dev/api/portfolio/base/
vectorbt features overview	vectorbt.dev/getting-started/features/

Cross-references to other binder modules

Module	Relevance
M2 — Python Data Stack	pandas DataFrames as input; numpy for signal arrays
M3 — ta library	Build indicators and signals that feed into <code>from_signals()</code>
M1 — DhanHQ	Use DhanHQ historical API for production-quality backtest data
M6 — Plotly	Extend vectorbt plots or build standalone charts from <code>pf.value()</code>
M7 — Python Standards	Type-annotate strategy functions: <code>close: pd.Series</code> , <code>freq: str</code>

End of Module 4. Next: Module 5 — Dashboard (Streamlit).

QuantIQ

Master Reference Binder

Module 5

Dashboard — Streamlit

Version	18 May 2026
Library	streamlit — pip install streamlit
Scope	Core API, widgets, layout, caching, secrets, deployment
Critical	Never commit .streamlit/secrets.toml to git
Pages	approx 28

5

Module Contents

§	Section	Page
1	Quick Reference Card	3
2	How Streamlit Works	5
3	Text & Display Elements	7
4	Data Display	9
5	Charts	11
6	Input Widgets	13
7	Layout — Columns, Tabs, Sidebar, Expander	17
8	State Management	20
9	Caching	22
10	Secrets Management (Dhan credentials)	24
11	Deployment — Community Cloud	26
12	QuantIQ Dashboard Pattern	27
13	See Also	29

1. Quick Reference Card

```
pip install streamlit
streamlit run app.py
```

All core functions at a glance

Function	Purpose
<code>st.title('text')</code>	Large page title
<code>st.header('text')</code>	Section header (H1)
<code>st.subheader('text')</code>	Sub-section header (H2)
<code>st.write(value)</code>	Smart display: text, df, dict, fig, anything
<code>st.markdown('**bold**')</code>	Markdown-formatted text
<code>st.code('snippet', language='python')</code>	Syntax-highlighted code block
<code>st.caption('text')</code>	Small grey caption text
<code>st.latex(r'E=mc^2')</code>	LaTeX equation
<code>st.divider()</code>	Horizontal rule
<code>st.dataframe(df)</code>	Interactive scrollable DataFrame table
<code>st.table(df)</code>	Static full-width table
<code>st.metric('label', value, delta)</code>	KPI card with delta arrow
<code>st.json(data)</code>	Collapsible JSON viewer
<code>st.line_chart(df)</code>	Simple line chart
<code>st.area_chart(df)</code>	Area chart
<code>st.bar_chart(df)</code>	Bar chart
<code>st.plotly_chart(fig)</code>	Interactive Plotly figure
<code>st.button('Click me')</code>	Button — returns True when clicked
<code>st.selectbox('label', options)</code>	Dropdown selector
<code>st.multiselect('label', options)</code>	Multi-select dropdown
<code>st.slider('label', min, max, value)</code>	Numeric slider
<code>st.text_input('label')</code>	Single-line text input
<code>st.number_input('label', min_value=0)</code>	Numeric input
<code>st.date_input('label')</code>	Date picker
<code>st.checkbox('label')</code>	Checkbox — returns bool
<code>st.radio('label', options)</code>	Radio button group
<code>st.file_uploader('label', type=['csv'])</code>	File upload widget
<code>col1, col2 = st.columns(2)</code>	Two equal-width columns
<code>col1, col2 = st.columns([2, 1])</code>	Weighted columns (2:1 ratio)
<code>tab1, tab2 = st.tabs(['A', 'B'])</code>	Tab containers
<code>st.sidebar.selectbox(...)</code>	Put any widget in the sidebar
<code>with st.expander('Show more'):</code>	Collapsible section
<code>with st.container():</code>	Logical grouping container
<code>st.empty()</code>	Placeholder — update content later
<code>st.success('Done!')</code>	Green success banner
<code>st.error('Failed!')</code>	Red error banner
<code>st.warning('Check this')</code>	Yellow warning banner

Function	Purpose
<code>st.info('FYI')</code>	Blue info banner
<code>st.spinner('Loading...')</code>	Loading spinner context manager
<code>st.progress(value)</code>	Progress bar (0.0–1.0)
<code>st.session_state['key']</code>	Per-session persistent state dict
<code>@st.cache_data</code>	Cache function returning data (DataFrame, etc.)
<code>@st.cache_resource</code>	Cache shared resource (connection, model)
<code>st.rerun()</code>	Force script rerun from top
<code>st.secrets['DHAN_CLIENT_ID']</code>	Read from <code>.streamlit/secrets.toml</code>

2. How Streamlit Works

Official docs: docs.streamlit.io | API reference: docs.streamlit.io/develop/api-reference

2.1 The rerun model

Every user interaction re-executes the entire Python script from top to bottom. This is by design — it keeps state management simple. The implications:

- Any variable defined outside a function is recomputed on every interaction.
- Use `@st.cache_data` for expensive computations (data loading, API calls).
- Use `@st.cache_resource` for one-time initialisation (connections, models).
- Use `st.session_state` to persist values across reruns within a session.
- Widget values are automatically preserved between reruns by Streamlit.

2.2 App structure

```
import streamlit as st
import pandas as pd
import plotly.express as px

# Page config (must be FIRST st call)
st.set_page_config(
    page_title='QuantIQ Dashboard',
    page_icon='chart_with_upwards_trend',
    layout='wide', # 'centered' (default) or 'wide'
    initial_sidebar_state='expanded' # or 'collapsed'
)

# Title and header
st.title('QuantIQ Trading Dashboard')
st.caption('NSE Algorithmic Trading | QuantIQ v1.0')

# Sidebar controls
with st.sidebar:
    symbol = st.selectbox('Symbol', ['RELIANCE.NS', 'TCS.NS', 'INFY.NS'])
    period = st.selectbox('Period', ['1mo', '3mo', '6mo', '1y', '2y'])

# Main content
st.header(f'{symbol} Analysis')
# ... rest of app
```

2.3 Running locally

```
# From project root
streamlit run dashboard/app.py

# With custom port
streamlit run dashboard/app.py --server.port 8502

# Disable file watcher (useful in Docker)
streamlit run dashboard/app.py --server.fileWatcherType none
```

2.4 File structure for QuantIQ

```
quantiq/
dashboard/
app.py # Main Streamlit entry point
pages/ # Multi-page: each .py = one page
01_overview.py
02_backtest.py
```

```
03_live.py
.streamlit/
secrets.toml # NEVER commit this file
config.toml # Theme and server config (safe to commit)
requirements.txt
```

3. Text & Display Elements

3.1 Text functions

Function	Notes
<code>st.title('QuantIQ Dashboard')</code>	Large title at top of page
<code>st.header('Market Overview')</code>	Section header — large
<code>st.subheader('Nifty 50')</code>	Sub-section header — medium
<code>st.write('Hello')</code>	Auto-detect type. Works for str, int, df, fig, etc.
<code>st.markdown('**bold** _italic_')</code>	Markdown. Supports :green[color], emoji, HTML disabled.
<code>st.caption('Updated: 09:15 IST')</code>	Small muted text for metadata, footnotes
<code>st.code('pip install ta', language='bash')</code>	Syntax-highlighted code block
<code>st.latex(r'\frac{P_t - P_{t-1}}{P_{t-1}}')</code>	LaTeX equation rendering
<code>st.divider()</code>	Horizontal separator line
<code>st.text('plain text')</code>	Plain monospace text (no markdown)

3.2 Feedback / status elements

Function	When to use
<code>st.success('Order placed: RELIANCE.NS BUY 100')</code>	Successful operation
<code>st.error('API token expired. Regenerate before 09:15.')</code>	Error state
<code>st.warning('Approaching rate limit: 8/10 orders/sec')</code>	Warning
<code>st.info('Market opens at 09:15 IST')</code>	Informational note
<code>st.exception(e)</code>	Display exception with traceback
<code>with st.spinner('Fetching data...'): data = fetch()</code>	Loading state
<code>st.progress(0.75, text='Loading...')</code>	Progress bar (0.0 to 1.0)
<code>st.toast('Token refreshed')</code>	Brief notification toast (bottom-right)
<code>st.balloons()</code>	Celebrate on success (animated balloons)

3.3 Media

```
st.image('logo.png', width=200)
st.image(pil_image, caption='Chart')
st.image('https://example.com/chart.png')

# Plotly figure as image (use st.plotly_chart instead for interactivity)
import plotly.express as px
fig = px.line(df, x='Date', y='Close')
st.plotly_chart(fig, use_container_width=True)
```


4. Data Display

4.1 DataFrame display

```
# Interactive (sortable, scrollable, searchable)
st.dataframe(df, use_container_width=True)

# With column config (v1.20+)
st.dataframe(
    df,
    column_config={
        'Close': st.column_config.NumberColumn(
            'Close Price (INR)', format='%.2f'
        ),
        'Returns': st.column_config.NumberColumn(
            'Daily Return', format='%.2%'
        ),
    },
    hide_index=True,
    use_container_width=True
)

# Static non-interactive table (full width, no scroll)
st.table(df.tail(10))
```

4.2 Metrics (KPI cards)

`st.metric()` shows a large value with a delta arrow. Essential for trading dashboards.

```
col1, col2, col3, col4 = st.columns(4)

col1.metric(
    label='RELIANCE.NS',
    value='■1,365.50',
    delta='+1.62%',
)
col2.metric(
    label='Portfolio Value',
    value='■1,24,320',
    delta='■2,410',
    delta_color='normal' # 'normal', 'inverse', 'off'
)
col3.metric('Sharpe Ratio', '1.42', '+0.08')
col4.metric('Max Drawdown', '-8.3%', '-0.2%', delta_color='inverse')
```

4.3 JSON and dict display

```
# Collapsible JSON viewer
st.json({'symbol': 'RELIANCE.NS', 'price': 1365.50, 'volume': 2_345_000})

# Expanded by default
st.json({'key': 'value'}, expanded=True)
```

5. Charts

For QuantIQ use `st.plotly_chart()` exclusively — Plotly gives interactive zoom, hover, range selectors, and candlestick support. Native `st.line_chart()` is fine for quick exploration.

5.1 Plotly charts (recommended)

```
import plotly.express as px
import plotly.graph_objects as go

# Line chart
fig = px.line(df, x=df.index, y='Close', title='RELIANCE.NS Close')
st.plotly_chart(fig, use_container_width=True)

# Candlestick chart
fig = go.Figure(data=[go.Candlestick(
    x=df.index,
    open=df['Open'], high=df['High'],
    low=df['Low'], close=df['Close'],
    name='RELIANCE.NS'
)])
fig.update_layout(
    title='RELIANCE.NS',
    xaxis_rangeslider_visible=False,
    height=500
)
st.plotly_chart(fig, use_container_width=True)

# Multi-line (portfolio equity + benchmark)
fig = px.line(pf_df, x='Date', y=['Portfolio', 'BuyHold'],
    title='Portfolio vs Buy-and-Hold',
    labels={'value': 'Value (INR)'})
st.plotly_chart(fig, use_container_width=True)
```

5.2 Native Streamlit charts (quick prototyping)

Function	Use for
<code>st.line_chart(df)</code>	Quick line chart from DataFrame
<code>st.area_chart(df)</code>	Cumulative fills under line
<code>st.bar_chart(df)</code>	Vertical bars
<code>st.scatter_chart(df, x='Returns', y='Volume')</code>	Scatter (v1.26+)

```
# Quick returns chart
st.line_chart(df[['Close', 'SMA20', 'SMA50']])

# RSI chart
st.line_chart(df['RSI'])
```

5.3 Combining indicator subplots with Plotly

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

fig = make_subplots(
    rows=3, cols=1,
    shared_xaxes=True,
    row_heights=[0.6, 0.2, 0.2],
    vertical_spacing=0.03
)

# Price + BB
fig.add_trace(go.Scatter(x=df.index, y=df['Close'], name='Close'), row=1, col=1)
fig.add_trace(go.Scatter(x=df.index, y=df['BB_upper'], name='Upper BB',
```

```
line=dict(dash='dash', color='gray')), row=1, col=1)
fig.add_trace(go.Scatter(x=df.index, y=df['BB_lower'], name='Lower BB',
line=dict(dash='dash', color='gray')), row=1, col=1)

# Volume
fig.add_trace(go.Bar(x=df.index, y=df['Volume'], name='Volume'), row=2, col=1)

# RSI
fig.add_trace(go.Scatter(x=df.index, y=df['RSI'], name='RSI'), row=3, col=1)
fig.add_hline(y=70, line_dash='dot', line_color='red', row=3, col=1)
fig.add_hline(y=30, line_dash='dot', line_color='green', row=3, col=1)

fig.update_layout(height=700, showlegend=True)
st.plotly_chart(fig, use_container_width=True)
```

6. Input Widgets

6.1 Button

```
if st.button('Refresh Data', type='primary'):
    st.cache_data.clear() # clear cache to force re-fetch
    st.rerun()

# Button types: 'primary' (filled), 'secondary' (outline, default), 'tertiary'
```

6.2 Selectbox & Multiselect

```
# Single selection
symbol = st.selectbox(
    'Select Stock',
    options=['RELIANCE.NS', 'TCS.NS', 'INFY.NS', 'HDFCBANK.NS'],
    index=0, # default selection
    key='symbol_select' # unique key for session state
)

# Multi-selection
symbols = st.multiselect(
    'Compare Stocks',
    options=['RELIANCE.NS', 'TCS.NS', 'INFY.NS', 'HDFCBANK.NS'],
    default=['RELIANCE.NS', 'TCS.NS']
)
```

6.3 Slider

```
# Integer slider
rsi_window = st.slider('RSI Window', min_value=5, max_value=30,
    value=14, step=1)

# Float slider
sl_pct = st.slider('Stop-Loss %', min_value=0.01, max_value=0.10,
    value=0.05, step=0.01, format='%.0f%%')

# Range slider (returns tuple)
low, high = st.slider('RSI Entry/Exit', 0, 100, (30, 70))
```

6.4 Text inputs

```
# Single-line
symbol = st.text_input('Custom Symbol', value='RELIANCE.NS',
    placeholder='e.g. TCS.NS')

# Multi-line
notes = st.text_area('Trade Notes', height=100)

# Numeric
capital = st.number_input('Starting Capital (INR)',
    min_value=10_000,
    max_value=10_000_000,
    value=100_000,
    step=10_000)
```

6.5 Date input

```
import datetime

start = st.date_input('Start Date',
    value=datetime.date(2024, 1, 1),
    min_value=datetime.date(2020, 1, 1),
    max_value=datetime.date.today())
end = st.date_input('End Date', value=datetime.date.today())
```

```
if start >= end:
    st.error('Start date must be before end date.')
```

6.6 Checkbox and Radio

```
# Checkbox
show_indicators = st.checkbox('Show Indicators', value=True)
if show_indicators:
    st.plotly_chart(indicator_fig)

# Radio
timeframe = st.radio('Timeframe', ['Daily', 'Weekly', 'Monthly'],
horizontal=True) # horizontal layout
```

6.7 File uploader

```
uploaded = st.file_uploader(
    'Upload CSV (OHLCV format)',
    type=['csv'],
    accept_multiple_files=False
)

if uploaded is not None:
    df = pd.read_csv(uploaded, parse_dates=['Date'], index_col='Date')
    st.success(f'Loaded {len(df)} rows')
    st.dataframe(df.head())
```

6.8 Forms (batch inputs)

Forms group widgets — only submitted on button press, preventing a rerun for every widget change.

```
with st.form('strategy_params'):
    st.subheader('Strategy Parameters')
    symbol = st.selectbox('Symbol', ['RELIANCE.NS', 'TCS.NS'])
    rsi_win = st.slider('RSI Window', 5, 30, 14)
    sl_pct = st.slider('Stop-Loss %', 1, 15, 5)
    submitted = st.form_submit_button('Run Backtest', type='primary')

    if submitted:
        with st.spinner('Running backtest...'):
            results = run_backtest(symbol, rsi_win, sl_pct/100)
            st.success('Backtest complete')
            st.dataframe(results)
```

7. Layout — Columns, Tabs, Sidebar, Expander

7.1 Columns

```
# Equal columns
col1, col2, col3 = st.columns(3)
with col1:
    st.metric('Nifty 50', '24,856', '+1.2%')
with col2:
    st.metric('Bank Nifty', '56,320', '+0.8%')
with col3:
    st.metric('Sensex', '81,500', '+1.1%')

# Weighted columns [left=2, right=1]
chart_col, ctrl_col = st.columns([2, 1])
with chart_col:
    st.plotly_chart(price_fig, use_container_width=True)
with ctrl_col:
    st.selectbox('Indicator', ['SMA', 'EMA', 'BB'])
```

7.2 Tabs

```
tab_chart, tab_stats, tab_trades = st.tabs(
    ['Price Chart', 'Statistics', 'Trades']
)

with tab_chart:
    st.plotly_chart(price_fig, use_container_width=True)

with tab_stats:
    st.dataframe(stats_df)

with tab_trades:
    st.dataframe(trades_df, use_container_width=True)
```

7.3 Sidebar

```
# Method 1: with block (recommended)
with st.sidebar:
    st.title('Controls')
    symbol = st.selectbox('Symbol', SYMBOLS)
    period = st.selectbox('Period', ['1mo', '3mo', '6mo', '1y'])
    st.divider()
    if st.button('Refresh', type='primary'):
        st.cache_data.clear()
        st.rerun()

# Method 2: st.sidebar.widget() inline
symbol = st.sidebar.selectbox('Symbol', SYMBOLS)
```

7.4 Expander

```
with st.expander('Show raw data', expanded=False):
    st.dataframe(df, use_container_width=True)

with st.expander('Backtest parameters', expanded=True):
    st.write(f'RSI window: {rsi_window}')
    st.write(f'Stop-loss: {sl_pct:.1%}')
```

7.5 Container and Empty (dynamic updates)

```
# Container: group elements, update all at once
status_box = st.empty() # single-element placeholder
status_box.info('Waiting for market open...')

# Later in code, replace content:
status_box.success('Market open. Bot running.')

# Container for multiple elements (replace all at once)
display = st.container()
with display:
    st.metric('P&L', pnl)
    st.dataframe(positions)
```

8. State Management

`st.session_state` is a dict-like object per user session. Values survive reruns within the same browser tab. Reset when the page refreshes.

8.1 Basic session state

```
import streamlit as st

# Initialise (check first to avoid overwriting on rerun)
if 'trade_count' not in st.session_state:
    st.session_state.trade_count = 0

if 'positions' not in st.session_state:
    st.session_state.positions = []

# Increment on button click
if st.button('Place Order'):
    st.session_state.trade_count += 1
    st.session_state.positions.append({'symbol': 'RELIANCE.NS'})

st.write(f'Orders placed this session: {st.session_state.trade_count}')
```

8.2 Widget keys and session state

Giving a widget a key automatically links it to `session_state`:

```
symbol = st.selectbox('Symbol', SYMBOLS, key='selected_symbol')

# Access the same value via session state
print(st.session_state.selected_symbol) # same as symbol

# Programmatically set a widget value via session state
# (Must be done BEFORE the widget call, not after)
if 'selected_symbol' not in st.session_state:
    st.session_state.selected_symbol = 'TCS.NS'
```

8.3 Callbacks

```
def on_symbol_change():
    """Called when symbol selectbox changes."""
    st.session_state.last_symbol = st.session_state.symbol_key
    # Clear cached data for old symbol
    load_price_data.clear()

symbol = st.selectbox(
    'Symbol',
    SYMBOLS,
    key='symbol_key',
    on_change=on_symbol_change
)
```


9. Caching

Caching is critical in Streamlit because the entire script reruns on every interaction. Without caching, every widget change triggers an API call, data download, or backtest.

9.1 @st.cache_data — for data-returning functions

Use for: DataFrames, dicts, lists, API responses, backtest results. Each caller gets its own copy of the cached data.

```
import streamlit as st
import yfinance as yf

@st.cache_data(ttl='1h') # cache expires after 1 hour
def load_price_data(symbol: str, period: str) -> pd.DataFrame:
    df = yf.download(symbol, period=period, auto_adjust=True, progress=False)
    return df

@st.cache_data(ttl='5m') # refresh every 5 minutes for quasi-live
def get_latest_price(symbol: str) -> float:
    df = yf.download(symbol, period='1d', interval='1m', progress=False)
    return float(df['Close'].iloc[-1])

# Usage (Streamlit skips the function body if args match cached call)
df = load_price_data('RELIANCE.NS', '1y')
price = get_latest_price('RELIANCE.NS')

# Manual cache clear (e.g., on Refresh button)
load_price_data.clear() # clear this function's cache
st.cache_data.clear() # clear ALL data caches
```

9.2 @st.cache_resource — for shared objects

Use for: database connections, API client instances, ML models. Shared across ALL users and sessions. Must be thread-safe.

```
from dhanhq import dhanhq

@st.cache_resource
def get_dhan_client() -> dhanhq:
    """Create one DhanHQ client shared across all sessions."""
    return dhanhq(
        st.secrets['DHAN_CLIENT_ID'],
        st.secrets['DHAN_ACCESS_TOKEN']
    )

dhan = get_dhan_client()

# Clear when token regenerated (e.g. daily refresh)
def refresh_token(new_token: str):
    st.secrets['DHAN_ACCESS_TOKEN'] = new_token # can't do this at runtime
    get_dhan_client.clear() # force re-init with new token
```

9.3 Cache comparison

Aspect	@st.cache_data	@st.cache_resource
Use for	DataFrames, dicts, lists, query results	Connections, clients, models
Shared between users?	No — each user gets own copy	Yes — one instance for all
Thread-safe requirement?	No	Yes
Return type	Must be pickleable	Any Python object
TTL support?	Yes — ttl='1h' or seconds	Yes

Aspect	@st.cache_data	@st.cache_resource
Unhashable args?	Use _ prefix to exclude	Use _ prefix to exclude

10. Secrets Management (Dhan credentials)

CRITICAL: Never commit `.streamlit/secrets.toml` to git. Never hardcode `DHAN_CLIENT_ID` or `DHAN_ACCESS_TOKEN` anywhere in code.

10.1 Local secrets file

Create `.streamlit/secrets.toml` in project root. Add to `.gitignore` immediately.

```
# .streamlit/secrets.toml
# DO NOT COMMIT THIS FILE

DHAN_CLIENT_ID = "1000000001"
DHAN_ACCESS_TOKEN = "eyJhbGciOiJIUzI1NiJ9..."

# Optional: group under a section
[dhan]
client_id = "1000000001"
access_token = "eyJhbGciOiJIUzI1NiJ9..."
```

10.2 .gitignore entry

```
# .gitignore
.streamlit/secrets.toml
.env
*.env
```

10.3 Accessing secrets in code

```
import streamlit as st

# Top-level keys
client_id = st.secrets['DHAN_CLIENT_ID']
access_token = st.secrets['DHAN_ACCESS_TOKEN']

# Section-based keys
client_id = st.secrets['dhan']['client_id']
access_token = st.secrets['dhan']['access_token']

# Using attribute notation
client_id = st.secrets.DHAN_CLIENT_ID

# Full DhanHQ client init with secrets
from dhanhq import dhanhq
dhan = dhanhq(st.secrets['DHAN_CLIENT_ID'],
st.secrets['DHAN_ACCESS_TOKEN'])
```

10.4 Community Cloud secrets (deployment)

When deploying to Streamlit Community Cloud, secrets are configured in the web UI — never in the repository.

1. Deploy app from GitHub repo (see §11)
2. Go to app Settings → Secrets
3. Paste the TOML content from your local `secrets.toml`
4. Save — the app will restart with secrets injected
5. Never put `secrets.toml` in the repo — Community Cloud will not read it from the repo

11. Deployment — Community Cloud

Streamlit Community Cloud is free for public repos and limited private repos. Docs: docs.streamlit.io/deploy/streamlit-community-cloud

11.1 Deploy steps

1. Push app to GitHub repo (Quant-IQ/quantiq)
2. Go to share.streamlit.io and sign in with GitHub
3. Click 'New app'
4. Select repo: Quant-IQ/quantiq | Branch: main | Main file path: dashboard/app.py
5. Click 'Advanced settings' → add secrets (paste secrets.toml content)
6. Click 'Deploy' — app goes live at `quantiq-[hash].streamlit.app`

11.2 requirements.txt for deployment

Community Cloud installs packages from requirements.txt at root. Pin exact versions for reproducibility.

```
# requirements.txt
streamlit>=1.35.0
pandas>=2.0.0
numpy>=1.24.0
yfinance>=0.2.40
ta==0.11.0
vectorbt==0.28.2
plotly>=5.20.0
dhanhq==2.1.0
```

11.3 config.toml (theme — safe to commit)

```
# .streamlit/config.toml
[theme]
primaryColor = '#3FB950' # QuantIQ green
backgroundColor = '#0D1117' # dark background
secondaryBackgroundColor = '#161B22' # surface
textColor = '#E6EDF3' # light text
font = 'sans-serif'

[server]
runOnSave = true # auto-reload on file save (local dev only)
```

11.4 Resource limits (Community Cloud free tier)

Resource	Limit
RAM	1 GB
CPU	Shared (burst available)
Disk	800 MB
Inactive sleep	App sleeps after 7 days of inactivity
Wakeup	App wakes on first visit (20–30 seconds)
Custom domain	Not supported on free tier
Private apps	Limited number on free tier

12. QuantIQ Dashboard Pattern

```
# dashboard/app.py
import streamlit as st
import pandas as pd
import plotly.graph_objects as go
import yfinance as yf
from ta.utils import dropna
from ta.momentum import RSIIndicator
from ta.volatility import BollingerBands

# ■■■ Page config
st.set_page_config(page_title='QuantIQ', page_icon='chart_with_upwards_trend',
layout='wide')

# ■■■ Sidebar
with st.sidebar:
    st.title('QuantIQ')
    symbol = st.selectbox('Symbol',
        ['RELIANCE.NS', 'TCS.NS', 'INFY.NS', 'HDFCBANK.NS'])
    period = st.selectbox('Period', ['3mo', '6mo', '1y', '2y'])
    rsi_w = st.slider('RSI Window', 5, 30, 14)
    if st.button('Refresh', type='primary'):
        load_data.clear(); st.rerun()

# ■■■ Data loading (cached)
@st.cache_data(ttl='30m')
def load_data(sym, per):
    df = yf.download(sym, period=per, auto_adjust=True, progress=False)
    df = dropna(df)
    df['RSI'] = RSIIndicator(df['Close'], 14).rsi()
    bb = BollingerBands(df['Close'], 20, 2)
    df['BB_u'] = bb.bollinger_hband()
    df['BB_l'] = bb.bollinger_lband()
    return df

df = load_data(symbol, period)

# ■■■ KPI row
last = df['Close'].iloc[-1]
prev = df['Close'].iloc[-2]
chg = (last - prev) / prev
k1, k2, k3, k4 = st.columns(4)
k1.metric(symbol, f'INR {last:,.2f}', f'{chg:+.2%}')
k2.metric('RSI', f"{df['RSI'].iloc[-1]:.1f}")
k3.metric('52W High', f"INR {df['High'].max():,.2f}")
k4.metric('52W Low', f"INR {df['Low'].min():,.2f}")

# ■■■ Chart
fig = go.Figure()
fig.add_trace(go.Scatter(x=df.index, y=df['Close'], name='Close'))
fig.add_trace(go.Scatter(x=df.index, y=df['BB_u'], name='Upper BB',
    line=dict(dash='dash', color='gray')))
fig.add_trace(go.Scatter(x=df.index, y=df['BB_l'], name='Lower BB',
    line=dict(dash='dash', color='gray')))
fig.update_layout(title=f'{symbol} Price', height=450)
st.plotly_chart(fig, use_container_width=True)

# ■■■ Tabs for details
t1, t2 = st.tabs(['Raw Data', 'Indicators'])
with t1:
    st.dataframe(df.tail(20), use_container_width=True)
with t2:
    st.line_chart(df[['RSI']])
```

13. See Also

Resource	URL
Streamlit docs	docs.streamlit.io
API reference	docs.streamlit.io/develop/api-reference
Secrets management	docs.streamlit.io/develop/concepts/connections/secrets-management
Caching	docs.streamlit.io/develop/concepts/architecture/caching
Session state	docs.streamlit.io/develop/api-reference/caching-and-state/st.session_state
Community Cloud deploy	docs.streamlit.io/deploy/streamlit-community-cloud
Community Cloud secrets	docs.streamlit.io/deploy/streamlit-community-cloud/deploy-your-app/secrets-management
<code>st.cache_data</code>	docs.streamlit.io/develop/api-reference/caching-and-state/st.cache_data
<code>st.cache_resource</code>	docs.streamlit.io/develop/api-reference/caching-and-state/st.cache_resource
GitHub	github.com/streamlit/streamlit
PyPI	pypi.org/project/streamlit/

Cross-references to other binder modules

Module	Relevance
M1 — DhanHQ	<code>st.secrets</code> for <code>DHAN_CLIENT_ID</code> / <code>DHAN_ACCESS_TOKEN</code> ; <code>@st.cache_resource</code> for dhan client
M2 — Python Data Stack	<code>st.dataframe(df)</code> for pandas display; load and transform data before displaying
M4 — vectorbt	<code>pf.value().to_frame()</code> for equity curve; <code>pf.stats()</code> in <code>st.dataframe()</code>
M6 — Plotly	<code>st.plotly_chart(fig)</code> for all interactive charts; candlestick + indicator subplots
M10 — Environment	<code>.streamlit/secrets.toml</code> in <code>.gitignore</code> ; <code>requirements.txt</code> for deployment

End of Module 5. Next: Module 6 — Charting (Plotly Python).

QuantIQ

Master Reference Binder

Module 6

Charting — Plotly Python

Version	18 May 2026
Library	plotly — pip install plotly
Scope	graph_objects, Plotly Express, subplots, layout, export
Focus	Financial charts: candlestick, OHLC, multi-panel indicator charts
Pages	approx 25

6

Module Contents

§	Section	Page
1	Quick Reference Card	3
2	px vs go — Which to Use	5
3	Candlestick Chart	7
4	OHLC Chart	10
5	Line and Scatter Charts	11
6	Bar Charts	13
7	Subplots — Multi-Panel Charts	14
8	Layout and Styling	17
9	Annotations, Shapes, and Reference Lines	20
10	Axes, Range, and Zoom	22
11	Export and Save	24
12	QuantIQ Standard Chart Patterns	25
13	See Also	28

1. Quick Reference Card

```
pip install plotly

import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
```

Trace types cheat sheet

Trace / function	Use for
<code>go.Candlestick(x, open, high, low, close)</code>	OHLC candlestick chart
<code>go.Ohlc(x, open, high, low, close)</code>	OHLC tick-mark chart (lighter)
<code>go.Scatter(x, y, mode='lines')</code>	Line chart
<code>go.Scatter(x, y, mode='markers')</code>	Scatter plot
<code>go.Scatter(x, y, mode='lines+markers')</code>	Line with point markers
<code>go.Bar(x, y)</code>	Vertical bar chart
<code>go.Histogram(x)</code>	Histogram
<code>go.Heatmap(z)</code>	2D heatmap (for correlation matrices)
<code>px.line(df, x='Date', y='Close')</code>	Quick line chart via Express
<code>px.bar(df, x='Date', y='Volume')</code>	Quick bar via Express
<code>px.scatter(df, x='x', y='y', color='col')</code>	Quick scatter via Express
<code>px.histogram(df, x='Returns')</code>	Quick histogram via Express

Figure structure

```
fig = go.Figure() # empty figure
fig.add_trace(go.Scatter(x=x, y=y)) # add trace
fig.update_layout(title='My Chart') # update layout
fig.update_xaxes(showgrid=True) # update x-axis
fig.update_yaxes(title_text='Price (INR)') # update y-axis
fig.show() # display in browser/Streamlit
fig.write_html('chart.html') # export to HTML
```

Most common `update_layout` kwargs

Kwarg	Type	Purpose
<code>title</code>	str / dict	Chart title: 'My Title' or dict(text='...', x=0.5)
<code>height</code>	int	Figure height in pixels (default 450)
<code>width</code>	int	Figure width in pixels
<code>showlegend</code>	bool	Show/hide legend
<code>legend</code>	dict	Legend position: dict(x=0.01, y=0.99)
<code>template</code>	str	Theme: 'plotly', 'plotly_dark', 'seaborn', 'none'
<code>xaxis_rangeslider_visible</code>	bool	Hide candlestick range slider (always False for QuantIQ)
<code>margin</code>	dict	dict(l=40, r=20, t=50, b=40) — pixel margins
<code>paper_bgcolor</code>	str	Outer background color: '#0D1117' for dark
<code>plot_bgcolor</code>	str	Plot area background: '#161B22' for dark
<code>font</code>	dict	dict(color='#E6EDF3', size=12) — global font
<code>hovermode</code>	str	'x unified', 'x', 'closest'

2. px vs go — Which to Use

	Plotly Express (px)	graph_objects (go)
Purpose	Quick exploratory charts	Production charts, full control
API style	One-line function calls	OOP: <code>add_trace()</code> , <code>update_layout()</code>
Multi-panel?	No (single figure)	Yes — use <code>make_subplots()</code>
Candlestick?	No	Yes — <code>go.Candlestick()</code>
Custom colours per candle?	No	Yes
Best for	Data exploration, quick prototypes	QuantIQ dashboard, financial charts

Rule for QuantIQ: use go for all financial charts. Use px only for quick internal analysis (returns distribution, correlation heatmap, etc.).

2.1 Plotly Express — basics

```
import plotly.express as px
import pandas as pd

# Line chart
fig = px.line(df, x=df.index, y='Close', title='Close Price')

# Multi-line
fig = px.line(df, x=df.index, y=['Close', 'SMA20', 'SMA50'],
title='Price with SMAs')

# Scatter
fig = px.scatter(df, x='Volume', y='Returns',
color='Direction', # colour by column
hover_data=['Close'], title='Volume vs Returns')

# Returns histogram
fig = px.histogram(df, x='Returns', nbins=50,
title='Daily Return Distribution')

# Correlation heatmap
fig = px.imshow(df.corr(), text_auto='.2f', title='Correlation Matrix')

fig.show()
```

2.2 graph_objects — basics

```
import plotly.graph_objects as go

fig = go.Figure()

fig.add_trace(go.Scatter(
x=df.index, y=df['Close'],
name='Close',
line=dict(color='#3FB950', width=1.5)
))

fig.add_trace(go.Scatter(
x=df.index, y=df['SMA20'],
name='SMA 20',
line=dict(color='#58A6FF', width=1, dash='dot')
))

fig.update_layout(
title='RELIANCE.NS with SMA20',
yaxis_title='Price (INR)',
height=450,
```

```
showlegend=True  
)  
fig.show()
```

3. Candlestick Chart

The primary chart type for QuantIQ price display. Green candles: close > open. Red candles: close < open.

3.1 Minimal candlestick

```
import plotly.graph_objects as go
import yfinance as yf

df = yf.download('RELIANCE.NS', period='3mo', auto_adjust=True)

fig = go.Figure(data=[go.Candlestick(
    x=df.index,
    open=df['Open'],
    high=df['High'],
    low=df['Low'],
    close=df['Close'],
    name='RELIANCE.NS'
)])

fig.update_layout(
    title='RELIANCE.NS',
    xaxis_rangeflider_visible=False, # always disable for trading charts
    height=500
)
fig.show()
```

3.2 Candlestick parameters

Parameter	Type	Notes
x	array-like	Dates or timestamps
open	array-like	Open prices
high	array-like	High prices
low	array-like	Low prices
close	array-like	Close prices
name	str	Trace name in legend and hover
increasing_line_color	str	Bullish candle color (default '#26a641' green)
decreasing_line_color	str	Bearish candle color (default '#ff0000' red)
increasing_fillcolor	str	Bullish body fill
decreasing_fillcolor	str	Bearish body fill
opacity	float	0.0–1.0 transparency
showlegend	bool	Show in legend
visible	bool / 'legendonly'	Show/hide trace
hovertext	array	Custom hover text per bar

3.3 Candlestick with SMAs and Bollinger Bands

```
from ta.trend import SMAIndicator
from ta.volatility import BollingerBands

sma20 = SMAIndicator(df['Close'], 20).sma_indicator()
sma50 = SMAIndicator(df['Close'], 50).sma_indicator()
bb = BollingerBands(df['Close'], 20, 2)

fig = go.Figure()
```

```

# Candlestick
fig.add_trace(go.Candlestick(
    x=df.index, open=df['Open'], high=df['High'],
    low=df['Low'], close=df['Close'], name='OHLC'
))

# Bollinger Bands (filled area)
fig.add_trace(go.Scatter(
    x=df.index.tolist() + df.index.tolist()[::-1],
    y=bb.bollinger_hband().tolist() + bb.bollinger_lband().tolist()[::-1],
    fill='toself', fillcolor='rgba(88,166,255,0.08)',
    line=dict(color='rgba(0,0,0,0)'), name='BB Band'
))

# BB lines
fig.add_trace(go.Scatter(
    x=df.index, y=bb.bollinger_hband(),
    line=dict(color='#58A6FF', dash='dash', width=1), name='BB Upper'
))
fig.add_trace(go.Scatter(
    x=df.index, y=bb.bollinger_lband(),
    line=dict(color='#58A6FF', dash='dash', width=1), name='BB Lower'
))

# SMAs
fig.add_trace(go.Scatter(
    x=df.index, y=sma20,
    line=dict(color='#F0883E', width=1), name='SMA 20'
))
fig.add_trace(go.Scatter(
    x=df.index, y=sma50,
    line=dict(color='#BC8CFF', width=1), name='SMA 50'
))

fig.update_layout(
    title='RELIANCE.NS — Price with BB and SMAs',
    xaxis_rangeslider_visible=False,
    height=500,
    hovermode='x unified'
)
fig.show()

```

3.4 Custom candle colours (QuantIQ theme)

```

fig.add_trace(go.Candlestick(
    x=df.index, open=df['Open'], high=df['High'],
    low=df['Low'], close=df['Close'],
    increasing_line_color='#3FB950', # QuantIQ green
    decreasing_line_color='#F85149', # QuantIQ red
    increasing_fillcolor='#3FB950',
    decreasing_fillcolor='#F85149',
    name='OHLC'
))

```

4. OHLC Chart

Lighter alternative to candlestick — tick marks instead of boxes. Renders faster for large datasets (>1000 bars).

```
fig = go.Figure(data=[go.Ohlc(
    x=df.index,
    open=df['Open'], high=df['High'],
    low=df['Low'], close=df['Close'],
    name='OHLC'
)])

fig.update_layout(
    title='RELIANCE.NS OHLC',
    xaxis_rangeslider_visible=False
)
fig.show()
```

5. Line and Scatter Charts

5.1 Line chart parameters

Parameter	Type	Notes
x	array	X values (dates, index)
y	array	Y values
mode	str	'lines', 'markers', 'lines+markers', 'text'
name	str	Legend label
line	dict	dict(color='#3FB950', width=1.5, dash='dot')
opacity	float	0.0–1.0
fill	str	'none', 'tozero', 'tonext', 'toend'
fillcolor	str	Area fill color (use rgba for transparency)
hovertemplate	str	Custom hover: '%{y:.2f} INR'
connectgaps	bool	Connect gaps in data (NaN values)
visible	bool / 'legendonly'	Start hidden but in legend

5.2 Line dash patterns

dash value	Renders as
'solid'	Solid line (default)
'dot'	Small dots
'dash'	Short dashes
'longdash'	Long dashes
'dashdot'	Alternating dash-dot
'longdashdot'	Long dash + dot

5.3 Entry/exit marker scatter

```
# Entry markers (green triangles up)
fig.add_trace(go.Scatter(
    x=df.index[entries],
    y=df['Close'][entries],
    mode='markers',
    name='Entry',
    marker=dict(
        symbol='triangle-up',
        size=10,
        color='#3FB950',
        line=dict(color='#1A7F37', width=1)
    )
))

# Exit markers (red triangles down)
fig.add_trace(go.Scatter(
    x=df.index[exits],
    y=df['Close'][exits],
    mode='markers',
    name='Exit',
    marker=dict(
        symbol='triangle-down',
        size=10,
        color='#F85149',
```

```
line=dict(color='#CF222E', width=1)
)
))
```

5.4 Scatter marker symbols

symbol	Shape
'circle'	Filled circle (default)
'circle-open'	Open circle
'square'	Square
'diamond'	Diamond
'triangle-up'	Up triangle (entry signal)
'triangle-down'	Down triangle (exit signal)
'star'	Star
'x'	X mark
'cross'	Plus sign

6. Bar Charts

Primarily used for volume bars in financial charts.

6.1 Volume bar chart with colour by direction

```
import numpy as np

# Colour volume bars by price direction
vol_colors = np.where(
    df['Close'] >= df['Open'],
    '#3FB950', # green on up days
    '#F85149' # red on down days
)

fig.add_trace(go.Bar(
    x=df.index,
    y=df['Volume'],
    name='Volume',
    marker_color=vol_colors,
    opacity=0.7,
    showlegend=False
))
```

6.2 Trade P&L; bar chart

```
pnl = pf.trades.records_readable[['Entry Time', 'PnL']].copy()
pnl['Color'] = np.where(pnl['PnL'] >= 0, '#3FB950', '#F85149')

fig = go.Figure(data=[go.Bar(
    x=pnl['Entry Time'],
    y=pnl['PnL'],
    marker_color=pnl['Color'],
    name='Trade P&L (INR)'
)])
fig.update_layout(title='Trade P&L Distribution', yaxis_title='INR')
fig.show()
```

7. Subplots — Multi-Panel Charts

`make_subplots()` creates a grid of subplot panels. `shared_xaxes=True` links the zoom/pan across all panels — essential for financial charts.

7.1 `make_subplots()` parameters

Parameter	Notes
<code>rows</code>	Number of rows in grid
<code>cols</code>	Number of columns in grid (usually 1 for financial charts)
<code>shared_xaxes</code>	True — pan/zoom is linked across all rows
<code>row_heights</code>	List of relative heights: [0.6, 0.2, 0.2] for price/volume/RSI
<code>vertical_spacing</code>	Gap between rows (default 0.2). Use 0.02–0.05 for trading charts
<code>subplot_titles</code>	Tuple of titles per subplot

7.2 Standard QuantIQ 3-panel chart: Price + Volume + RSI

```
from plotly.subplots import make_subplots
import plotly.graph_objects as go
import numpy as np

fig = make_subplots(
    rows=3, cols=1,
    shared_xaxes=True,
    row_heights=[0.6, 0.2, 0.2],
    vertical_spacing=0.03,
    subplot_titles=('Price', 'Volume', 'RSI (14)')
)

# Row 1: Candlestick + SMAs
fig.add_trace(go.Candlestick(
    x=df.index, open=df['Open'], high=df['High'],
    low=df['Low'], close=df['Close'],
    name='OHLC',
    increasing_line_color='#3FB950',
    decreasing_line_color='#F85149'
), row=1, col=1)

fig.add_trace(go.Scatter(
    x=df.index, y=df['SMA20'],
    line=dict(color='#F0883E', width=1), name='SMA20'
), row=1, col=1)

# Row 2: Volume bars
vol_colors = np.where(df['Close'] >= df['Open'], '#3FB950', '#F85149')
fig.add_trace(go.Bar(
    x=df.index, y=df['Volume'],
    marker_color=vol_colors, name='Volume', showlegend=False
), row=2, col=1)

# Row 3: RSI
fig.add_trace(go.Scatter(
    x=df.index, y=df['RSI'],
    line=dict(color='#BC8CFF', width=1.2), name='RSI'
), row=3, col=1)
fig.add_hline(y=70, line_dash='dot', line_color='red', row=3, col=1)
fig.add_hline(y=30, line_dash='dot', line_color='green', row=3, col=1)

# Layout
fig.update_layout(
    title='RELIANCE.NS — Full Technical Chart',
    height=700,
```

```

axis_rangeslider_visible=False,
hovermode='x unified',
showlegend=True
)
fig.update_yaxes(title_text='Price (INR)', row=1, col=1)
fig.update_yaxes(title_text='Volume', row=2, col=1)
fig.update_yaxes(title_text='RSI', range=[0,100], row=3, col=1)
fig.show()

```

7.3 4-panel chart: Price + Volume + MACD + ATR

```

fig = make_subplots(
    rows=4, cols=1, shared_xaxes=True,
    row_heights=[0.5, 0.15, 0.2, 0.15],
    vertical_spacing=0.02
)

# Row 1: Candlestick
fig.add_trace(go.Candlestick(
    x=df.index, open=df['Open'], high=df['High'],
    low=df['Low'], close=df['Close'], name='OHLC'
), row=1, col=1)

# Row 2: Volume
fig.add_trace(go.Bar(
    x=df.index, y=df['Volume'], name='Volume', showlegend=False
), row=2, col=1)

# Row 3: MACD
fig.add_trace(go.Scatter(
    x=df.index, y=df['MACD'], line=dict(color='#3FB950'), name='MACD'
), row=3, col=1)
fig.add_trace(go.Scatter(
    x=df.index, y=df['MACD_signal'],
    line=dict(color='#F0883E', dash='dot'), name='Signal'
), row=3, col=1)
macd_colors = np.where(df['MACD_hist'] >= 0, '#3FB950', '#F85149')
fig.add_trace(go.Bar(
    x=df.index, y=df['MACD_hist'],
    marker_color=macd_colors, name='Histogram', showlegend=False
), row=3, col=1)

# Row 4: ATR
fig.add_trace(go.Scatter(
    x=df.index, y=df['ATR'],
    line=dict(color='#58A6FF'), name='ATR (14)'
), row=4, col=1)

fig.update_layout(height=750, axis_rangeslider_visible=False,
    hovermode='x unified')
fig.show()

```

8. Layout and Styling

8.1 `update_layout()` — full reference

Key	Example value	Effect
<code>title</code>	'RELIANCE.NS Chart'	Chart title text
<code>title_x</code>	0.5	Title horizontal position (0=left, 1=right)
<code>height</code>	500	Figure height in pixels
<code>width</code>	900	Figure width (None = auto)
<code>showlegend</code>	True	Show/hide legend
<code>legend</code>	<code>dict(x=0.01,y=0.99,bgcolor='rgba(0,0,0,0)')</code>	Legend position and style
<code>hovermode</code>	'x unified'	'x unified' shows all traces at cursor x
<code>template</code>	'plotly_dark'	Dark mode template
<code>paper_bgcolor</code>	'#0D1117'	Outer background (QuantIQ dark)
<code>plot_bgcolor</code>	'#161B22'	Plot area background (QuantIQ surface)
<code>font</code>	<code>dict(color='#E6EDF3')</code>	Global font colour
<code>margin</code>	<code>dict(l=60,r=20,t=50,b=40)</code>	Pixel margins
<code>xaxis_ranglider_visible</code>	False	Remove range slider (always False for OHLC)

8.2 QuantIQ dark theme template

```

QUANTIQLAYOUT = dict(
    paper_bgcolor='#0D1117',
    plot_bgcolor='#161B22',
    font=dict(color='#E6EDF3', size=12),
    xaxis=dict(
        gridcolor='#30363D',
        showgrid=True,
        ranglider=dict(visible=False)
    ),
    yaxis=dict(
        gridcolor='#30363D',
        showgrid=True,
    ),
    legend=dict(
        bgcolor='rgba(13,17,23,0.7)',
        bordercolor='#30363D',
        borderwidth=1
    ),
    hovermode='x unified',
    hoverlabel=dict(
        bgcolor='#161B22',
        bordercolor='#30363D',
        font_color='#E6EDF3'
    )
)

# Apply to any figure
fig.update_layout(**QUANTIQLAYOUT)

```

8.3 `update_xaxes / update_yaxes`

```

# Apply to all axes
fig.update_xaxes(showgrid=True, gridcolor='#30363D')
fig.update_yaxes(showgrid=True, gridcolor='#30363D')

# Apply to specific subplot row

```

```
fig.update_yaxes(title_text='Price (INR)', row=1, col=1)
fig.update_yaxes(title_text='Volume', row=2, col=1)
fig.update_yaxes(range=[0, 100], row=3, col=1) # fix RSI range

# Hide dates with no trading (weekend gaps)
import pandas as pd
all_dates = pd.date_range(df.index.min(), df.index.max())
trading_dates = df.index
dt_breaks = [d for d in all_dates if d not in trading_dates]
fig.update_xaxes(
    rangebreaks=[dict(values=dt_breaks)]
)
```

9. Annotations, Shapes, and Reference Lines

9.1 Horizontal reference lines

```
# Add RSI overbought/oversold lines
fig.add_hline(y=70, line_dash='dot', line_color='red',
annotation_text='Overbought', row=3, col=1)
fig.add_hline(y=30, line_dash='dot', line_color='green',
annotation_text='Oversold', row=3, col=1)

# Add a price support level
fig.add_hline(y=1350, line_dash='dash', line_color='#F0883E',
annotation_text='Support: 1350', row=1, col=1)
```

9.2 Vertical reference lines (events)

```
# Mark an event date
fig.add_vline(
x='2025-01-15',
line_dash='dash',
line_color='#BC8CFF',
annotation_text='Budget 2025',
annotation_position='top right'
)
```

9.3 Shaded regions

```
# Shade a time period (e.g., drawdown, event window)
fig.add_vrect(
x0='2025-03-01', x1='2025-03-31',
fillcolor='rgba(248, 81, 73, 0.1)',
layer='below',
line_width=0,
annotation_text='Correction',
annotation_position='top left'
)

# Horizontal band (e.g., RSI neutral zone)
fig.add_hrect(
y0=45, y1=55,
fillcolor='rgba(88,166,255,0.05)',
layer='below',
line_width=0
)
```

9.4 Text annotations

```
fig.add_annotation(
x='2024-10-15',
y=df['Close']['2024-10-15'],
text='Entry signal',
showarrow=True,
arrowhead=2,
arrowcolor='#3FB950',
font=dict(color='#3FB950', size=11),
bgcolor='rgba(13,17,23,0.7)'
)
```

10. Axes, Range, and Zoom

10.1 Date axis range selector buttons

```
# Add zoom range buttons to the x-axis
fig.update_xaxes(
    rangeselector=dict(
        buttons=[
            dict(count=1, label='1W', step='week', stepmode='backward'),
            dict(count=1, label='1M', step='month', stepmode='backward'),
            dict(count=3, label='3M', step='month', stepmode='backward'),
            dict(count=6, label='6M', step='month', stepmode='backward'),
            dict(count=1, label='1Y', step='year', stepmode='backward'),
            dict(step='all', label='All'),
        ],
        bgcolor='#161B22',
        activecolor='#3FB950'
    ),
    rangeslider=dict(visible=False) # keep slider off
)
```

10.2 Fix y-axis range

```
# Fix RSI y-axis to 0-100
fig.update_yaxes(range=[0, 100], row=3, col=1)

# Auto-range with padding
fig.update_yaxes(autorange=True)
```

10.3 Log scale

```
# Log scale y-axis (useful for long-term price charts)
fig.update_yaxes(type='log')
```

10.4 Tick formatting

```
# Currency format on y-axis
fig.update_yaxes(tickprefix='INR ', tickformat=',.0f')

# Percentage format
fig.update_yaxes(tickformat='%.1%')

# Volume in millions
fig.update_yaxes(tickformat='.2s', row=2, col=1) # e.g. '5.23M'
```

11. Export and Save

Method	Output
<code>fig.show()</code>	Open in browser (local) or display in notebook
<code>fig.write_html('chart.html')</code>	Standalone interactive HTML (no dependencies)
<code>fig.write_image('chart.png')</code>	Static PNG (requires kaleido: <code>pip install kaleido</code>)
<code>fig.write_image('chart.svg')</code>	Scalable SVG
<code>fig.write_image('chart.pdf')</code>	Vector PDF
<code>fig.write_json('chart.json')</code>	Plotly JSON spec (re-import with <code>go.Figure(json)</code>)
<code>fig.to_html(include_plotlyjs='cdn')</code>	HTML string with CDN-linked Plotly (small file)
<code>fig.to_image(format='png', width=1200, height=600)</code>	Bytes object (for API response)

Export notes

- `kaleido` is required for PNG/SVG/PDF/JPEG export. `pip install kaleido`. Not required for HTML.
- `write_html()` output is the best option for Streamlit file download buttons.
- For Streamlit: use `st.plotly_chart(fig)` — never `fig.show()`.

12. QuantIQ Standard Chart Patterns

12.1 Equity curve vs benchmark

```
# From vectorbt pf object
equity = pf.value().rename('Strategy')
bh = pf_bh.value().rename('Buy & Hold')

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=equity.index, y=equity,
    name='Strategy', line=dict(color='#3FB950', width=2)
))
fig.add_trace(go.Scatter(
    x=bh.index, y=bh,
    name='Buy & Hold', line=dict(color='#58A6FF', width=1.5, dash='dot')
))
fig.update_layout(
    title='Portfolio vs Buy-and-Hold',
    yaxis_title='Portfolio Value (INR)',
    height=400,
    **QUANTIQ_LAYOUT
)
st.plotly_chart(fig, use_container_width=True)
```

12.2 Drawdown chart

```
equity = pf.value()
rolling_max = equity.cummax()
drawdown = (equity - rolling_max) / rolling_max * 100

fig = go.Figure()
fig.add_trace(go.Scatter(
    x=drawdown.index, y=drawdown,
    fill='tozeroy',
    fillcolor='rgba(248,81,73,0.2)',
    line=dict(color='#F85149', width=1),
    name='Drawdown'
))
fig.update_layout(
    title='Portfolio Drawdown',
    yaxis_title='Drawdown (%)',
    height=250, **QUANTIQ_LAYOUT
)
st.plotly_chart(fig, use_container_width=True)
```

12.3 Returns histogram

```
returns = pf.returns().dropna() * 100 # as percentage

fig = go.Figure(data=[go.Histogram(
    x=returns,
    nbinsx=50,
    name='Daily Return',
    marker_color='#3FB950',
    opacity=0.75
)])
fig.add_vline(x=0, line_color='white', line_width=1)
fig.update_layout(
    title='Return Distribution',
    xaxis_title='Daily Return (%)',
    yaxis_title='Frequency',
    height=300, **QUANTIQ_LAYOUT
)
```

12.4 Correlation heatmap

```
import plotly.express as px

corr = close_df.pct_change().dropna().corr()

fig = px.imshow(
    corr,
    text_auto='.2f',
    color_continuous_scale='RdYlGn',
    zmin=-1, zmax=1,
    title='Stock Correlation Matrix'
)
fig.update_layout(height=400)
st.plotly_chart(fig, use_container_width=True)
```

13. See Also

Resource	URL
Plotly Python docs	plotly.com/python/
API reference	plotly.com/python-api-reference/
Figure reference (all traces)	plotly.com/python/reference/
Candlestick reference	plotly.com/python/reference/candlestick/
Figure factory	plotly.com/python/figure-factories/
Plotly GitHub	github.com/plotly/plotly.py
Plotly PyPI	pypi.org/project/plotly/
kaleido (static export)	github.com/plotly/Kaleido
make_subplots docs	plotly.com/python/subplots/
Financial charts guide	plotly.com/python/figure-factories/#financial

Cross-references to other binder modules

Module	Relevance
M2 — Data Stack	pandas DataFrames as x/y data input for all traces
M3 — ta library	Indicator values (SMA, RSI, BB, MACD) as trace y values
M4 — vectorbt	pf.value(), pf.returns(), pf.trades data for equity/return charts
M5 — Streamlit	st.plotly_chart(fig, use_container_width=True) for dashboard display

End of Module 6. Next: Module 7 — Python Standards (PEP 8/257/484, Google Style).

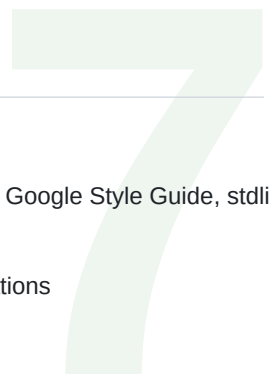
QuantIQ

Master Reference Binder

Module 7

Python Standards

Version	18 May 2026
Scope	PEP 8 (style), PEP 257 (docstrings), PEP 484 (type hints), Google Style Guide, stdlib logging
Source	peps.python.org / google.github.io/styleguide/pyguide.html
Convention	QuantIQ uses Google-style docstrings and full type annotations
Pages	approx 30



Module Contents

§	Section	Page
1	Quick Reference Card	3
2	PEP 8 — Style Guide	5
3	PEP 257 — Docstring Conventions	11
4	Google Python Style Guide — Docstrings	13
5	PEP 484 — Type Hints	18
6	stdlib logging	23
7	QuantIQ Code Standards	28
8	See Also	30

1. Quick Reference Card

Naming conventions at a glance

Identifier type	Convention	Example
Module / package	lower_snake_case	data_pipeline, ta_indicators
Class	UpperCamelCase	DhanClient, PriceDataLoader
Function / method	lower_snake_case	fetch_ohlcv, calculate_rsi
Variable	lower_snake_case	close_price, rsi_window
Constant	UPPER_SNAKE_CASE	MAX_ORDER_SIZE, TOKEN_EXPIRY_HOURS
Private method	_leading_underscore	_validate_token, _build_payload
Name-mangled attribute	__double_leading	__secret_key
Type alias	UpperCamelCase	ClosePrice = pd.Series

Formatting rules at a glance

Rule	Value
Indentation	4 spaces. Never tabs.
Max line length	88 characters (Black formatter default) or 79 (PEP 8 strict)
Blank lines	2 between top-level defs; 1 between methods inside class
Imports	One per line. stdlib first, then third-party, then local. Alphabetical within groups.
Strings	Single or double — pick one and be consistent. QuantIQ uses double quotes.
Trailing commas	Use on multi-line sequences: last item followed by comma.
Semicolons	Never.
Spaces around operators	<code>x = 1 + 2</code> (yes). <code>x=1+2</code> (no).
Spaces in slices	<code>df[1:3]</code> (no spaces). <code>df[1 : 3]</code> (only if slice has expressions).
f-strings	Preferred over <code>.format()</code> or <code>%</code> formatting.

Google-style docstring skeleton

```
def place_order(
    symbol: str,
    quantity: int,
    price: float,
) -> dict[str, str]:
    """Place a limit order via DhanHQ API.

    Args:
        symbol: NSE security ID (e.g. '11536' for RELIANCE).
        quantity: Number of shares to buy.
        price: Limit price in INR.

    Returns:
        A dict with 'orderId' and 'orderStatus' keys.

    Raises:
        ValueError: If quantity is zero or negative.
        DhanAPIError: If the API returns an error response.
    """
    ...
```

2. PEP 8 — Style Guide

Official: peps.python.org/pep-0008/ | Mirror: pep8.org

2.1 Indentation

```
# Good – 4 spaces
def fetch_price(symbol: str) -> float:
    response = dhan.get_ltp(symbol)
    return response['ltp']
```

```
# Bad – tabs or 2 spaces
def fetch_price(symbol: str) -> float:
    response = dhan.get_ltp(symbol) # tab
```

Continuation lines: align with opening delimiter or use hanging indent of 4 spaces:

```
# Aligned with opening delimiter
result = some_function(
    arg1,
    arg2)

# Hanging indent (preferred for long args)
result = some_function(
    arg1,
    arg2,
    arg3,
)
```

2.2 Line length

```
# Good – under 88 chars
entries = (rsi < 30) & (close > sma50) & (adx > 25)
```

```
# Bad – too long
entries = (rsi_indicator.rsi() < 30) & (close_prices > sma50_indicator.sma_indicator()) & (adx > 25)
```

Break long lines with backslash or parentheses:

```
# Parentheses – preferred
entries = (
    (rsi_indicator.rsi() < 30)
    & (close_prices > sma50_indicator.sma_indicator())
    & (adx > 25)
)
```

2.3 Blank lines

```
class DhanClient:
    """DhanHQ API client."""

    def __init__(self, client_id: str, token: str) -> None:
        self.client_id = client_id
        self._token = token

    def place_order(self, symbol: str, qty: int) -> dict:
        ...

    def get_positions(self) -> list:
        ...

class OrderManager: # two blank lines before top-level class
    ...
```

2.4 Imports

```
# Good – one import per line, grouped, alphabetical within group
import os
import sys
from datetime import date, datetime

import numpy as np
import pandas as pd
from ta.momentum import RSIIndicator

from quantiq.data import loader
from quantiq.strategy import signals
```

```
# Bad – multiple on one line, wrong order
import pandas, numpy, os
from ta.momentum import RSIIndicator, MACD, StochasticOscillator
```

2.5 Whitespace in expressions

```
# Good
x = 1
y = x + 2
df['Returns'] = df['Close'].pct_change()
result = func(arg1, arg2)
slice_ = df[1:3]
```

```
# Bad
x=1
y = x+2
df[ 'Returns' ] = df['Close'].pct_change ( )
result = func( arg1, arg2 )
slice_ = df[1 : 3]
```

2.6 Naming conventions

Type	Convention	Notes
Package	all_lowercase	No hyphens. data_pipeline not data-pipeline.
Module	lower_snake_case	ta_indicators.py, dhan_client.py
Class	UpperCamelCase	DhanClient, BacktestEngine
Exception	UpperCamelCase + Error	DhanAPIError, TokenExpiredError
Function	lower_snake_case	calculate_rsi(), fetch_ohlc()
Method	lower_snake_case	Same as function
Variable	lower_snake_case	rsi_window, stop_loss_pct
Constant	UPPER_SNAKE_CASE	MAX_ORDERS_PER_SEC = 10
Private (convention)	_single_leading	_validate_symbol, _token
Avoid	I, O, l	Confusable with 1, 0, 1

2.7 String quotes

PEP 8 allows single or double — be consistent within a project. QuantIQ uses double quotes.

```
# Good – double quotes (QuantIQ standard)
message = "Order placed: RELIANCE.NS"
f_string = f"Symbol: {symbol}, Price: {price:.2f}"
```

2.8 Comparisons

```
# Good
if value is None:
...
if isinstance(obj, pd.DataFrame):
...
if not my_list:
...
```



```
# Bad
if value == None:
...
if type(obj) == pd.DataFrame:
...
if len(my_list) == 0:
...
```

2.9 Exception handling

```
# Good – name the exception, log it
try:
    response = dhan.place_order(...)
except DhanAPIError as e:
    logger.error(f"Order failed: {e}")
    raise
except (ValueError, KeyError) as e:
    logger.warning(f"Bad input: {e}")
```

```
# Bad – bare except swallows everything
try:
    response = dhan.place_order(...)
except:
    pass
```

3. PEP 257 — Docstring Conventions

Official: peps.python.org/pep-0257/

PEP 257 defines WHERE to put docstrings and the basic structure. QuantIQ uses Google style (§4) for the content format.

3.1 Where to write docstrings

- Every public module, class, method, and function **MUST** have a docstring.
- Private methods (`_leading_underscore`) should have docstrings if non-trivial.
- Docstring goes as the first statement after `def/class`, before any code.
- Use triple double-quotes always: `"""` even for one-liners.

3.2 One-liner docstrings

```
def total_return(start: float, end: float) -> float:
    """Return the percentage return from start to end value."""
    return (end - start) / start
```

One-liner rules: fits on one line, no blank line before/after, describes WHAT not HOW, ends with a period.

3.3 Multi-line docstrings

```
def calculate_sharpe(
    returns: pd.Series,
    risk_free: float = 0.065,
) -> float:
    """Calculate the annualised Sharpe ratio.

    The Sharpe ratio measures risk-adjusted return.
    A ratio above 1.0 is generally considered acceptable;
    above 2.0 is very good.

    Args:
        returns: Daily return Series.
        risk_free: Annual risk-free rate (decimal). Default 6.5%.

    Returns:
        Annualised Sharpe ratio.
    """
    rf_daily = risk_free / 252
    excess = returns - rf_daily
    return (excess.mean() / excess.std()) * (252 ** 0.5)
```

4. Google Python Style Guide — Docstrings

Official: google.github.io/styleguide/pyguide.html | Section: §3.8 Comments and Docstrings

QuantIQ uses Google-style docstrings exclusively. All section headers (Args, Returns, Raises, etc.) are followed by a colon and content is indented 4 spaces.

4.1 Standard sections

Section	When to use	Notes
Args:	Function has parameters	List each param: name, type (optional), description
Returns:	Function returns non-None	Describe type and content of return value
Yields:	Generator function	Like Returns but for yield
Raises:	Function can raise exceptions	List each exception type and when it's raised
Example:	Usage is non-obvious	Optional. Show a code example.
Note:	Important constraint	Optional. Use for hard constraints.
Attributes:	Class attributes	List class-level attributes (in class docstring)

4.2 Function docstring

```
def fetch_ohlcv(
    symbol: str,
    from_date: str,
    to_date: str,
    interval: str = 'day',
    oi: bool = False,
) -> pd.DataFrame:
    """Fetch historical OHLCV data from DhanHQ.

    Args:
        symbol: DhanHQ security ID (e.g. '11536' for RELIANCE).
        from_date: Start date in 'YYYY-MM-DD' format.
        to_date: End date in 'YYYY-MM-DD' format (exclusive).
        interval: Bar interval. One of 'day', '1', '5', '15', '25', '60'.
        oi: If True, include Open Interest column (derivatives only).

    Returns:
        DataFrame indexed by datetime with columns:
        Open, High, Low, Close, Volume, (OI if oi=True).

    Raises:
        ValueError: If from_date >= to_date.
        DhanAPIError: If the API returns DH-90x error code.
        TokenExpiredError: If the 24-hour access token has expired.

    Example:
    >>> df = fetch_ohlcv('11536', '2024-01-01', '2025-01-01')
    >>> df.shape
    (252, 5)
    """
    ...
```

4.3 Class docstring

```
class DhanClient:
    """Wrapper around the DhanHQ Python SDK.

    Handles authentication, token refresh, and rate-limiting.
    Configured via .streamlit/secrets.toml (never hardcode credentials).
```

```

Attributes:
client_id: DhanHQ client identifier.
max_orders_per_sec: Hard rate limit (SEBI mandate: 10/sec).

Example:
>>> client = DhanClient.from_secrets()
>>> client.get_positions()
"""

max_orders_per_sec: int = 10

def __init__(self, client_id: str, access_token: str) -> None:
    """Initialise client with credentials.

    Args:
    client_id: DhanHQ client ID.
    access_token: 24-hour JWT access token.
    """
    self.client_id = client_id
    self._token = access_token

```

4.4 Module-level docstring

```

"""Data loading utilities for QuantIQ.

This module provides functions for fetching OHLCV data from
DhanHQ historical API and yfinance, with built-in caching,
validation, and NSE ticker normalisation.

Typical usage::

from quantiq.data import loader
df = loader.fetch_ohlc('11536', '2024-01-01', '2025-01-01')
"""

from __future__ import annotations

import logging
from datetime import date
...

```

4.5 Common docstring mistakes

```

# Bad – describes HOW not WHAT
def rsi(close: pd.Series, window: int = 14) -> pd.Series:
    """Loop over the close series, compute RS ratio, return RSI."""

```

```

# Good – describes WHAT
def rsi(close: pd.Series, window: int = 14) -> pd.Series:
    """Calculate the Relative Strength Index (RSI)."""

```

```

# Bad – missing period, repeats function name verbatim
def calculate_rsi():
    """calculate_rsi"""

```

```

# Good – summarises purpose
def calculate_rsi() -> pd.Series:
    """Return RSI-14 as a Series with the same index as input."""

```

5. PEP 484 — Type Hints

Official: peps.python.org/pep-0484/

Type hints make code self-documenting and enable IDE completion and mypy static checking. QuantIQ requires type hints on all public functions and class attributes.

5.1 Imports for type hints

```
# Python 3.10+ – use built-in generics directly
from __future__ import annotations # enable new syntax in 3.9

# stdlib typing (still needed for some types)
from typing import Any, Callable, Optional, Union

# Common third-party
import pandas as pd
import numpy as np
```

5.2 Basic type annotations

```
# Variables
symbol: str = 'RELIANCE.NS'
quantity: int = 100
price: float = 1365.50
is_live: bool = False

# Function signatures
def buy(
    symbol: str,
    quantity: int,
    price: float,
    stop_loss: float | None = None,
) -> dict[str, str]:
    ...

# Return None explicitly
def log_trade(trade: dict) -> None:
    ...

# Class attribute annotations
class Config:
    client_id: str
    max_orders: int = 10
    symbols: list[str]
```

5.3 Common type annotations for QuantIQ

Annotation	Meaning
str	String
int	Integer
float	Float
bool	Boolean
None	No return value (use -> None)
pd.Series	pandas Series (any dtype)
pd.DataFrame	pandas DataFrame
np.ndarray	NumPy array
dict[str, str]	Dict with str keys and str values
dict[str, Any]	Dict with str keys, any values

Annotation	Meaning
<code>list[str]</code>	List of strings
<code>list[dict]</code>	List of dicts
<code>tuple[str, int]</code>	Tuple with str then int
<code>str None</code>	str or None (preferred over <code>Optional[str]</code>)
<code>int float</code>	int or float
<code>Callable[[str], float]</code>	Function taking str, returning float
<code>type[DhanClient]</code>	The class itself, not an instance

5.4 Type aliases (for readability)

```
# Type aliases reduce verbosity
ClosePrice = pd.Series
OHLCVData = pd.DataFrame
SecurityId = str
OrderId = str
Symbol = str

# Use them in signatures
def calculate_rsi(close: ClosePrice, window: int = 14) -> ClosePrice:
    ...

def fetch_ohlc(symbol: Symbol) -> OHLCVData:
    ...
```

5.5 Optional and Union (legacy syntax vs new)

Old (Python 3.9-)	New (Python 3.10+)	Meaning
<code>Optional[str]</code>	<code>str None</code>	str or None
<code>Union[str, int]</code>	<code>str int</code>	str or int
<code>List[str]</code>	<code>list[str]</code>	List of strings
<code>Dict[str, int]</code>	<code>dict[str, int]</code>	Dict str->int
<code>Tuple[str, int]</code>	<code>tuple[str, int]</code>	(str, int) tuple

QuantIQ uses Python 3.11+ so always use the new syntax. Add `from __future__ import annotations` at top of file for 3.9/3.10 compatibility.

5.6 Annotating pandas-heavy code

```
import pandas as pd
import numpy as np
from typing import Any

def build_features(
    df: pd.DataFrame,
    rsi_window: int = 14,
    bb_window: int = 20,
) -> pd.DataFrame:
    """Add technical indicator columns to a price DataFrame."""
    ...

def backtest(
    close: pd.Series,
    entries: pd.Series,
    exits: pd.Series,
    init_cash: float = 100_000.0,
    fees: float = 0.001,
) -> dict[str, float]:
    """Run backtest and return performance metrics."""
    ...
```

```
# Numpy array annotation
def signals_to_array(signals: pd.Series) -> np.ndarray:
    return signals.to_numpy()
```

6. stdlib logging

Official: docs.python.org/3/library/logging.html | HOWTO: docs.python.org/3/howto/logging.html

Use the stdlib logging module — never `print()` in production code. SEBI Feb 2025 requires audit logs with ≥ 5 year retention for Phase 4 bots.

6.1 Log levels

Level	When to use	Method
DEBUG	Detailed diagnostic (not shown in production)	<code>logger.debug()</code>
INFO	Normal operational events (startup, token refresh, orders)	<code>logger.info()</code>
WARNING	Approaching limits, recoverable issues	<code>logger.warning()</code>
ERROR	Order failed, API error, strategy error	<code>logger.error()</code>
CRITICAL	Bot is shutting down, unrecoverable failure	<code>logger.critical()</code>

6.2 Module-level logger (correct pattern)

```
# At top of every module – module-level logger
import logging

logger = logging.getLogger( __name__ )

# Bad – root logger or print()
logging.basicConfig(level=logging.DEBUG) # configures root logger globally
print('Order placed') # never in production
```

6.3 Logger configuration (in main entry point only)

```
# bot/main.py or app.py – configure ONCE at startup
import logging
import sys
from logging.handlers import TimedRotatingFileHandler
from pathlib import Path

def setup_logging(
    log_dir: str = 'logs',
    log_level: int = logging.INFO,
) -> None:
    """Configure root logger for QuantIQ bot."""
    Path(log_dir).mkdir(parents=True, exist_ok=True)

    fmt = logging.Formatter(
        '%(asctime)s | %(name)-20s | %(levelname)-8s | %(message)s',
        datefmt='%Y-%m-%d %H:%M:%S'
    )

    # Console handler
    console = logging.StreamHandler(sys.stdout)
    console.setLevel(logging.INFO)
    console.setFormatter(fmt)

    # File handler – rotates daily, keeps 1825 days (5 years)
    file_handler = TimedRotatingFileHandler(
        filename=f'{log_dir}/quantiq.log',
        when='midnight',
        interval=1,
        backupCount=1825, # 5 years (SEBI requirement)
        encoding='utf-8'
    )
    file_handler.setLevel(logging.DEBUG)
    file_handler.setFormatter(fmt)
```



```

root_logger = logging.getLogger()
root_logger.setLevel(logging.DEBUG)
root_logger.addHandler(console)
root_logger.addHandler(file_handler)

# Call once at startup
setup_logging()

```

6.4 Using the logger in code

```

import logging
logger = logging.getLogger(__name__)

class OrderManager:
    def place_order(self, symbol: str, qty: int, price: float) -> dict:
        logger.info(f'Placing order: {symbol} qty={qty} price={price:.2f}')
        try:
            response = self._dhan.place_order(symbol, qty, price)
            logger.info(
                f'Order accepted: orderId={response["orderId"]} '
                f'status={response["orderStatus"]}'
            )
            return response
        except DhanAPIError as e:
            logger.error(f'Order failed: {symbol} | error={e}')
            raise

    def on_startup(self) -> None:
        logger.info('QuantIQ bot starting')
        logger.debug(f'Config: max_orders_per_sec={MAX_ORDERS_PER_SEC}')

    def on_rate_limit_warning(self, current_rate: float) -> None:
        logger.warning(
            f'Approaching rate limit: {current_rate:.1f}/10 orders/sec'
        )

```

6.5 Structured trade audit log

For SEBI compliance, log every order event with a structured format that can be queried later:

```

import json, logging
from datetime import datetime

audit_logger = logging.getLogger('quantiq.audit')

def log_order_event(
    event_type: str, # 'PLACED', 'FILLED', 'CANCELLED', 'REJECTED'
    order_id: str,
    symbol: str,
    side: str,
    quantity: int,
    price: float,
    algo_id: str = 'QUANTIQ_v1',
) -> None:
    record = {
        'ts': datetime.now().isoformat(),
        'event': event_type,
        'order_id': order_id,
        'symbol': symbol,
        'side': side,
        'qty': quantity,
        'price': price,
        'algo_id': algo_id,
    }
    audit_logger.info(json.dumps(record))

```

6.6 Logging best practices

Practice	Why
<code>logger = logging.getLogger(__name__)</code>	Gives hierarchical logger names (<code>quantiq.data</code> , <code>quantiq.bot</code>)
Never <code>print()</code> in production	Logs are buffered, timestamped, levelled, and rotatable
Log at boundaries	Log entry/exit of every API call and order placement
Use lazy f-string formatting	<code>logger.debug('val=%s', val)</code> — message only built if <code>DEBUG</code> enabled
Never log secrets	Do not log <code>access_token</code> , <code>client_id</code> , or any credential
<code>TimedRotatingFileHandler</code>	Rotates daily, supports 5+ year retention easily
Separate audit logger	Keep trade records in a dedicated log file for compliance
<code>exc_info=True</code>	<code>logger.error('Failed', exc_info=True)</code> — logs full traceback

7. QuantIQ Code Standards

These are the project-specific conventions for the QuantIQ codebase, derived from PEP 8, Google Style, and project decisions. All contributors must follow these without exception.

7.1 File and module conventions

Rule	Standard
Module docstring	Every .py file must start with a module docstring
from __future__ import annotations	Include in every module for forward reference support
Encoding	UTF-8. Add # -*- coding: utf-8 -*- only if non-ASCII in source.
Shebang	Only in entry points: #!/usr/bin/env python3
File length	Max 500 lines. Split into modules if longer.
Formatter	Black (line length 88). Run before every commit.
Linters	Ruff or flake8. Zero warnings policy.

7.2 Function conventions

Rule	Standard
Type hints	Required on all public functions. Optional on private helpers.
Docstrings	Required on all public functions. Use Google style.
Max function length	30 lines of actual code. Extract helpers if longer.
Single responsibility	One function does one thing. Name it as a verb phrase.
Pure functions preferred	Avoid side effects in computation functions.
Default args	Never use mutable defaults: def f(x, items=None): items = items or []

7.3 Constant definitions

```
# quantiq/constants.py – all project-wide constants

# DhanHQ API limits (SEBI mandated)
MAX_ORDERS_PER_SEC: int = 10
TOKEN_EXPIRY_HOURS: int = 24
MAX_ORDER_MODIFICATIONS: int = 25

# NSE trading hours (IST)
MARKET_OPEN: str = '09:15'
MARKET_CLOSE: str = '15:30'
PRE_OPEN: str = '09:00'

# Risk defaults
DEFAULT_RISK_PCT: float = 0.01 # 1% per trade
DEFAULT_BROKERAGE: float = 0.001 # 0.1% round-trip
```

7.4 Complete annotated function example

```
from __future__ import annotations
import logging
import pandas as pd
from ta.momentum import RSIIndicator
from ta.volatility import AverageTrueRange

logger = logging.getLogger(__name__)

def generate_signals(
```

```

df: pd.DataFrame,
rsi_window: int = 14,
rsi_oversold: float = 30.0,
rsi_overbought: float = 70.0,
) -> tuple[pd.Series, pd.Series]:
    """Generate long entry and exit signals based on RSI.

    Entry: RSI crosses above oversold threshold.
    Exit: RSI crosses above overbought threshold.

    Args:
    df: OHLCV DataFrame with DatetimeIndex.
    rsi_window: RSI lookback period. Defaults to 14.
    rsi_oversold: RSI level below which market is oversold.
    rsi_overbought: RSI level above which market is overbought.

    Returns:
    Tuple of (entries, exits) boolean Series.

    Raises:
    ValueError: If df does not contain required OHLCV columns.
    """
    required = {'Open', 'High', 'Low', 'Close', 'Volume'}
    if not required.issubset(df.columns):
        missing = required - set(df.columns)
        raise ValueError(f'Missing columns: {missing}')

    rsi = RSIIndicator(df['Close'], window=rsi_window).rsi()
    logger.debug(f'RSI calculated: window={rsi_window}, last={rsi.iloc[-1]:.1f}')

    entries = (rsi > rsi_oversold) & (rsi.shift(1) <= rsi_oversold)
    exits = (rsi > rsi_overbought) & (rsi.shift(1) <= rsi_overbought)

    logger.info(
        f'Signals: {entries.sum()} entries, {exits.sum()} exits '
        f'over {len(df)} bars'
    )
    return entries, exits

```

8. See Also

Resource	URL
PEP 8 — Style Guide	peps.python.org/pep-0008/
PEP 8 mirror	pep8.org/
PEP 257 — Docstrings	peps.python.org/pep-0257/
PEP 484 — Type Hints	peps.python.org/pep-0484/
PEP 526 — Variable annotations	peps.python.org/pep-0526/
Google Python Style Guide	google.github.io/styleguide/pyguide.html
Google Style docstrings (§3.8)	google.github.io/styleguide/pyguide.html#38-comments-and-docstrings
stdlib logging	docs.python.org/3/library/logging.html
Logging HOWTO	docs.python.org/3/howto/logging.html
Black formatter	github.com/psf/black
Ruff linter	github.com/astrol-sh/ruff
mypy type checker	mypy-lang.org/

Cross-references to other binder modules

Module	Relevance
M8 — Version Control	Conventional Commits uses consistent naming like style here; Black in pre-commit hook
M10 — Environment	pyproject.toml configures Black, Ruff, mypy; requirements-dev.txt for dev tools
All modules	Type-annotated examples throughout M1-M6 follow the QuantIQ standard defined here

End of Module 7. Next: Module 8 — Version Control and DevOps.

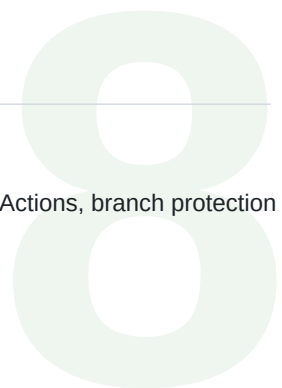
QuantIQ

Master Reference Binder

Module 8

Version Control & DevOps

Version	18 May 2026
Scope	Conventional Commits, Semantic Versioning, Git, GitHub Actions, branch protection
Repo	github.com/Quant-IQ/quantiq
Pages	approx 22



Module Contents

§	Section	Page
1	Quick Reference Card	3
2	Conventional Commits	5
3	Semantic Versioning (SemVer)	9
4	Git Essentials	11
5	Branching Strategy	15
6	GitHub Actions — CI/CD	17
7	Branch Protection	20
8	QuantIQ Workflow	21
9	See Also	22

1. Quick Reference Card

Commit format

```
<type>(<scope>): <short description>

[optional body]

[optional footer(s)]
```

Commit types cheat sheet

Type	Purpose	SemVer bump
feat	New feature or capability	MINOR
fix	Bug fix	PATCH
docs	Documentation only	none
style	Formatting, whitespace (no logic change)	none
refactor	Code restructure (no fix/feature)	none
perf	Performance improvement	PATCH
test	Add or fix tests	none
build	Build system, dependencies (pip, requirements.txt)	none
ci	CI/CD configuration (.github/workflows/)	none
chore	Maintenance, tooling, misc	none
revert	Revert a previous commit	PATCH
BREAKING CHANGE	Footer token for breaking API change	MAJOR

QuantIQ scopes

Scope	Area
data	Data ingestion, yfinance, DhanHQ historical
broker	DhanHQ orders, auth, WebSocket
strategy	Signal generation, indicators
backtest	vectorbt simulation
dashboard	Streamlit app
bot	Phase 4 live trading bot
config	Configuration, constants, secrets
deps	Dependency updates
docs	Documentation changes

Branch naming

Pattern	Example
feat/	feat/broker/token-refresh-scheduler
fix/	fix/data/yfinance-ns-suffix-validator
docs/	docs/contributing-guide-update
chore/	chore/upgrade-vectorbt-0.28.2

Pattern	Example
release/	release/1.2.0

Git daily commands

Command	Purpose
<code>git status</code>	Check working tree state
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes
<code>git add -p</code>	Interactively stage hunks (not whole files)
<code>git commit -m 'feat(data): add NSE validator'</code>	Commit with Conventional Commits message
<code>git push origin feat/data/nse-validator</code>	Push feature branch
<code>git pull --rebase origin main</code>	Sync with main (rebase, not merge)
<code>git log --oneline -20</code>	Last 20 commits, compact
<code>git stash</code> / <code>git stash pop</code>	Temporarily shelve/restore changes

2. Conventional Commits

Spec: conventionalcommits.org/en/v1.0.0/ | GitHub: github.com/conventional-commits/conventionalcommits.org

Conventional Commits is a lightweight convention on top of commit messages that provides a structured, machine-readable history. It enables automated changelog generation and version bumping.

2.1 Full format

```
<type>[optional scope]: <description>

[optional body]

[optional footer(s)]
```

Field	Rules
	Lowercase. One of the types in the cheat sheet. Required.
[scope]	Optional. Lowercase. Noun in parentheses: (data), (broker), (bot)
	Lowercase. Present tense. No full stop at end. Max 72 chars total first line.
[body]	Optional. Separated by blank line. Explains WHY not HOW.
[footer]	Optional. 'BREAKING CHANGE: ...' or 'Closes #123'. Each on own line.

2.2 Examples — good commits

```
# Simple fix
fix(data): append .NS suffix when fetching NSE tickers from yfinance

# Feature with scope
feat(broker): add daily token refresh scheduler

The refresh runs at 08:45 IST every weekday, 30 minutes before
market open, to ensure token validity before the first order.

Closes #42

# Breaking change
feat(strategy): change signal API to return (entries, exits) tuple

BREAKING CHANGE: generate_signals() now returns a tuple instead
of a dict. Callers must unpack: entries, exits = generate_signals(df)

# Docs
docs: add CONTRIBUTING.md with PR workflow and coding standards

# Dependency update
build(deps): upgrade vectorbt from 0.28.1 to 0.28.2

# CI change
ci: add pytest job to pull request workflow

# Revert
revert: feat(broker): add experimental order batching

This reverts commit abc1234. Batching caused race conditions
at the 10 orders/sec rate limit.
```

2.3 Examples — bad commits

Bad message	Problem	Better
fixed stuff	Not descriptive, not typed	fix(data): correct empty DataFrame return on NSE ticker timeout

Bad message	Problem	Better
WIP	No information	<code>docs(strategy): add RSI crossover strategy draft (wip)</code>
Updated requirements.txt	Not typed	<code>build(deps): pin ta==0.11.0 for reproducibility</code>
feat: Added VWAP	Uppercase first letter, added (past tense)	<code>feat(strategy): add manual VWAP calculation for intraday</code>
minor changes	Vague	<code>style(dashboard): reformat Streamlit layout columns</code>

2.4 Footer tokens

Token	Usage	Example
BREAKING CHANGE:	Major breaking API change	BREAKING CHANGE: <code>fetch_ohlcv()</code> now returns DataFrame not dict
Closes #N	Close a GitHub issue	Closes #42
Fixes #N	Fix a bug tracked in issue	Fixes #87
Refs #N	Reference without closing	Refs #15
Co-authored-by:	Credit a co-author	Co-authored-by: RT <rt@quant-iq.net>
Reviewed-by:	Code reviewer credit	Reviewed-by: RS <rs@quant-iq.net>

2.5 Why it matters for QuantIQ

- Automated changelog: tools like `git-cliff` or `standard-version` parse commit history into `CHANGELOG.md`.
- Version automation: PATCH on fix, MINOR on feat, MAJOR on BREAKING CHANGE.
- PR discipline: reviewers can scan commit list and understand every change without reading diffs.
- Blame traceability: when a bug appears, the commit type + scope instantly narrows the search.
- SEBI audit readiness: structured git history is part of the audit trail.

3. Semantic Versioning (SemVer)

Spec: semver.org

3.1 Format

MAJOR.MINOR.PATCH

Examples: 1.0.0 1.2.0 1.2.3 2.0.0

Part	Increment when	Example
MAJOR	Breaking API change — callers must update code	1.x.x -> 2.0.0
MINOR	New backward-compatible feature added	1.2.x -> 1.3.0
PATCH	Backward-compatible bug fix or small improvement	1.2.3 -> 1.2.4

3.2 Pre-release and build metadata

```
1.0.0-alpha # Alpha – not stable
1.0.0-alpha.1 # Alpha iteration
1.0.0-beta.2 # Beta
1.0.0-rc.1 # Release candidate
1.0.0 # Stable release
1.0.0+build.20260518 # Build metadata (ignored in precedence)
```

3.3 SemVer + Conventional Commits mapping

Commit type	SemVer bump
fix:	PATCH
perf:	PATCH
revert:	PATCH
feat:	MINOR (resets PATCH to 0)
BREAKING CHANGE footer	MAJOR (resets MINOR and PATCH to 0)
docs:, style:, test:, ci:, chore:	No version bump

3.4 QuantIQ versioning plan

Milestone	Version	Notes
Initial dev (current)	0.x.x	Major=0 means public API not stable
First working backtest pipeline	0.1.0	feat: first backtest working
Dashboard live	0.2.0	feat: Streamlit dashboard
Phase 4 bot paper trading	0.3.0	feat: bot paper trading mode
Phase 4 bot live trading	1.0.0	First stable release for live orders
SEBI-compliant audit logging	1.1.0	feat: 5-year audit log system
Breaking API refactor	2.0.0	BREAKING CHANGE: signal API

4. Git Essentials

Official docs: git-scm.com/doc | Pro Git book (free): git-scm.com/book/en/v2 | Interactive: learnitbranching.js.org

4.1 Core Git commands

Command	What it does
<code>git init</code>	Initialise a new repo in current directory
<code>git clone</code>	Clone remote repo
<code>git status</code>	Show working tree status
<code>git diff</code>	Unstaged changes vs last commit
<code>git diff --staged</code>	Staged changes vs last commit
<code>git add</code>	Stage file
<code>git add -p</code>	Interactively stage hunks (recommended)
<code>git add .</code>	Stage all changes (use carefully)
<code>git commit -m 'msg'</code>	Commit staged changes
<code>git commit --amend</code>	Modify last commit (before push)
<code>git log --oneline</code>	Compact commit history
<code>git log --oneline --graph</code>	Graph of branches
<code>git show</code>	Show commit details and diff
<code>git stash</code>	Temporarily save uncommitted changes
<code>git stash pop</code>	Restore last stash
<code>git restore</code>	Discard unstaged changes to file
<code>git restore --staged</code>	Unstage a file
<code>git reset HEAD~1</code>	Undo last commit (keep changes staged)
<code>git reset --hard HEAD~1</code>	Undo last commit and discard changes (DANGER)
<code>git revert</code>	Create reverse commit (safe for shared branches)

4.2 Remote operations

Command	What it does
<code>git remote -v</code>	List remotes with URLs
<code>git fetch origin</code>	Download from remote without merging
<code>git pull --rebase origin main</code>	Fetch + rebase onto remote main
<code>git push origin</code>	Push branch to remote
<code>git push -u origin</code>	Push + set upstream tracking
<code>git push --force-with-lease</code>	Force push (safer: fails if remote has new commits)

4.3 Branching

Command	What it does
<code>git branch</code>	List local branches
<code>git branch -a</code>	List all branches (local + remote)
<code>git checkout -b feat/broker/token-refresh</code>	Create and switch to new branch
<code>git switch main</code>	Switch to main

Command	What it does
<code>git switch -c feat/new-feature</code>	Create and switch (modern syntax)
<code>git branch -d feat/done</code>	Delete merged local branch
<code>git push origin --delete feat/done</code>	Delete remote branch

4.4 Merging vs Rebasing

Operation	Command	When to use
Merge	<code>git merge feat/branch</code>	Merge feature branch to main (via PR, not CLI)
Rebase	<code>git rebase main</code>	Update feature branch with latest main (before PR)
Squash	<code>git rebase -i HEAD~N</code>	Clean up N commits before PR
Fast-forward	<code>git merge --ff-only</code>	Merge only if no divergence (no merge commit)

QuantIQ rule: never push directly to main. All changes via pull requests. Rebase feature branches onto main before opening PR.

4.5 .gitignore for QuantIQ

```
# Python
__pycache__/
*.pyc
*.pyo
.eggs/
*.egg-info/
dist/
build/

# Virtual environments
.venv/
venv/
env/

# Secrets (CRITICAL – never commit)
.env
.streamlit/secrets.toml

# Data files (may be large)
data/raw/*.csv
logs/*.log

# IDE
.idea/
.vscode/
*.swp

# OS
.DS_Store
Thumbs.db

# Jupyter
.ipynb_checkpoints/
*.ipynb

# Test artifacts
.pytest_cache/
.coverage
htmlcov/
```

5. Branching Strategy

QuantIQ uses a simplified GitHub Flow (not full GitFlow). Simple, fast, appropriate for a 12-person team.

5.1 Branch structure

Branch	Purpose	Who pushes
main	Production-ready code always. Protected. Never push directly.	Merged PRs only
feat//	New features. Branch from main, merge back via PR.	Individual contributors
fix//	Bug fixes. Same lifecycle as feat/.	Individual contributors
docs/	Documentation-only. Lightweight — can merge without review.	Any contributor
chore/	Tooling, deps, CI. No logic changes.	Any contributor
release/	Release preparation (update version, changelog, final testing).	Lead only

5.2 Feature branch workflow

```
# 1. Create branch from main
git switch main
git pull --rebase origin main
git switch -c feat/data/nse-ticker-validator

# 2. Work — commit often with Conventional Commits
git add -p
git commit -m 'feat(data): add NSE ticker suffix validation'

# 3. Keep branch up to date with main
git fetch origin
git rebase origin/main

# 4. Push and open PR
git push -u origin feat/data/nse-ticker-validator
# Open PR on GitHub -> review -> merge

# 5. Clean up after merge
git switch main
git pull --rebase origin main
git branch -d feat/data/nse-ticker-validator
```

5.3 PR checklist

- ☐ All commits follow Conventional Commits format
- ☐ PR title follows Conventional Commits format
- ☐ Tests pass in CI (GitHub Actions)
- ☐ No secrets committed (check .gitignore and git log)
- ☐ Code follows PEP 8 + Google Style (run Black + Ruff)
- ☐ Docstrings on all new public functions
- ☐ Type hints on all new public functions
- ☐ CONTRIBUTING.md and CLAUDE.md updated if new external dependencies added

6. GitHub Actions — CI/CD

Docs: docs.github.com/en/actions | Quickstart: docs.github.com/en/actions/quickstart

GitHub Actions YAML files live in `.github/workflows/` in the repo. Each file is a workflow. Workflows are triggered by events (push, pull_request, schedule, etc.).

6.1 CI workflow — PR checks

```
# .github/workflows/ci.yml
name: CI

on:
  pull_request:
    branches: [main]
  push:
    branches: [main]

jobs:
  lint-and-test:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          pip install -r requirements.txt
          pip install black ruff pytest

      - name: Run Black formatter check
        run: black --check --diff .

      - name: Run Ruff linter
        run: ruff check .

      - name: Run tests
        run: pytest tests/ -v --tb=short
```

6.2 Dependency review on PR

```
# .github/workflows/dependency-review.yml
name: Dependency Review

on:
  pull_request:

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    permissions:
      contents: read
      pull-requests: write
    steps:
      - uses: actions/checkout@v4
      - uses: actions/dependency-review-action@v4
```

6.3 Workflow syntax reference

Key	Description
on:	Trigger events: push, pull_request, schedule, workflow_dispatch
on.push.branches:	Restrict push trigger to specific branches
on.schedule:	Cron trigger: '15 8 * * 1-5' = 08:15 UTC Mon-Fri
jobs..runs-on:	Runner: 'ubuntu-latest', 'windows-latest', 'macos-latest'
jobs..steps:	List of steps to execute
uses: actions/checkout@v4	Built-in action: clone the repo
uses: actions/setup-python@v5	Built-in action: install Python
run:	Run shell command
env:	Set environment variables for a step or job
secrets:	Access repo secrets: \${ secrets.DHAN_ACCESS_TOKEN }
needs:	Job dependency: needs: [lint-and-test]
if:	Conditional: if: github.ref == 'refs/heads/main'
timeout-minutes:	Kill job if it runs too long
strategy.matrix:	Run job with multiple parameter sets

6.4 Scheduled token refresh alert (example)

```
# .github/workflows/token-reminder.yml
name: Daily Token Reminder

on:
  schedule:
    - cron: '45 3 * * 1-5' # 08:45 IST = 03:45 UTC Mon-Fri

jobs:
  remind:
    runs-on: ubuntu-latest
    steps:
      - name: Remind to regenerate Dhan token
        run: |
          echo 'Reminder: regenerate DhanHQ access token before 09:15 IST'
          echo 'Token expires every 24 hours per SEBI mandate'
          # Add Discord/Slack webhook notification here
```

7. Branch Protection

Docs: docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches

Branch protection rules prevent accidental direct pushes to main and enforce review and CI requirements.

7.1 Recommended settings for main branch

Setting	Recommended value	Why
Require a pull request before merging	ON	No direct pushes to main
Required approving reviews	1	At least one peer review
Dismiss stale PR approvals when new commits pushed	ON	Re-review after changes
Require status checks to pass	ON	CI must pass
Required status checks	CI: lint-and-test	Enforce Green CI
Require branches to be up to date	ON	Prevent merge of stale PRs
Require conversation resolution before merge	ON	All review comments addressed
Restrict who can push	repo Leads only	Only leads can merge
Allow force pushes	OFF	Protect history
Allow deletions	OFF	Cannot delete main

7.2 Setting up via GitHub UI

1. Go to github.com/Quant-IQ/quantiq
2. Settings -> Branches
3. Click 'Add branch protection rule'
4. Branch name pattern: main
5. Enable the settings from the table above
6. Click 'Create'

8. QuantIQ Git Workflow Summary

8.1 Daily contributor workflow

```
# Start of work session
git switch main
git pull --rebase origin main

# Create branch
git switch -c feat/strategy/rsi-crossover-signal

# Make changes, commit incrementally
git add -p
git commit -m 'feat(strategy): add RSI oversold entry condition'
git add -p
git commit -m 'test(strategy): add unit tests for RSI crossover'

# Before opening PR: rebase and check
git fetch origin
git rebase origin/main
black . # format
ruff check . # lint
pytest tests/ # test

# Push and open PR
git push -u origin feat/strategy/rsi-crossover-signal
# -> go to GitHub -> open PR with Conventional Commit title
```

8.2 Role of handles in workflow

Role	Git responsibility
RS (Lead)	Merge PRs, manage releases, maintain branch protection
Dev roles (AK, NS, ShS)	Feature branches, CI-passing PRs, follow commit convention
Quant roles (SS, RT, AV, GT)	Strategy branches, backtest scripts, signal modules
Data roles (AR, AJ, SmS)	Data pipeline branches, fetcher modules
Docs roles (EB, ShS)	Docs branches, update CONTRIBUTING.md, CLAUDE.md

8.3 Release checklist

- ☐ Checkout main and pull latest
- ☐ Create release/X.Y.Z branch
- ☐ Update version in pyproject.toml: version = 'X.Y.Z'
- ☐ Update CHANGELOG.md (generated from git log or git-cliff)
- ☐ Open PR: chore(release): bump version to X.Y.Z
- ☐ After merge: git tag vX.Y.Z main && git push origin vX.Y.Z
- ☐ Create GitHub Release from tag with CHANGELOG extract

9. See Also

Resource	URL
Conventional Commits spec	conventionalcommits.org/en/v1.0.0/
Semantic Versioning	semver.org/
Git official docs	git-scm.com/doc
Pro Git book (free)	git-scm.com/book/en/v2
Learn Git Branching (interactive)	learngitbranching.js.org/
GitHub Docs	docs.github.com/
GitHub Actions quickstart	docs.github.com/en/actions/quickstart
GitHub branch protection	docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches
QuantIQ repo	github.com/Quant-IQ/quantiq
QuantIQ Pages	quant-iq.github.io/quantiq
git-cliff changelog generator	github.com/orhun/git-cliff

Cross-references to other binder modules

Module	Relevance
M7 — Python Standards	Black and Ruff run in CI (§6.1); commit types align with naming conventions
M10 — Environment	pyproject.toml holds version string; pre-commit hooks run Black/Ruff before commits
M0 — Master Index	Cross-reference matrix shows M8 as dependency of M10

End of Module 8. Next: Module 9 — Regulatory (SEBI and NSE).

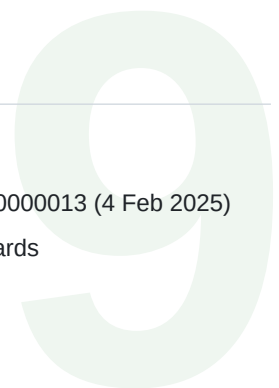
QuantIQ

Master Reference Binder

Module 9

Regulatory — SEBI & NSE

Version	18 May 2026
Primary source	SEBI Circular SEBI/HO/MIRSD/MIRSD-PoD/P/CIR/2025/0000013 (4 Feb 2025)
NSE source	NSE/INVG/67858 (5 May 2025) — Implementation Standards
Full framework live	April 1, 2026 (all brokers)
Critical rule	10 OPS threshold: below = no registration needed
Pages	approx 22



Module Contents

§	Section	Page
1	Quick Reference — Hard Constraints	3
2	Regulatory Timeline	5
3	SEBI Circular — Full Text Summary	6
4	NSE Implementation Standards (NSE/INVG/67858)	11
5	QuantIQ Compliance Checklist	16
6	Circular URL Reference	18
7	Glossary	19
8	See Also	20

1. Quick Reference — Hard Constraints

READ THIS PAGE BEFORE TOUCHING ANY LIVE ORDER CODE.

Every rule below is legally binding on Dhan (your broker) and by extension on your API usage. Violations can result in your API access being revoked by Dhan without notice.

Order rate — the most critical rule

Rule	Detail
THRESHOLD: 10 OPS	Orders Per Second per exchange per client. If your bot stays BELOW 10 OPS, no algo registration with the exchange is required. Above 10 OPS: you must register the algo with NSE/BSE through Dhan. QuantIQ Phase 4 must stay under 10 OPS at all times.
How OPS is measured	Based on calendar clock second. e.g. 10 orders in the second from 09:15:30.000 to 09:15:30.999 = 10 OPS = at threshold. 11th order in that same second = crosses threshold.
Rate limit in code	Implement a token bucket or sliding window limiter. Never rely on Dhan to throttle — they will reject orders above the limit with DH-904 error.

Access and authentication rules

Rule	Detail
Static IP mandatory	One primary + one optional secondary static IP must be whitelisted with Dhan before placing any order. 7-day cooldown after setting — plan ahead.
OAuth + 2FA only	All other authentication mechanisms discontinued. TOTP or equivalent 2FA required on every login session.
24-hour token expiry	Access tokens expire every 24 hours. Regenerate before 09:15 IST daily for Phase 4 bot. Token cannot be refreshed after expiry — must generate fresh.
Unique API key per client	Open APIs not permitted. Each client gets a unique vendor-client API key. Dhan enforces this at infrastructure level.

Operational rules

Rule	Detail
Algo tagging	Every algo order must carry the exchange-provided unique identifier (algo ID). Dhan assigns this when you register. Orders without algo ID are rejected if above the OPS threshold.
Daily session logout	No persistent sessions across trading days. Bot must logout and re-authenticate daily. Session cannot be carried over midnight.
Audit trail 5 years	All algo orders and trades must be audit-logged. Logs must be available for at least 5 years. Use TimedRotatingFileHandler (see M7 §6).
Kill switch	Exchange can halt all orders from a specific algo ID at any time. Your bot must handle sudden order rejection gracefully — never crash on DH-904.
Grievance via broker only	All complaints about algo trading must go through Dhan, not directly to NSE. Dhan is solely responsible for investor grievances.

2. Regulatory Timeline

Date	Event
9 Dec 2021	SEBI publishes discussion paper 'Algorithmic Trading by Retail Investors'. First public articulation of the framework concept.
4 Feb 2025	SEBI issues binding circular SEBI/HO/MIRSD/MIRSD-PoD/P/CIR/2025/0000013. Master circular for retail algo framework. Original implementation: Aug 1, 2025.
5 Feb 2025	NSE issues initial circular NSE/INVG/66524 acknowledging the SEBI circular.
1 Apr 2025	Implementation standards formulation deadline (Broker Industry Standards Forum).
5 May 2025	NSE issues implementation standards NSE/INVG/67858 with full technical details (static IP, OPS threshold = 10, OAuth, 2FA, audit trail).
22 Jul 2025	Exchanges issue detailed operational modalities.
29 Jul 2025	SEBI extends implementation from Aug 1 to Oct 1, 2025 (broker representations).
30 Sep 2025	SEBI issues second extension: glide path for brokers. Milestones set.
1 Oct 2025	Brokers with ready systems can go live. Others on glide path.
31 Oct 2025	Milestone 1: Each broker must register at least one retail algo product with exchange.
30 Nov 2025	Milestone 2: Completion of registration of retail algo products.
3 Jan 2026	Milestone 3: All brokers must complete at least one mock session.
5 Jan 2026	Brokers failing milestones barred from onboarding new API algo clients.
1 Apr 2026	FULL FRAMEWORK mandatory for all brokers. All operational modalities apply.
18 May 2026	This binder printed (current date). Framework is fully live.

3. SEBI Circular — Full Text Summary

Circular: SEBI/HO/MIRSD/MIRSD-PoD/P/CIR/2025/0000013 | Date: 4 February 2025 | URL: sebi.gov.in/legal/circulars/feb-2025/safer-participation-of-retail-investors-in-algorithmic-trading_91614.html

The following is a structured summary of the circular's provisions, organised by section. Quoted passages are reproduced from the official PDF for precision. Read the full circular at the URL above.

3.1 Context and purpose

SEBI issued this circular in response to growing retail demand for algo trading access via APIs. The framework builds on the 2012 algo guidelines and the 2021 discussion paper, aiming to balance innovation with investor protection.

"In order to facilitate safer participation of retail investors in algo trading, with Stock Brokers and Stock Exchanges playing the required roles in risk management, it has been decided to review and refine the existing regulatory framework to ensure proper checks and balances, to safeguard investor interest as well as integrity of the market."
— §3

3.2 Section I — API Usage for Algo Trading

The core framework for API-based algo trading:

Para	Rule	Engineering implication
I(a)	Broker = principal; Algo Provider = agent.	Dhan owns the API and bears compliance responsibility. You, as an API user, are the 'Algo Provider' (agent).
I(b)	All algo orders via API must be tagged with exchange-provided unique identifier.	Every order must carry the algo ID assigned by NSE/BSE when you register. Dhan SDK handles this automatically when you use the registered algo.
I(c)	Self-developed retail algos only need registration if they cross the OPS threshold.	Below 10 OPS = no registration needed. Above = must register with exchange through Dhan. Family sharing permitted (spouse, dependent children/parents).
I(d)(i)	Broker must detect and categorise orders above threshold as algo orders.	Dhan's infrastructure handles this. Your rate limiter keeps you below 10 OPS.
I(d)(ii)	No open APIs. Unique vendor-client API key + static IP whitelist mandatory.	Dhan enforces unique API key. You must call <code>DhanLogin.set_ip()</code> before first order. 7-day cooldown after IP setting.
I(d)(iii)	OAuth-based authentication only. All other auth mechanisms discontinued.	Use TOTP or OAuth flow. Direct username+password API calls are discontinued.
I(d)(iv)	Two-factor authentication mandatory for API access.	TOTP (see M1 §3.6) or OAuth flow includes 2FA by design.
I(d)(v)	Broker deals with empanelled algo providers only.	QuantIQ uses Dhan directly — not a third-party algo provider. No empanelment needed as long as orders stay under 10 OPS.

3.3 Section II — Broker Responsibilities

Para	Rule
II(a)	Broker provides algo trading only after exchange permission for each algo.
II(b)	All algo orders tagged with exchange-unique identifier. Broker seeks exchange approval for any modification to approved algos.
II(c)	Broker solely responsible for investor grievances related to algo trading. Broker monitors APIs for prohibited activities.

3.4 Section III — Empanelment of Algo Providers

Applicable to third-party algo vendors, not self-developed retail algos below the OPS threshold. QuantIQ is not affected by this section as long as the bot stays under 10 OPS.

3.5 Section IV — Exchange Responsibilities

Rule	Detail
Testing SOP	Exchanges maintain Standard Operating Procedure for algo testing.
Surveillance	Continuous surveillance on all algo orders including simulation testing.
Kill switch	Exchange can halt all orders from a specific algo ID at any time. Your bot must handle this without crashing.
Operational modalities	Exchanges issue detailed FAQs/modalities covering risk management, empanelment, registration, confidentiality, data flow.
Registration TAT	Exchanges specify turnaround time for algo registration: fast-track (execution algos) and normal track.

3.6 Section V — Categorisation of Algos

Category	Definition	QuantIQ impact
White Box / Execution Algo	Logic fully disclosed and replicable. User can see how it works.	QuantIQ strategies are white-box by nature — no additional requirement.
Black Box Algo	Logic not known to the user or not replicable. Proprietary algorithms.	Not applicable to QuantIQ. All strategy logic is visible in the codebase.

Black box algo providers must: register as Research Analysts with SEBI; maintain a detailed research report for each algo; register fresh algo on any logic change. QuantIQ is not in this category.

3.7 Verbatim key provisions

"Brokers shall not permit open APIs and allow access only through a unique vendor client specific API key and static IP whitelisted by the broker to ensure identification and traceability of the algo provider and the end user (i.e. investor)." — §I(d)(ii)

"Brokers shall have OAuth (Open Authentication) based authentication only and all other authentication mechanisms shall be discontinued." — §I(d)(iii)

"Algos developed by tech-savvy retail investors themselves, using programming knowledge, shall also be registered with the Exchange, through their broker, only if they cross the specified order per second threshold." — §I(c)

4. NSE Implementation Standards (NSE/INVG/67858)

Circular: NSE/INVG/67858 | Date: 5 May 2025 | URL: nseindia.com/resources/exchange-communication-circulars

This circular translates SEBI's Feb 2025 framework into concrete technical standards. It is the engineering spec for algo API compliance on NSE. Every QuantIQ Phase 4 feature must satisfy these standards.

4.1 Section A — API Access Standards

Standard	Detail	QuantIQ action
Static IP mandatory	Clients must provide at least one static IP (primary). Secondary optional for redundancy. Must be provided to broker before any API order placement.	Call <code>DhanLogin.set_ip()</code> once with primary IP. Document in bot startup log.
One static IP or two	Client may give ONE primary, or primary + secondary IP.	Single static IP is sufficient for QuantIQ.
IP change request	Written request / email from registered email / 2FA-validated request to change.	Plan IP changes 7 days in advance. Cannot change day-before live trading.
Written consent	Client must give written consent to broker for API access including for internet-based trading.	This is handled at Dhan account onboarding, not in code.

4.2 Section B — APIs without Algo Registration (below OPS threshold)

This is the most important section for QuantIQ Phase 4.

Standard	Detail
OPS threshold: 10	Threshold Order Per Second = 10 OPS per exchange. Measured on calendar clock second (not rolling window).
Below threshold = no registration	If the bot stays below 10 OPS on any exchange, no algo registration is required with NSE or BSE.
Automated systems permitted	Clients can use API for automated trading systems, computer algorithms, or software-based solutions to automate buying and selling.
Risk management required	The API system must have risk management features including position limits, order value limits, loss limits.
Audit trail: 5 years	Audit trail for all API orders/trades available for at least 5 years.
2FA required	System must authenticate client access through two-factor authentication.
Password expiry	Automatic password expiry at end of reasonable duration, re-initialisation on fresh password entry.

4.3 OPS rate limiter implementation

Your bot **MUST** implement a software rate limiter before calling any Dhan order API. Use a token bucket or sliding window approach:

```
import time
import threading
from collections import deque

class OPSRateLimiter:
    """Token bucket rate limiter: max 10 orders/sec per SEBI mandate."""

    MAX_OPS: int = 10 # SEBI/NSE hard limit

    def __init__(self) -> None:
        self._window: deque = deque()
```

```

self._lock = threading.Lock()

def can_place_order(self) -> bool:
    """Return True if placing an order now stays within 10 OPS."""
    now = time.monotonic()
    with self._lock:
        # Remove timestamps older than 1 second
        while self._window and now - self._window[0] >= 1.0:
            self._window.popleft()
        if len(self._window) >= self.MAX_OPS:
            return False
        self._window.append(now)
        return True

def wait_and_place(self) -> None:
    """Block until an order slot is available."""
    while not self.can_place_order():
        time.sleep(0.05) # check every 50ms

# Usage
rate_limiter = OPSRateLimiter()
rate_limiter.wait_and_place() # blocks if at limit
response = dhan.place_order(...)

```

4.4 Section C — Registered Algos (above OPS threshold)

If QuantIQ ever needs to exceed 10 OPS, these rules apply:

Rule	Detail
Register with exchange	Must register the algorithm with each exchange it will be used on.
Exchange-specific algo ID	Each registered algo gets a unique ID from the exchange. All orders must carry this ID.
Simplified process for execution algos	Execution algos (white box, transparent logic) get fast-track registration.
Re-registration on logic change	Any change to the algo logic requires re-registration.
Mock sessions	Brokers must participate in exchange-organised mock trading sessions.

QuantIQ Phase 4 must stay below 10 OPS to avoid this process. If strategy evolution demands >10 OPS, consult Dhan for registration.

4.5 Section D and E — Broker and Algo Provider Algos

These sections cover algos created by Dhan itself and third-party algo vendors registered with NSE. Not directly applicable to QuantIQ's self-built strategies.

4.6 Audit trail requirements

"The broker should ensure sound audit trail for all IBT / STWT / Client API / Vendor API orders and trades and be able to provide identification of actual user and user-id for all such orders and trades. The audit trail data should be available for at least 5 years." — NSE/INVG/67858 §I(a)

Engineering implication: every order placed by the Phase 4 bot must be logged with: timestamp, user ID (dhanClientId), symbol, side, quantity, price, order ID, order status. Logs stored with TimedRotatingFileHandler at backupCount=1825 (5 years). See M7 §6.5.

4.7 Security requirements

Requirement	Detail	Implementation
Password protection	API system must have password protection for user/user-ID.	Dhan access token serves as the credential. Stored in secrets.toml (M5 §10).

Requirement	Detail	Implementation
Automatic password expiry	Password expires at end of reasonable duration.	24-hour token expiry satisfies this. Regenerate daily before 09:15 IST.
Re-initialisation on fresh password	Fresh credential on each session.	New access token generated daily by bot startup script.
2FA authentication	API access must authenticate via 2FA.	TOTP-based token generation satisfies this (see M1 §3.2/3.6).

5. QuantIQ Compliance Checklist

Verify every item before Phase 4 goes live. Each item maps to a specific regulatory requirement.

Authentication & access

[] Check	SEBI/NSE ref	Binder ref
[] Static IP set via DhanLogin.set_ip()	NSE/INVG/67858 §A	M1 §3.5
[] Primary IP documented in config (not hardcoded in code)	NSE/INVG/67858 §A	M10 §2
[] OAuth + TOTP configured in Dhan account	SEBI §I(d)(iii,iv)	M1 §3.6
[] Daily access token regeneration before 09:15 IST implemented	SEBI §I(d)(ii)	M1 §3.2
[] Token stored in secrets.toml (not committed to git)	M5 §10	M8 §5
[] Bot verifies token validity via /profile on startup	M1 §3.7	M1 §3.7

Order rate control

[] Check	SEBI/NSE ref	Binder ref
[] OPS rate limiter implemented in code (max 10/sec)	NSE/INVG/67858 §B	M9 §4.3
[] Rate limiter tested under load (10+ rapid order calls)	NSE/INVG/67858 §B	M4
[] Bot handles DH-904 (rate limit error) gracefully, never crashes	M1 §11	M1 §4
[] Order queue processed sequentially, not concurrently, if near limit	NSE/INVG/67858 §B	M9 §4.3

Audit logging

[] Check	SEBI/NSE ref	Binder ref
[] Every order logged: ts, clientId, symbol, side, qty, price, orderId, status	NSE/INVG/67858 §I(a)	M7 §6.5
[] TimedRotatingFileHandler with backupCount=1825 (5 years)	NSE/INVG/67858 §I(a)	M7 §6.3
[] Log files NOT in git repo (in .gitignore)	Security	M8 §4.5
[] Audit log survives bot crash (writes on each order, not at end)	NSE/INVG/67858 §I(a)	M7 §6
[] Log includes algo ID / bot version for traceability	SEBI §II(b)	M7 §6.5

Session management

[] Check	SEBI/NSE ref	Binder ref
[] Bot does NOT persist sessions across trading days	SEBI framework	M1 §3.2
[] Daily logout implemented after market close (15:30 IST)	SEBI framework	M1 §3.2
[] Bot startup sequence: login -> verify token -> set IP -> warmup -> ready	NSE/INVG/67858 §A	M1
[] Bot shutdown sequence: cancel open orders -> logout -> log shutdown	SEBI §II	M7 §6

Risk management

[] Check	SEBI/NSE ref	Binder ref
[] Daily loss limit implemented: bot stops if P&L; < -X%	NSE/INVG/67858 §B	M4
[] Position limit implemented: max open positions per symbol	NSE/INVG/67858 §B	M4
[] Max order value cap per order implemented	NSE/INVG/67858 §B	M1 §4
[] Kill switch: bot can be manually stopped without open orders	SEBI §IV(a)(iii)	M1
[] Backtest completed on 2+ years of data before going live	Best practice	M4

6. Circular URL Reference

Document	URL	Date
SEBI master circular (primary)	sebi.gov.in/legal/circulars/feb-2025/safer-participation-of-retail-investors-in-algorithmic-trading_91614.html	4 Feb 2025
SEBI extension circular (Apr 2025)	sebi.gov.in/legal/circulars/apr-2025/extension-of-timeline-for-formulation-of-implementation-standards-pertaining-to-sebi-circular-on-safer-participation-of-retail-investors-in-algorithmic-trading-_93166.html	Apr 2025
SEBI extension circular (Sep 2025)	sebi.gov.in/legal/circulars/sep-2025/extension-of-timeline-for-implementation-of-sebi-circular-dated-february-04-2025-on-safer-participation-of-retail-investors-in-algorithmic-trading-_96979.html	30 Sep 2025
NSE implementation standards (primary)	nsearchives.nseindia.com/content/circulars/INVG67858.pdf	5 May 2025
NSE operational modalities (Jul 2025)	nseindia.com/resources/exchange-communication-circulars (search NSE/INVG/67858)	22 Jul 2025
SEBI circular index (live — track new amendments)	sebi.gov.in/sebiweb/home/HomeAction.do?doListingAll=yes&search;=Algorithmic	Live
NSE circular search portal	nseindia.com/resources/exchange-communication-circulars	Live
SEBI 2021 discussion paper	sebi.gov.in/reports-and-statistics/reports/dec-2024/participation-of-retail-investors-in-algorithmic-trading_89837.html	9 Dec 2021

7. Glossary

Term	Definition
Algo / Algorithm	An automated trading system using pre-defined logic for order placement.
Algo Provider	Entity providing algo trading facility to investors via broker API. Acts as broker's agent.
API	Application Programming Interface — the technical channel for algo orders.
Black Box Algo	Algo with undisclosed logic, not replicable or transparent to user.
Broker	Stock broker (Dhan in QuantIQ's case). Principal under the SEBI framework.
DMA	Direct Market Access — existing facility for institutional algo trading.
Empanelment	Registration of algo providers with stock exchanges. Not required for self-built retail algos below OPS threshold.
Execution Algo / White Box	Transparent algo where logic is fully accessible and replicable by user.
Family	For the purpose of algo sharing: self, spouse, dependent children, dependent parents.
IBT	Internet-Based Trading — category of trading access that includes API trading.
ISF	Broker's Industry Standards Forum — formulated OPS threshold and implementation standards.
Kill Switch	Emergency function to halt all trading activity from a specific algo ID.
MII	Market Infrastructure Institution — exchanges, depositories, clearing corporations.
NSE	National Stock Exchange of India Limited.
OAuth	Open Authentication — secure authorization framework without sharing login credentials.
OPS	Orders Per Second — rate limit for algo classification. Threshold: 10 OPS per exchange.
SEBI	Securities and Exchange Board of India — primary market regulator.
Static IP	Fixed, unchanging IP address. Mandatory for API order placement.
STWT	Smart Terminal With Triggers — another category of trading access.
TOTP	Time-based One-Time Password — 2FA method used with Dhan API.

8. See Also

Resource	URL
SEBI website	sebi.gov.in
SEBI circular index (all algo)	sebi.gov.in/sebiweb/home/HomeAction.do?doListingAll=yes&search;=Algorithmic
NSE circulars portal	nseindia.com/resources/exchange-communication-circulars
NSE market data	nseindia.com/market-data
NSE securities list (EQUITY_L.csv)	nseindia.com/market-data/securities-available-for-trading
Dhan API rate limit FAQ	dhan.co/support/platforms/dhanhq-api/how-many-orders-can-i-placed-using-dhan-api/
Zerodha Varsity — markets overview	zerodha.com/varsity/modules/

Cross-references to other binder modules

Module	Relevance
M1 — DhanHQ	Set IP (§3.5), TOTP/OAuth (§3.2/3.6), token refresh, /profile check on startup
M7 — Python Standards	TimedRotatingFileHandler for 5-year audit logs (§6.3), structured audit log (§6.5)
M5 — Streamlit	secrets.toml for token storage (§10), never commit credentials
M4 — vectorbt	Complete backtest required before live trading (compliance checklist §5)
M0 — Master Index	Hard constraints master list cross-references this module throughout

End of Module 9. Next: Module 10 — Environment and Tooling.

QuantIQ

Master Reference Binder

Module 10

Environment & Tooling

Version	18 May 2026
Scope	python-dotenv, venv/pip, requirements.txt, pyproject.toml, pre-commit, .gitignore
Python version	$\geq 3.11, < 3.13$ (QuantIQ constraint)
Editor	VS Code (project standard — not VSCodium)
Pages	approx 18

10

Module Contents

§	Section	Page
1	Quick Reference Card	3
2	python-dotenv — Environment Variables	4
3	Virtual Environments (venv)	6
4	pip — Package Management	8
5	requirements.txt — Dependency Pinning	10
6	pyproject.toml — Project Configuration	12
7	pre-commit Hooks	15
8	VS Code Setup for QuantIQ	16
9	QuantIQ .gitignore (Complete)	17
10	See Also	18

1. Quick Reference Card

Project setup from scratch

```
# 1. Clone repo
git clone https://github.com/Quant-IQ/quantiq.git
cd quantiq

# 2. Create virtual environment
python3.11 -m venv .venv
source .venv/bin/activate # macOS/Linux
# .venv\Scripts\activate # Windows

# 3. Install dependencies
pip install -r requirements.txt
pip install -r requirements-dev.txt

# 4. Set up secrets
cp .streamlit/secrets.toml.example .streamlit/secrets.toml
# Edit secrets.toml with your Dhan credentials

# 5. Install pre-commit hooks
pre-commit install

# 6. Verify
python -c "import vectorbt, dhanhq, streamlit; print('OK')"
```

Most-used pip commands

Command	Purpose
<code>pip install -r requirements.txt</code>	Install all pinned project dependencies
<code>pip install -r requirements-dev.txt</code>	Install dev tools (Black, Ruff, pytest, pre-commit)
<code>pip install <package>==<version></code>	Install specific version
<code>pip install --upgrade <package></code>	Upgrade a package
<code>pip freeze > requirements.txt</code>	Snapshot current env to requirements.txt
<code>pip list --outdated</code>	Show packages with newer versions
<code>pip show <package></code>	Package info: version, location, deps
<code>pip uninstall <package></code>	Uninstall a package
<code>pip check</code>	Verify no dependency conflicts

python-dotenv one-liners

Code	Effect
<code>from dotenv import load_dotenv; load_dotenv()</code>	Load .env into os.environ
<code>os.getenv('DHAN_CLIENT_ID')</code>	Read env var (returns None if missing)
<code>os.environ['DHAN_ACCESS_TOKEN']</code>	Read env var (raises KeyError if missing)
<code>load_dotenv(override=True)</code>	Override existing env vars with .env values
<code>load_dotenv('.env.local')</code>	Load from a specific file path

2. python-dotenv — Environment Variables

GitHub: github.com/theskumar/python-dotenv | PyPI: pypi.org/project/python-dotenv/ | Version: 1.2.2 | Requires: Python >=3.10

python-dotenv loads environment variables from a .env file into os.environ. Use for local development. In production (Streamlit Community Cloud) use st.secrets instead (M5 §10). Never commit .env to git.

2.1 Install

```
pip install python-dotenv==1.2.2
```

2.2 .env file format

Create .env in project root. Add to .gitignore immediately.

```
# .env – local development only. NEVER COMMIT.

# DhanHQ credentials
DHAN_CLIENT_ID=1000000001
DHAN_ACCESS_TOKEN=eyJhbGciOiJIUzI1NiJ9...

# Environment flag
ENV=development
LOG_LEVEL=DEBUG

# Optional: custom data directory
DATA_DIR=/home/user/quantiq_data
```

.env syntax rules:

- KEY=VALUE — no spaces around =
- # starts a comment line
- Empty lines are ignored
- No quotes needed for values unless value contains spaces
- Quoted values: KEY='value with spaces' or KEY="value"
- Multiline values: KEY="line1\nline2"

2.3 Loading in code

```
import os
from dotenv import load_dotenv

# Load at app entry point (before any os.getenv calls)
load_dotenv() # reads .env from cwd or parent dirs

# Read values
client_id = os.getenv('DHAN_CLIENT_ID')
access_token = os.getenv('DHAN_ACCESS_TOKEN') # raises if missing

# Safe pattern with default
env = os.getenv('ENV', 'development')

# In bot/main.py – load once at startup
if __name__ == '__main__':
    load_dotenv()
    setup_logging()
    main()
```

2.4 Validation pattern (recommended)

```
import os
from dotenv import load_dotenv

REQUIRED_ENV_VARS = [
    'DHAN_CLIENT_ID',
    'DHAN_ACCESS_TOKEN',
]

def validate_env() -> None:
    """Raise on startup if required env vars are missing."""
    load_dotenv()
    missing = [k for k in REQUIRED_ENV_VARS if not os.getenv(k)]
    if missing:
        raise EnvironmentError(
            f"Missing required environment variables: {missing}\n"
            "Check .env file or secrets.toml."
        )

validate_env() # call at module import time
```

2.5 dotenv vs st.secrets

	python-dotenv (.env)	st.secrets (secrets.toml)
Use when	Local development, bot script, non-Streamlit code	Streamlit dashboard, Community Cloud deployment
File location	.env in project root	.streamlit/secrets.toml
Access	os.getenv('KEY')	st.secrets['KEY']
Commit?	NEVER	NEVER (paste into Community Cloud UI)
Format	KEY=VALUE	TOML: KEY = 'value'

3. Virtual Environments (venv)

venv is part of the Python stdlib. Use it for every project. Never install QuantIQ dependencies into the system Python.

3.1 Create and activate

```
# Create (use Python 3.11 explicitly)
python3.11 -m venv .venv

# Activate
source .venv/bin/activate # macOS/Linux (bash/zsh)
.venv\Scripts\activate # Windows (cmd)
.venv\Scripts\Activate.ps1 # Windows (PowerShell)

# Verify
which python # should point to .venv/bin/python
python --version # should be 3.11.x

# Deactivate when done
deactivate
```

3.2 Naming convention

QuantIQ convention: always name the venv directory .venv (dot prefix). This is already in .gitignore and hidden in most file explorers.

3.3 venv lifecycle

Task	Command
Create	python3.11 -m venv .venv
Activate (Linux/Mac)	source .venv/bin/activate
Activate (Windows)	.\venv\Scripts\activate
Deactivate	deactivate
Delete and recreate	rm -rf .venv && python3.11 -m venv .venv
Check Python path	which python (Linux/Mac) or where python (Windows)
Install project deps	pip install -r requirements.txt
Verify installed	pip list

3.4 VS Code integration

Open Command Palette (Cmd/Ctrl+Shift+P) -> 'Python: Select Interpreter' -> choose .venv/bin/python. VS Code will auto-activate on terminal open.

```
# .vscode/settings.json – commit this to repo
{
  "python.defaultInterpreterPath": "./.venv/bin/python",
  "python.formatting.provider": "none",
  "editor.formatOnSave": true,
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter"
  }
}
```

4. pip — Package Management

4.1 Core pip commands

Command	Purpose
<code>pip install <pkg></code>	Install latest version
<code>pip install <pkg>==1.2.3</code>	Install exact version (preferred for reproducibility)
<code>pip install <pkg>>=1.2,<2</code>	Install within version range
<code>pip install -r requirements.txt</code>	Install all from requirements file
<code>pip install -r requirements-dev.txt</code>	Install dev tools
<code>pip install -e .</code>	Install current package in editable mode (development)
<code>pip freeze</code>	List all installed packages with pinned versions
<code>pip freeze > requirements.txt</code>	Snapshot environment (careful: includes everything)
<code>pip list</code>	Human-readable list of installed packages
<code>pip list --outdated</code>	Packages with newer versions available
<code>pip show <pkg></code>	Package details: version, location, dependencies
<code>pip uninstall <pkg></code>	Uninstall a package
<code>pip check</code>	Check for dependency conflicts
<code>pip cache purge</code>	Clear pip download cache
<code>pip install --no-cache-dir -r requirements.txt</code>	Install without cache (CI/CD)

4.2 Version specifiers

Specifier	Meaning	Use
<code>==1.2.3</code>	Exact version	Production requirements.txt
<code>>=1.2.3</code>	Greater than or equal	requirements-dev.txt ranges
<code>>=1.2,<2</code>	Range	When major version changes are breaking
<code>~=1.2.3</code>	Compatible: <code>>=1.2.3,<1.3</code>	Same minor series
<code>!=1.2.3</code>	Exclude one version	Known bad version
<code>*</code>	Any version	Never use in requirements files

4.3 Upgrading packages safely

```
# Check what would change
pip install <pkg> --dry-run

# Upgrade single package
pip install --upgrade <pkg>

# After upgrading, retest, then pin:
pip freeze | grep <pkg> >> requirements.txt

# Never run:
pip install --upgrade -r requirements.txt # upgrades everything
# This can break things. Upgrade one package at a time.
```


5. requirements.txt — Dependency Pinning

QuantIQ uses two requirements files: requirements.txt (runtime) and requirements-dev.txt (development tools). Both are committed to git.

5.1 requirements.txt — production/runtime

```
# requirements.txt
# Generated: 18 May 2026
# Python: 3.11.x

# Broker API
dhanhq==2.1.0

# Data
pandas>=2.0.0,<3.0
numpy>=1.24.0,<2.0
yfinance>=0.2.50

# Technical analysis
ta==0.11.0

# Backtesting (open-source only – NOT >=1.0 which is VectorBT PRO)
vectorbt==0.28.2
numba>=0.57,<0.61
llvmlite>=0.40,<0.44

# Dashboard
streamlit>=1.35.0
plotly>=5.20.0

# Environment
python-dotenv==1.2.2

# Utilities
requests>=2.28.0
websockets>=12.0
```

5.2 requirements-dev.txt — development tools

```
# requirements-dev.txt
# Dev/test tools – not needed in production

# Install project deps first
-r requirements.txt

# Formatting
black>=24.0

# Linting
ruff>=0.4.0

# Type checking
mypy>=1.8
pandas-stubs>=2.0

# Testing
pytest>=8.0
pytest-cov>=5.0

# Git hooks
pre-commit>=3.7

# Export tools
kaleido>=0.2.1 # Plotly PNG/PDF export
```

```
# Jupyter (optional, for research notebooks)
jupyterlab>=4.0
```

5.3 Pinning strategy

Package category	Strategy	Reason
Safety-critical (dhanhq, vectorbt)	==exact	Breaking changes in minor versions
Stable libraries (pandas, numpy)	>=X.Y,<Z	Some flexibility, but avoid major bumps
Dev tools (black, ruff, pytest)	>=X.Y	Tool updates rarely break production code
New/unknown packages	==exact first	Pin on first use; relax after testing

5.4 Updating dependencies

```
# Check for outdated packages
pip list --outdated

# Test upgrade of one package in isolation
pip install dhanhq==2.2.0
pytest tests/ # run tests

# If tests pass, update requirements.txt
# Change: dhanhq==2.1.0
# To: dhanhq==2.2.0

# Commit as: build(deps): upgrade dhanhq from 2.1.0 to 2.2.0
```

6. pyproject.toml — Project Configuration

pyproject.toml is the modern Python project config file (PEP 517/518). It replaces setup.py and setup.cfg. Configure Black, Ruff, mypy, and pytest here alongside project metadata.

6.1 Complete QuantIQ pyproject.toml

```
[project]
name = "quantiq"
version = "0.3.0"
description = "QuantIQ – NSE Algorithmic Trading"
readme = "README.md"
license = {file = "LICENSE"}
requires-python = ">=3.11,<3.13"
authors = [
  {name = "QuantIQ Team", email = "quantiq.team@quant-iq.net"}
]
keywords = ["algorithmic trading", "NSE", "India", "quantitative finance"]

dependencies = [
  "dhanhq==2.1.0",
  "pandas>=2.0.0,<3.0",
  "numpy>=1.24.0,<2.0",
  "yfinance>=0.2.50",
  "ta==0.11.0",
  "vectorbt==0.28.2",
  "streamlit>=1.35.0",
  "plotly>=5.20.0",
  "python-dotenv==1.2.2",
]

[project.optional-dependencies]
dev = [
  "black>=24.0",
  "ruff>=0.4.0",
  "mypy>=1.8",
  "pytest>=8.0",
  "pytest-cov>=5.0",
  "pre-commit>=3.7",
]

[project.urls]
Repository = "https://github.com/Quant-IQ/quantiq"
Homepage = "https://quant-iq.github.io/quantiq"

# --- Black formatter ---
[tool.black]
line-length = 88
target-version = ['py311']
include = '\.pyi?$'

# --- Ruff linter ---
[tool.ruff]
line-length = 88
target-version = "py311"

[tool.ruff.lint]
select = [
  "E", # pycodestyle errors
  "F", # pyflakes
  "I", # isort
  "N", # pep8-naming
  "UP", # pyupgrade
  "B", # flake8-bugbear
]
ignore = ["E501"] # line length handled by Black
```

```
[tool.ruff.lint.isort]
known-first-party = ["quantiq"]

# --- mypy type checker ---
[tool.mypy]
python_version = "3.11"
strict = false
warn_unused_ignores = true
disallow_untyped_defs = true
ignore_missing_imports = true

# --- pytest ---
[tool.pytest.ini_options]
testpaths = ["tests"]
addopts = [
    "-v",
    "--tb=short",
    "--cov=quantiq",
    "--cov-report=term-missing",
]

# --- Coverage ---
[tool.coverage.run]
source = ["quantiq"]
omit = ["tests/*", ".venv/*"]
```

7. pre-commit Hooks

pre-commit runs checks automatically before every git commit. If any check fails, the commit is aborted. Keeps the codebase clean without manual effort. Docs: pre-commit.com

7.1 Install and setup

```
# Install pre-commit
pip install pre-commit

# Install git hooks (run once per clone)
pre-commit install

# Run manually on all files
pre-commit run --all-files

# Update hook versions
pre-commit autoupdate
```

7.2 .pre-commit-config.yaml for QuantIQ

```
# .pre-commit-config.yaml
# Commit this file to the repo
repos:

# Basic file hygiene
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-json
    - id: check-toml
    - id: check-merge-conflict
    - id: debug-statements
    - id: detect-private-key
    - id: check-added-large-files
  args: ['--maxkb=500']

# Black formatter
- repo: https://github.com/psf/black
  rev: 24.4.2
  hooks:
    - id: black
  language_version: python3.11

# Ruff linter
- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.4.7
  hooks:
    - id: ruff
  args: [--fix]

# No secrets committed
- repo: https://github.com/Yelp/detect-secrets
  rev: v1.5.0
  hooks:
    - id: detect-secrets
  args: ['--baseline', '.secrets.baseline']
```

7.3 What happens on commit

- git commit triggers pre-commit hooks automatically
- trailing-whitespace: removes trailing spaces

- end-of-file-fixer: ensures files end with newline
- detect-private-key: blocks commits containing API keys or tokens
- check-added-large-files: blocks files > 500 KB (prevents accidental data commits)
- black: auto-formats Python code to 88-char lines
- ruff: lints and auto-fixes common errors
- If any hook fails: commit aborted, fix the issues, re-commit

8. VS Code Setup for QuantIQ

QuantIQ uses VS Code (not VSCodium — project standard). Recommended extensions and workspace settings.

8.1 Recommended extensions

Extension	ID	Purpose
Python	ms-python.python	Python language support, IntelliSense
Black Formatter	ms-python.black-formatter	Auto-format on save
Ruff	charliermarsh.ruff	Linting and auto-fix
Pylance	ms-python.vscode-pylance	Type checking, fast autocomplete
GitLens	eamodio.gitlens	Enhanced git blame, history
GitHub Copilot	github.copilot	AI code completion (optional)
Jupyter	ms-toolsai.jupyter	Notebook support for research
TOML	tamasfe.even-better-toml	pyproject.toml syntax highlighting
dotenv	mikestead.dotenv	.env file syntax highlighting

8.2 .vscode/settings.json (commit to repo)

```
{
  "python.defaultInterpreterPath": "../.venv/bin/python",
  "editor.formatOnSave": true,
  "editor.rulers": [88],
  "[python]": {
    "editor.defaultFormatter": "ms-python.black-formatter",
    "editor.formatOnSave": true
  },
  "ruff.fixAll": true,
  "ruff.organizeImports": true,
  "files.exclude": {
    "**/__pycache__": true,
    "**/*.pyc": true,
    ".venv": true
  },
  "python.testing.pytestEnabled": true,
  "python.testing.pytestArgs": ["tests"]
}
```

8.3 Running and debugging

```
# Run Streamlit dashboard
streamlit run dashboard/app.py

# Run bot script
python bot/main.py

# Run tests
pytest tests/ -v

# VS Code launch.json for bot (F5 debug)
# .vscode/launch.json:
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Run Bot",
      "type": "python",
      "request": "launch",
      "program": "${workspaceFolder}/bot/main.py",
```

```
"envFile": "${workspaceFolder}/.env"
},
{
  "name": "Run Dashboard",
  "type": "python",
  "request": "launch",
  "module": "streamlit",
  "args": ["run", "dashboard/app.py"]
}
]
```


9. QuantIQ .gitignore (Complete)

Complete .gitignore for the Quant-IQ/quantiq repository. Copy-paste to project root and never modify without team discussion.

```
# =====
# QuantIQ .gitignore
# github.com/Quant-IQ/quantiq
# =====

# === SECRETS (NEVER COMMIT) ===
.env
.env.*
!.env.example
.streamlit/secrets.toml
.secrets.baseline

# === PYTHON ===
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
*.egg-info/
dist/
build/
*.egg
.eggs/

# === VIRTUAL ENVIRONMENTS ===
.venv/
venv/
env/
ENV/

# === LOGS (audit logs stored on server, not in repo) ===
logs/
*.log
*.log.*

# === DATA FILES ===
data/raw/
data/processed/
*.csv
*.parquet
*.h5
# Exception: keep sample/test data
!data/test/*.csv

# === JUPYTER ===
.ipynb_checkpoints/
*.ipynb

# === TESTING ===
.pytest_cache/
.coverage
htmlcov/
.tox/

# === TOOLS ===
.mypy_cache/
.ruff_cache/

# === IDE ===
.vscode/*.json
!.vscode/settings.json
!.vscode/launch.json
.idea/
```

```
*.swp
*.sw0
*~

# === OS ===
.DS_Store
.DS_Store?
._*
.Spotlight-V100
.Trashes
ehthumbs.db
Thumbs.db

# === VECTORBT ===
vbt_data/

# === DEPLOYMENT ===
.streamlit/secrets.toml
# Note: .streamlit/config.toml IS committed
```

10. See Also

Resource	URL
python-dotenv GitHub/docs	github.com/theskumar/python-dotenv
python-dotenv PyPI	pypi.org/project/python-dotenv/
Python venv docs	docs.python.org/3/library/venv.html
pip user guide	pip.pypa.io/en/stable/
pip version specifiers	packaging.python.org/en/latest/specifications/version-specifiers/
pyproject.toml guide	packaging.python.org/en/latest/guides/writing-pyproject-toml/
PEP 517 (build system)	peps.python.org/pep-0517/
PEP 518 (build deps)	peps.python.org/pep-0518/
Black formatter	github.com/psf/black
Ruff linter	github.com/astral-sh/ruff
pre-commit framework	pre-commit.com
mypy type checker	mypy-lang.org/
VS Code Python extension	marketplace.visualstudio.com/items?itemName=ms-python.python

Cross-references to other binder modules

Module	Relevance
M5 — Streamlit	secrets.toml vs .env (§2.5); .streamlit/ must be in .gitignore
M7 — Python Standards	Black and Ruff configured here (§6.1); pre-commit runs them (§7)
M8 — Version Control	.gitignore here is definitive (§9); pyproject.toml has version string
M9 — Regulatory	logs/ in .gitignore but stored securely on server for 5-year retention
M1 — DhanHQ	DHAN_CLIENT_ID and DHAN_ACCESS_TOKEN stored in .env locally

End of Module 10.

QuantIQ Master Reference Binder — Complete.

All 11 modules (M0–M10) cover the full technical and regulatory surface of the QuantIQ algorithmic trading project. Reprint individual modules when a library has a major version update or when SEBI/NSE issue new circulars. See M0 §8 for versioning and reprint guidance.

Module	Title	Pages
M0	Master Index & Usage Guide	10
M1	DhanHQ v2 — Broker API	35
M2	Python Data Stack (pandas, numpy, yfinance)	20
M3	Technical Analysis — ta library	20
M4	Backtesting — vectorbt	20
M5	Dashboard — Streamlit	21
M6	Charting — Plotly Python	21

Module	Title	Pages
M7	Python Standards (PEP 8/257/484, Google Style, logging)	18
M8	Version Control & DevOps	15
M9	Regulatory — SEBI & NSE	13
M10	Environment & Tooling	~15
	TOTAL	~208

github.com/Quant-IQ/quantiq | quant-iq.github.io/quantiq | quantiq.team@quant-iq.net